

SAPIENZA
UNIVERSITÀ DI ROMA

Enhancing Real-Time Point-Cloud Streaming through In-Network Computing

Ingegneria Informatica, Automatica e Gestionale "Antonio Ruberti"
Master's Degree in Engineering in Computer Science

Leonardo Chiarparin
ID number 2016363

Advisor
Prof. Marco Polverini

Academic Year 2025 / 2026

To Mum and Dad, who with love, sacrifice and tireless devotion have walked beside me at every step of my life, sustaining me through the hardest moments and instilling in me the strength to keep believing in my dreams.

To my grandparents, whose affection and closeness have always understood and inspired me, even in the least considered circumstances of my childhood, enriching my days with lessons and deeply held values.

To my cousins, whose company and example have long been a precious point of reference, sharing singular experiences and building together the extraordinary memories I shall treasure for ever.

To my aunts and uncles, whose warmth and generosity have never let me want for anything, making each shared occasion more joyful and contributing indispensably to the person I have become.

To the people who care for me and whose trust has moved me always to give my utmost, and in particular to my friends and to everyone who has offered me their help, even if only for an instant, showing how every gesture, however modest, can make the difference and turn the ordinary into something special.

To those who are no longer with us, but whose memory lives on in my heart and lights my way without ceasing.

With gratitude and appreciation, I dedicate this achievement to all of you.

A mamma e papà, che con amore, sacrificio e instancabile dedizione mi hanno accompagnato in ogni passo della mia vita, sostenendomi nei momenti più difficili e infondendomi la forza di credere sempre nei miei sogni.

Ai miei nonni, che con affetto e vicinanza mi hanno sempre compreso e ispirato, anche nelle circostanze meno ponderate della mia infanzia, arricchendo le mie giornate con insegnamenti e valori profondi.

Ai miei cugini, che con la loro compagnia e il loro esempio sono sempre stati un punto di riferimento prezioso, condividendo esperienze uniche e costruendo insieme ricordi straordinari che custodirò in eterno.

Ai miei zii, che con il loro calore e la loro generosità non mi hanno mai fatto mancare nulla, rendendo ogni occasione condivisa più gioiosa e contribuendo in modo indispensabile alla mia crescita.

Alle persone che mi vogliono bene e che, con la loro fiducia, mi hanno motivato a dare sempre il massimo, in particolare ai miei amici e a chiunque mi abbia offerto il proprio aiuto, anche solo per un istante, dimostrando come ogni gesto, per quanto modesto, possa fare la differenza e trasformare l'ordinario in qualcosa di speciale.

A coloro che non sono più tra noi, ma la cui memoria vive nel mio cuore e illumina costantemente il mio cammino.

Con gratitudine e riconoscenza, dedico questo traguardo a tutti voi.

Abstract

The spread of immersive services is moving audiovisual delivery towards volumetric representations, in which dense and irregular **Point Clouds** must be acquired, compressed, transmitted, and rendered under stringent latency constraints. The reference architecture considered in this thesis bundles heterogeneous duties within coarse application boundaries, which restrict placement flexibility and confine the network to a transport role. Building on **In-Network Computing** (**INC**), the current study examines which functions of such a system can be resolved while traffic is in transit. The starting point is a live workflow inspired by **Video-based Point Cloud Compression** (**V-PCC**), which projects spatial samples onto images compressed by hardware engines for **High Efficiency Video Coding** (**HEVC**), also known as **H.265**. The design is first rebuilt on the **Data Plane Development Kit** (**DPDK**) as seven independent units that can be observed and deployed in isolation. Part of the logic is then moved onto the forwarding path according to a **data-dependency criterion**, which separates packet-by-packet reductions, quantities defined only by a complete frame, and operations bound to specialised hardware. An intermediary node thus accumulates point-level statistics, resolves the centroid-dependent maximum radius at the end of each frame, and exports the projection metadata that the encoding stage would otherwise recompute. A **runtime characterisation** on 300 frames at 30 fps of the **Loot** sequence from the **8i Voxelized Full Bodies** (**8iVFB**) dataset is kept separate from a capture-enabled **quality assessment**, based on **Mean Squared Error** (**MSE**), **Peak Signal-to-Noise Ratio** (**PSNR**), and **Structural Similarity Index Measure** (**SSIM**) on the geometry **atlas** and on full-cloud distances after alignment. Across three baseline runs, the median source-to-terminal latency settles at 216.104 **ms**, against 337.599 ms for the corresponding **Graphics Processing Unit** (**GPU**) reference implementation, a reduction of about 36 % attributable to the redesign as a whole. A paired **ablation** then isolates the **offload**. Returning the computation to the **Encoder** raises its median active-processing time from 17.577 ms to 46.821 and 25.601 ms over two repetitions, while the valid points are preserved frame by frame and the aggregate geometry-**luma** indicators differ only marginally. The main benefit therefore lies in recovered processing **headroom** rather than in a proportional end-to-end gain. Although confined to a single sequence, the thesis offers evidence on the opportunities and limits of moving selected tasks closer to the **Data Plane** of a volumetric stack in real time.

Contents

1 Introduction **1**
1.1 Context and Motivation 1
1.2 From XR to Point Cloud Streaming 3
1.3 Limits of Real-Time Volumetric Workflows 5
1.4 PDPs and In-Network Computing 7
1.4.1 DPDK as a Software Data Plane 8
1.5 Study Approach and Objective 9
1.5.1 Geometric Offload 11
1.5.2 Experimental Campaign 11
1.6 Research Questions and Contributions 13
1.7 Thesis Structure 15

2 Background **18**
2.1 Foundations of Point Cloud Streaming 18
2.1.1 Volumetric Representations 18
2.1.2 The 8iVFB Dataset 19
2.1.3 MPEG Point Cloud Compression 20
2.1.4 High Efficiency Video Coding 21
2.1.5 Hardware-Accelerated Processing 22
2.1.6 Transfer of Volumetric Content 23
2.1.7 Quality Assessment 24
2.2 Computation Bound to Packet Traversal 25
2.2.1 Origins and Definition 25
2.2.2 Motivations and Expected Benefits 26
2.2.3 Representative Systems 27
2.2.4 Constraints of the Execution Substrate 28
2.2.5 Placement within the Compute Continuum 29
2.2.6 Implications for Volumetric Pipelines 29
2.3 Data Plane Programmability 30

2.3.1 Programmable Switching Pipelines 30
2.3.2 Hardware and Software Targets 31
2.3.3 The Data Plane Development Kit 32
2.3.4 Service Function Chaining and the NSH 33

3 Reference Pipeline 35
3.1 Architecture and Interfaces 35
3.2 Acquisition and Isochronous Injection 36
3.2.1 Pacing and the Synchronisation Header 37
3.3 Temporal Selection at the Flow Controller 37
3.4 Encoder: from Geometry to Video 38
3.4.1 Ingestion and Object-Centred Framing 39
3.4.2 Fitting the Projection Volume 40
3.4.3 Orthogonal Views and Depth Arbitration 41
3.4.4 The Three Map Layers 41
3.4.5 Atlas and Super-Frame Assembly 43
3.4.6 Coding and Dispatch 45
3.5 Reconstruction and Control at the Client 45
3.5.1 Arrival and Decoding 46
3.5.2 Unpacking the Composed Picture 46
3.5.3 Occupancy Filtering 47
3.5.4 Inverse Projection and Restitution 48
3.5.5 Interaction and Display 49
3.6 Instrumentation and Metrics 49
3.6.1 Quantities Valid Across the Chain 49
3.6.2 Measurements Bound to a Single Role 50
3.6.3 Fusion and Reading of the Records 50

4 Methodology 51
4.1 Anatomy of the Redistributed Graph 51
4.1.1 Placement Units and Their Adjacencies 51
4.1.2 State Carried Along the Path 54
4.1.3 Protocol Stack and Datagram Budget 55
4.1.4 Selecting the Site of the Geometric Frontier 57
4.2 The Source and Its Absolute Clock 58
4.2.1 Residency of the Prepared Material 58
4.2.2 Deadlines and the Admitted Rate 59
4.2.3 Serialisation and Cutting of the Frame 61
4.2.4 Acceptance Pressure at the Local Ring 61

4.2.5 Closing the Sequence 62
4.3 Geometry Resolved While Forwarding 62
4.3.1 Validation of a Source Datagram 63
4.3.2 Reduction Maintained Across Packets 64
4.3.3 Resolution at the Completion Barrier 65
4.3.4 Replacement of the Envelope in Place 67
4.3.5 Termination of the Reverse Regulation Path 67
4.3.6 The Node Without Its Computation 68
4.4 The Hub Around the Unaware Functions 68
4.4.1 Ports, Routes, and Payload Semantics 69
4.4.2 Classification by Ingress and Shape 69
4.4.3 Retention of the Index Across a Function 70
4.4.4 Ownership of Buffered Bursts 71
4.5 The Compression Unit as a Hybrid Function 71
4.5.1 Assembly Conducted as Packets Arrive 72
4.5.2 Admission of the Offloaded Frontier 72
4.5.3 The Geometry Without the Offload 73
4.5.4 Device Memory and Cooperative Servicing 73
4.5.5 The Fused Projection Kernel 74
4.5.6 Layout of the Composed Picture 74
4.5.7 A Coding Session That Outlives the Frame 75
4.5.8 Attribution of an Asynchronous Bitstream 76
4.5.9 Regulation Driven by Measured Load 76
4.5.10 Output, Capture, and the Ordered Drain 78
4.6 Rebuilding the Cloud and Serving the Viewer 78
4.6.1 Ingestion and the Bounded Media Queue 79
4.6.2 A Decoding Session Held Open 79
4.6.3 Suppression of Boundary Artefacts 80
4.6.4 Return to Three Dimensions 80
4.6.5 Honouring the Requested Stance 81
4.6.6 Emission and Closure 81
4.7 The Last Aware Boundary 82
4.7.1 Termination of the Forward Route 82
4.7.2 Classification of the Returning Command 83
4.8 The Terminal and Its Presentation Frontier 83
4.8.1 Reassembly and Validation 83
4.8.2 Publication Without a Lock 84
4.8.3 Issuing a Stance and Confirming It 85

4.8.4 The Asynchronous Bridge 85
4.8.5 The Browser Client 85
4.8.6 Capture of the Terminal Cloud 86
4.9 Applications That Surround the Chain 87
4.9.1 Bringing the Host into Condition 87
4.9.2 Withdrawal of Cores from the Scheduler 87
4.9.3 Wiring and Start-Up Order 88
4.9.4 Entry Points and the Build Boundary 89
4.9.5 Offline Preparation of the Sequence 90
4.9.6 Post-Stream Geometric Assessment 90
4.10 What Each Unit Records 91
4.10.1 Anchors Common to Every Record 92
4.10.2 Integrity, Rate, and Local Pressure 93
4.10.3 Admission, Acquisition, and the Service Loop 93
4.10.4 Projection, Coding, and Device Intervals 94
4.10.5 Populations, Stance, and Presentation 94
4.10.6 Offline Columns and the Limits of Addition 95

5 Performance Evaluation 97
5.1 The Questions the Campaign Has to Settle 97
5.1.1 One Independent Change at a Time 97
5.1.2 Runtime and Fidelity Are Not Measured Together 98
5.2 The Host Held Still 98
5.2.1 Processor Affinity and Where Each Unit Runs 98
5.2.2 Removing Seven Processors from the Scheduler 99
5.2.3 Memory Reserved Before Anything Starts 100
5.2.4 What the Build Nails Down 100
5.3 Two Families of Execution 101
5.4 Constants That Specify Each Run 102
5.5 The Conditions Evaluated 103
5.5.1 The Baseline Repeated Three Times 104
5.5.2 Moving the Frontier Back 104
5.5.3 Two Captures for One Statement 105
5.5.4 Bounds on the Source 106
5.5.5 Datagram Rate and Media Grouping 106
5.5.6 A Run Reserved for Looking 107
5.6 Admitting and Reading a Run 108
5.6.1 Checks a Trace Must Pass 108
5.6.2 The Window and the Comparator 108

5.7 What These Measurements Are Not 109
5.7.1 The Condition of the Machine 109
5.7.2 Frontiers That Do Not Compose 110
5.7.3 The Two Levels of Comparison 110
5.8 Repeating the Campaign Elsewhere 111
5.8.1 Bringing the Experiment Up 111
5.8.2 Adapting the Build to Other Hardware 111
5.8.3 The Manifest a Result Carries 112
5.9 What the Next Chapter Reports 113

6 Results 114
6.1 The Traces Admitted to Analysis 114
6.2 The Repeated Baseline 114
6.2.1 Spread of the Unchanged Configuration 115
6.2.2 Cadence Sustained at the Source 116
6.3 The Chain Against the Reference Implementation 117
6.4 The Placement Ablation 117
6.4.1 Where the Computation Is Paid 117
6.4.2 The Margin Recovered at the Compression Unit 119
6.4.3 Residency and the Timing Behind It 120
6.4.4 The Pace Observed Along the Route 121
6.4.5 Progressive Frontiers of a Frame 122
6.4.6 What the Pair Establishes 123
6.5 Fidelity Across the Two Placements 124
6.5.1 Population and Coded Signal 124
6.5.2 Deltas Taken Frame by Frame 126
6.5.3 The Two Alignment Modes 127
6.5.4 The Frame Selected for Inspection 128
6.6 Sensitivity of the Operating Point 130
6.6.1 Packet Capacity and Media Grouping 131
6.6.2 Residency of the Source Material 132
6.6.3 Pressure Imposed Through the Datagram Rate 132
6.6.4 Admission Against the Final Boundary 134
6.7 The Cost of an Interaction 135
6.8 What the Measurements Establish 136

7 Conclusions 138
7.1 What the Redistribution Achieved 138
7.2 Where the Evidence Stops 139

7.3 What Should Be Attempted Next 139

Bibliography **141**

Acknowledgements **148**

List of Figures

1.1 Scope of the processing addressed in this work 3
1.2 Taxonomy of operations by information dependency 6
1.3 Map of the re-engineered platform 10
1.4 Overview of the experimental programme 12

2.1 Explicit representations of a single object 19
2.2 Processing stages of V-PCC 21
2.3 Spatial and temporal organisation in HEVC 22
2.4 Location of computation within the data path 26
2.5 Abstract pipeline of a programmable switch 31
2.6 DPDK components for two linked containers 33
2.7 Format of the NSH 34

3.1 Modules of the reference system 35
3.2 Stages of the Camera 36
3.3 Organisation of the Flow Controller 38
3.4 Processing steps of the Encoder 39
3.5 G-buffer layers of the face that captures the subject 42
3.6 Cross-shaped layout of the six views 43
3.7 Arrangement of the transmitted picture 44
3.8 Stages and feedback pathways of the Client 46
3.9 Occupancy across the border of the subject 47

4.1 Placement graph of the proposed platform 53
4.2 Protocol stacks of the transported forms 56
4.3 One cycle of the source loop 58
4.4 Regimes of the in-path service 63
4.5 Flows and index progression at the transit hub 69
4.6 Structure of the compression unit 72
4.7 States and predicates of the regulator 78

4.8 Stages of the reconstruction unit 79
4.9 Duties of the egress node in both directions 82
4.10 Delivery boundary between the terminal and the browser 83
4.11 Web client during a streaming session 86

6.1 Terminal delay of the repeated and ablated conditions 115
6.2 Source cadence held across the campaign 116
6.3 Active processing along the route 118
6.4 Composition of the work at the compression unit 119
6.5 Residency of a frame at each placement 120
6.6 Phases of the compression unit 121
6.7 Arrival spacing at each boundary 122
6.8 Elapsed time at successive exits 123
6.9 Point population 125
6.10 Coded and reconstructed fidelity 126
6.11 Distribution of the geometry-luma indicators 126
6.12 Alignment forms 128
6.13 Depth atlas on either side of the coder 129
6.14 Appearance and mask across the same frontier 129
6.15 Terminal delay and completeness 131
6.16 Control behaviour based on the datagram rate 133
6.17 Frames complete at every boundary 134

Chapter 1

Introduction

1.1 Context and Motivation

Over the last decade, the demand for engaging and interactive applications has led the multimedia sector, from conferencing and gaming to entertainment, to introduce three-dimensional content into infrastructures originally designed for planar video, marking a significant turning point for the field. **Virtual Reality** (**VR**), **Augmented Reality** (**AR**), **Mixed Reality** (**MR**), and the broader family commonly referred to as **Extended Reality** (**XR**) can no longer be treated as isolated graphics problems. They constitute end-to-end systems in which sensing, geometric preparation, compression, transport, and user control must cooperate along a single delay-sensitive path. Studies on XR over **Beyond-5G** (**B5G**) networks reflect this shift, since they treat latency, **Quality of Experience** (**QoE**), edge capacity, and function placement as joint design variables, and regard communication and computation as complementary facets of one distributed platform rather than as separate layers joined by a fixed interface [16, 60, 62]. **Volumetric media** are among the clearest expressions of this convergence. A conventional video frame arranges its pixels on a predetermined raster and fixes most of the viewing geometry before transmission, whereas a spatial model must retain enough structure for the scene to be explored from positions and orientations unknown at capture time. The rendered image consequently depends on actions taken after the session has started. Delivery becomes tied to the movements of the observer far more closely than in traditional broadcasting, and the viewer gains the **Six Degrees of Freedom** (**6DoF**) that distinguish volumetric playback from fixed-perspective footage. The same latitude makes the medium attractive for telepresence, remote assistance, cultural heritage, and collaborative work, but it also markedly increases both the payload to be conveyed and the calculation required before anything reaches the screen. Among the available formats, **Point Clouds** are central to this thesis. They

describe an object or an environment through a set of samples scattered in space, each carrying its coordinates and, optionally, attributes such as colour or reflectance. Since no surface connectivity is imposed, the representation naturally accommodates irregular shapes and animated subjects, at the cost of the regularity that ordinary images take for granted. A single frame may comprise hundreds of thousands of points, and consecutive frames continuously alter their arrangement and visibility [34, 39, 69]. The processing chain must accordingly absorb not only a heavy bitrate but also operations whose access patterns, parallelism, and memory behaviour differ substantially from those of pixel grids. **Projection-based coding** is one of the most effective means of containing this complexity. Techniques developed within the **Video-based Point Cloud Compression** (**V-PCC**) paradigm map each volumetric frame onto two-dimensional **atlases**, so that mature video tools can be applied to geometry and texture alike. The implementation from which this research departs adopts the same principle and relies on hardware-assisted **High Efficiency Video Coding** (**HEVC**), also known as **H.265**, to keep pace with the incoming stream. It should nonetheless be regarded as a custom projection-and-video chain rather than as a standard-compliant V-PCC or **Geometry-based Point Cloud Compression** (**G-PCC**) codec, and the evidence reported in later chapters characterises that specific realisation. The practical difficulty, however, is not confined to moving an unusually large asset. A live volumetric service is a sequence of tightly coupled activities, namely acquisition, projection, encoding, transfer, decoding, reconstruction, and visualisation, and the final result materialises only once all of them have contributed within the same scheduling window. Optimising a single link in isolation thus offers no guarantee of a comparable benefit for the whole. This consideration carries particular weight under real-time assumptions. At a nominal source rate of 30 fps a new frame is emitted approximately every 33.3 ms, yet this interval is not a strict per-frame deadline. In a pipelined architecture several frames legitimately coexist along the chain, so throughput, node-local service time, queue occupancy, and end-to-end delay remain distinct, albeit correlated, quantities. A modest reduction in a recurrent step may then prove relevant even when the perceived outcome does not change proportionally, for instance when it prevents buffer growth, restores slack, or limits the jitter propagated to subsequent stages. Conversely, a module that appears comfortably fast when examined alone may still destabilise delivery once it must sustain a continuous cadence alongside every other job on the same machine. The solution adopted as a starting point already distributes its duties over several application-level services. Its main limitation, for the present purposes, lies not in a lack of decentralisation but in the **granularity** at which this is expressed. Its **Encoder**- and **Client**-side components bundle responsibilities with

heterogeneous data dependencies and resource affinities into monolithic deployment units. The author of the reference work, **Lacaria**, identifies the decomposition of such blocks into finer, independently placeable elements as the natural continuation of her effort, which directly motivates the present study [38]. The nature of the workload reinforces this argument. Point Cloud operations do not depend on their input uniformly, and the information each of them consumes varies as much as its cost. The location of a task should therefore follow from what it needs to produce a correct result, not merely from the process that happens to host it. From this perspective, the forwarding path itself becomes a candidate **execution site**. When everything a routine requires is already visible while packets traverse an intermediate node, deferring its evaluation until the whole frame has been reassembled downstream is a design choice, not a mathematical necessity. Whether this can be revisited, to what extent, and with which measurable consequences is the question from which this investigation proceeds. Figure 1.1 delimits the boundaries within which that enquiry is posed and subsequently examined in this thesis.

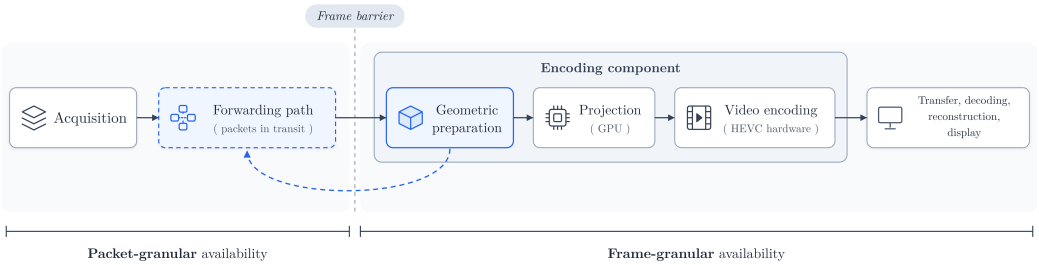


Figure 1.1. Scope of the processing addressed in this work

1.2 From XR to Point Cloud Streaming

Although XR applications differ considerably in headsets, interaction paradigms, and rendering back-ends, they share a common requirement, namely that the entire system must react to changes in the user state or in the environment quickly enough to preserve the perceived continuity of the experience. Alongside the network-oriented view recalled above, the literature on the **compute continuum** models immersive services as graphs of microservices whose components may be hosted on end devices, edge nodes, or cloud resources according to their observed cost. In this strand, the **placement** of a function has become an object of study in its own right, and an architecture unable to expose its internals is difficult to position appropriately. Volumetric material stands out as the most demanding class of content in this landscape and, for the same reason, as the most instructive to analyse. Each phase of its workflow may call for a different memory layout and computing substrate. Raw

samples are irregular and geometry-dependent, projection is data-parallel and maps naturally onto many-core chips, encoded media benefit from dedicated video engines, packet forwarding introduces queueing and contention effects of its own, and the terminal must once again interpret geometry. The choice of representation already entails a compromise. **Polygonal meshes** offer explicit connectivity and integrate smoothly with mature graphics toolchains, but depend on a surface-extraction step that proves fragile for moving and noisy subjects. Conversely, **voxel grids** impose a regular spatial partition that simplifies indexing, at the price of a footprint that grows with resolution. Point Clouds occupy the middle ground, as they preserve the per-sample flexibility of the capture without committing to a predefined tessellation, which explains their prevalence in research prototypes. Publicly available discretised sequences, and in particular the **8i Voxelized Full Bodies** (**8iVFB**) collection, have become a de facto reference for the discipline, since they supply dense full-body recordings at a fixed geometric precision and allow competing solutions to be exercised on an identical source population. Standardisation has followed two complementary directions. V-PCC segments each cloud into patches, packs geometry, attribute, and occupancy information into image planes, and entrusts them to a conventional two-dimensional codec, thereby inheriting decades of investment in consumer silicon. G-PCC operates natively in three dimensions through octree decomposition and predictive schemes, and is better suited to sparser inputs such as **Light Detection and Ranging** (**LiDAR**) scans. Both approaches reduce the payload substantially, but neither removes the preparatory and post-decoding stages surrounding the codec, and their reference implementations were not conceived for the low-delay, always-on regime of real-time operation. Latency-sensitive designs instead retain the projection principle while simplifying the prescribed patch machinery, trading compression efficiency for a steady and hardware-friendly runtime behaviour. The delivery side has been explored just as extensively. **Field of View** (**FoV**) and visibility-aware methods restrict transmission to the regions the observer can actually see, tiling and layered bitstreams enable progressive refinement, delivery frameworks such as **Dynamic Adaptive Streaming over HTTP** (**DASH**) and its low-latency variants govern bitrate selection under fluctuating throughput, and receiver-side enhancement trades bandwidth for local computation. Each of these techniques addresses a genuine overhead, and their cumulative effect is what makes the medium viable outside the laboratory. They nonetheless rest on an implicit boundary, since processing is assumed to occur at the endpoints, while the intervening network is treated as a conduit whose only relevant properties are capacity, delay, and loss. **Hardware acceleration** has proved equally decisive, yet no less bounded. **Graphics Processing Units** (**GPUs**) perform view transformations at remarkable

speed, and fixed-function codec blocks reduce encoding and decoding to a fraction of the cost that general-purpose code would incur. The logic surrounding them nevertheless remains on the critical path. Buffer staging, format conversion, output pacing, statistics derivation, synchronisation, and queue management all lie beyond their reach, so a deployment may combine generous links with powerful co-processors and still fall short of its nominal cadence. Live Point Cloud streaming is, in this sense, as much a matter of where each operation runs as of how each frame is coded. Data models, compression standards, distribution strategies, and the associated quality metrics are examined in Chapter 2, while the next section turns to what this view implies for the design of application-level chains.

1.3 Limits of Real-Time Volumetric Workflows

The obstacle addressed by this thesis is **structural**, not purely computational. Volumetric pipelines comprise activities with sharply diverging dependencies and runtime profiles, yet application-oriented designs routinely gather several of them inside a single deployable block. This organisation is sound from a software-engineering standpoint, but it binds location to the enclosing process rather than to each constituent step, and three recurrent effects follow. The first is the presence of **barriers**. A module typically reassembles an entire frame before starting its next phase. Such behaviour is unavoidable for calculations whose output depends on the whole cloud, but far less justified when the same module also performs summations that could advance while the packets are still arriving. Deferring them merely postpones the moment at which a usable value exists and concentrates that effort downstream. As arrivals are continuous and consumption is episodic, such a checkpoint is also where a **backlog** most naturally accumulates. The second is **heterogeneous resource coupling**. A logical Encoder may simultaneously handle depacketisation, per-cloud geometric statistics, GPU projection, host-to-device transfers, the feeding of the **NVENC** session, and workload feedback, duties with little in common beyond their host. Projection must stay adjacent to the accelerator that runs it, whereas a lightweight stateful aggregation carries no such restriction and could run on any node the traffic crosses. When the two are packaged together, an orchestrator can relocate only the component as a whole, though the measured pressure concerns a small fraction of its load. The third is **bottleneck migration**. Accelerating a compressor or a compute kernel frequently exposes costs that were previously secondary, so the constraint shifts to another stage rather than disappearing. The benefit of any local improvement can therefore only be judged on the chain that absorbs it. In the reference architecture, these effects converge on the Encoder and the Client, which

combine processor-, accelerator-, codec-, and network-specific functions behind a single container boundary. Exposing such seams is a precondition for any placement experiment, since a function whose execution site cannot be described on its own can be neither moved nor assessed afterwards. Not all operations, however, obey the same criterion. Depending on the data each task draws on, they fall into at least three classes. The first comprises **packet-progressive reductions**, whose state can be refreshed whenever further points arrive, as for coordinate sums, component-wise minima and maxima, and active-point counts, all of which compose incrementally, with packet order explicitly fixed for reproducibility. The second comprises **frame-completing quantities**, which cannot be resolved before the last sample has been seen, since they refer to a parameter that the frame itself defines. The canonical example is the **maximum radius** about the definitive **centroid**, since every distance is measured from a centre that remains unknown while the population is incomplete. The third comprises **frame-global and hardware-bound operations**, which require a fully materialised representation, dedicated silicon, or both, as for pose transformation, multi-view projection, visibility and depth-conflict resolution, atlas packing, video encoding and decoding, and final reconstruction. A frame-completing quantity, although unavailable before the barrier, need not be produced where it is consumed, and that nuance is decisive here. Figure 1.2 illustrates the classification, grounded in information dependency rather than in an arbitrary separation between network and application code. It also clarifies what processing on the **Data Plane** means. The expression might suggest that every displaced routine is invoked per datagram, whereas the reading adopted here is deliberately narrower. Incremental refinement proceeds during traversal wherever the data flow permits, while values that depend on every point are finalised at the barrier, ahead of the stage that would otherwise recompute them from the assembled frame.

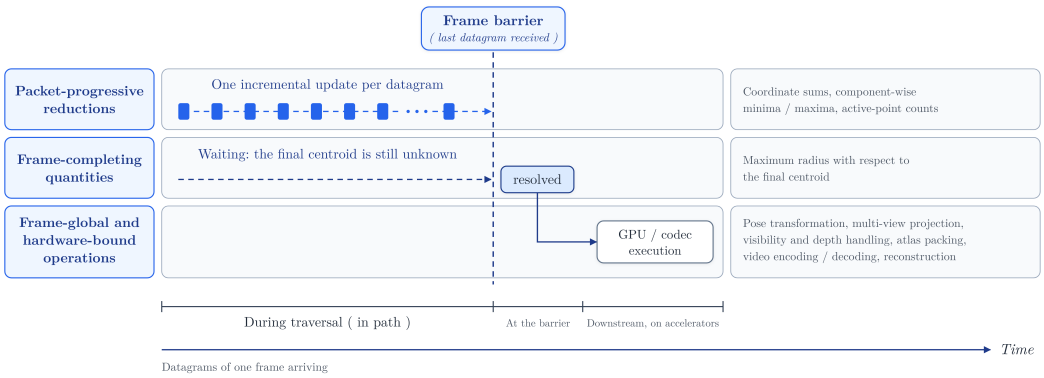


Figure 1.2. Taxonomy of operations by information dependency

An additional condition follows from this classification and constrains the experimental setup throughout. Moving a procedure is meaningful only if its former owner actually abandons it. Evaluating the same geometry at an intermediate node while the Encoder keeps deriving it independently would inflate the total work and prove nothing about where it should run. The upstream result must therefore replace the local one whenever it is valid, and the corresponding control run must return that responsibility to the Encoder while removing it from the forwarding path. Thus understood, an **offload** is a handover, never a duplication.

1.4 PDPs and In-Network Computing

Conventional communication architectures draw a sharp line between forwarding and computation, since bits travel from one endpoint to another, while the interpretation of their contents takes place before they are sent or after they are delivered. The advent of **Programmable Data Planes** (**PDPs**) has progressively eroded this separation, turning packet parsing, forwarding behaviour, header manipulation, and a bounded set of stateful actions into properties defined by the operator rather than fixed by the vendor. **Domain-specific languages** (**DSLs**) such as **Programming Protocol-Independent Packet Processors** (**P4**), together with programmable switching **Application-Specific Integrated Circuits** (**ASICs**), **Smart Network Interface Cards** (**SmartNICs**), **Field-Programmable Gate Arrays** (**FPGAs**), and high-performance software frameworks, have turned the transit path into a substrate that can be programmed, not merely configured. **In-Network Computing** (**INC**) builds on this capability by treating intermediate elements as legitimate hosts for carefully selected tasks. Its aim is not to replace edge or cloud resources with switches, but to bind a calculation to traversal whenever the necessary inputs are visible in flight and early intervention can reduce the bytes moved, merge partial results, shorten a dependency chain, or relieve a downstream stage. The idea has gained steady momentum over the past decade, to the point that the **Internet Research Task Force** has hosted a research group devoted to **Computing in the Network** [37]. The literature now documents it across markedly different fields. **SwitchML** aggregates model updates inside the switch during distributed training, **NetCache** serves frequently requested keys directly from the fabric, **In-band Network Telemetry** (**INT**) lets in-flight packets record per-hop metrics, and **SilkRoad** achieves stateful layer-4 load balancing on commodity ASICs. Surveys of the field converge on a conclusion that also governs this thesis, namely that the benefit stems from matching an operation to its execution environment, not from relocation as such, and that existing platforms differ substantially in expressiveness,

state capacity, numeric support, and line-rate throughput. SwitchML illustrates the point, since the absence of floating-point units in switching pipelines forces it to convert its operands into fixed-point integers before summation. The same reasoning transfers naturally to volumetric streaming. The per-frame statistics that the Encoder derives before projection, namely sums, extrema, and point counts, are reductions over a flow of datagrams, structurally analogous to what SwitchML performs on gradients. A node crossed by every sample of a frame can therefore obtain them while relaying and attach the outcome to the traffic, relieving the component that handles the reassembled cloud of the entire burden. Before asking whether such a step could eventually be mapped onto a constrained hardware target, however, two prerequisites must be met. The task has to be semantically compatible with en-route evaluation, and changing its location has to produce an observable effect on the system. A software Data Plane suits this purpose well, as it retains general-purpose flexibility, including floating-point arithmetic, memory of non-trivial size, and common libraries, while exposing input and output far more transparently than standard sockets, which hide queues and headers from view.

1.4.1 DPDK as a Software Data Plane

The **Data Plane Development Kit** (**DPDK**) is the toolkit adopted in this work [13]. It grants **user-space** access to network ports and their queues, whether they belong to physical adapters or to virtual devices, so that application code fetches, inspects, modifies, and emits traffic while managing descriptor lifetimes and ring usage directly. **Poll Mode Drivers** (**PMDs**) remove the interrupt-driven kernel path from ordinary I / O, leaving the processing loop and its cost open to measurement by the implementation itself. These traits matter for the present investigation in a specific way. An in-path node must retain per-frame context, accumulate geometric evidence over many datagrams, detect when a frame is complete, append new metadata to what it relays, and remain accountable for its own queue occupancy throughout. Distinguishing active processing time from ring pressure and transmit **backpressure**, a distinction on which the experimental argument rests, is far more tractable when every receive and send call is issued explicitly than when scheduling and buffering sit behind generic abstractions. This choice does not imply that kernel networking is inadequate, as mature protocol stacks, isolation mechanisms, and portable interfaces keep it preferable for most workloads. DPDK is employed because packet visibility and function placement are the very variables under study. Beyond its performance characteristics, it also plays a methodological role, providing a controllable setting in which communication and selected routines run within a single observable scope. For a Point Cloud service,

the practical significance of this instrument lies in the shift from frame- to packet-granular processing. In an application-level design, information arrives frame by frame, and every consumer inherits that granularity whether or not its mathematics require it. Once logic can run on the forwarding path itself, packet-progressive reductions advance at every arrival, and the geometric metadata that the Encoder requires before projection is already available when the last datagram of a frame leaves the node. The effort is not eliminated but performed earlier, elsewhere, and largely over the stream rather than over a reassembled frame. The component that would otherwise have borne it recovers the corresponding **headroom**, and the chain gains a degree of freedom, in the choice of where a calculation runs, that a dense deployment boundary had previously suppressed.

1.5 Study Approach and Objective

The aim of this thesis is to determine which responsibilities of a real-time Point Cloud streaming pipeline can be transferred from coarse application components to an explicit packet-processing path, and to quantify how the transfer affects per-component workload, end-to-end stability, terminal latency, and reconstruction fidelity. The intention is neither to migrate the volumetric workflow into network devices nor to dispense with the compute and codec resources that the remaining stages require. The work instead singles out the operations whose dependencies allow an earlier evaluation, obtains the corresponding values while traffic progresses through the graph, and lets downstream units use them rather than derive them locally. Meeting it calls for an instrumented testbed, and the investigation unfolds in two phases that must be kept analytically distinct. In the first, the starting system is re-engineered into a finer-grained architecture built on DPDK, which exposes forwarding and computation boundaries and disentangles concerns previously co-located. In the second, the geometric frontier is shifted within that structure, so that its influence can be assessed apart from the broader redesign. This distinction underpins the validity of the results. Contrasting the legacy deployment with the packet-oriented one characterises the transition as a whole, in which language, transport, decomposition, control ownership, and codec handling vary at once. Only alternative placements within the same platform allow an effect to be attributed to location. The nodes that interpret the additional context were initially modelled on the forwarder role of the **Service Function Chaining** (**SFC**) architecture [22]. That label, however, understates what they do, since each combines classification, relaying, state retention, and envelope manipulation, and one also carries out application-domain computation. They are instead named after their position along the route, independently of any

chaining vocabulary, as the **Ingress Processing Node** (**IPN**), the **Transit Processing Node** (**TPN**), and the **Egress Processing Node** (**EPN**). The notions borrowed from that framework, namely an explicit path identifier and a decreasing service index carried in an in-band context header modelled on the **Network Service Header** (**NSH**), serve a descriptive purpose and are not offered as an interoperable implementation of either specification [53]. Figure 1.3 depicts the resulting topology, which decomposes the workflow into seven placement units against the four dense services of the reference. A **Camera** injects packetised Point Cloud frames on a fixed absolute timeline, and the IPN hosts the geometry-aware processing under examination. The TPN is the hub through which traffic enters and leaves the Encoder and the **Decoder**. Since both are deliberately unaware of the service path, the TPN strips the context envelope before delivery and restores it afterwards. The Encoder assembles complete frames, consumes the upstream geometry when it is valid, runs a six-view GPU projection, and feeds a persistent NVENC session. The Decoder performs **NVDEC** decoding and rebuilds the cloud, the EPN terminates the forward route, and the **User** closes the delivery and issues the viewing commands. Its components are written chiefly in C, C++, and the **Compute Unified Device Architecture** (**CUDA**), while hardware coding is delegated to **FFmpeg** and the **NVIDIA Video Codec SDK**. Each unit runs in its own container, attached to its neighbours through direct **virtio-** and **vhost-user** links, with no intermediate switching tier. To shield the timing measurements from extraneous activity, container **Central Processing Unit** (**CPU**) sets pin every unit to specific logical cores, one for each polling loop. With the sole exception of the User, which shares core 0 with operating-system housekeeping, they are withdrawn from the general-purpose kernel scheduler through the **isolcpus** boot parameter, so ordinary load balancing skips them. The Camera is launched last, once every unit along the route has been initialised and stands ready.

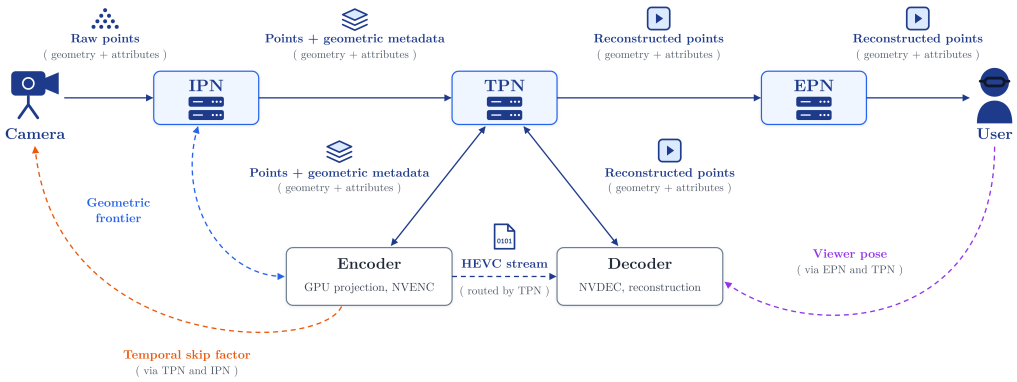


Figure 1.3. Map of the re-engineered platform

Three boundaries show what the decomposition achieves, each a responsibility the reference kept in one block. First, the original Client handled media decoding, volumetric reconstruction, interaction, and presentation. The Decoder now takes both of those and the User the rest, so that a stage bound to the video hardware and one bound to the display no longer share a unit. Second, ownership of the **Temporal** mechanism changes. Regulation was previously decided by the terminal and enforced by a **Flow Controller** after the Camera, so a discarded frame had already been sent. The Encoder now derives the skip factor from its load model, and the Camera applies it before packetisation, sparing a suppressed frame both steps. Third, the **Pose** path runs the other way. Instead of travelling upstream through projection, encoding, transport, and decoding, it stops at the Decoder, which applies the latest orientation and leaves the source untouched. The two reverse loops are independent, one driven by workload, the other by the viewer. The payload itself also changes along the chain, from raw samples to samples enriched with metadata, then to compressed media and rebuilt clouds, so the site of a computation rests not on throughput alone but on the form the data takes at each hop.

1.5.1 Geometric Offload

The functions selected for relocation must satisfy three conditions. Their input must be present in, or derivable from, the datagrams the node observes. Their intermediate state must be maintainable incrementally without altering the final quantity. Their output must be usable downstream, so that the native computation can be suppressed. Accordingly, the IPN accumulates coordinate sums, component-wise extrema, and the active-point count, then derives the centroid, the axis-aligned **extent**, and the **bounding-box centre** once the population is known. At the barrier, it resolves the maximum radius around that mean point through a second pass over the coordinates retained in transit, and produces the scaling factors and bounding descriptors the projection kernel consumes. Everything beyond that boundary stays where its dependencies place it, including the conversion of the points into the layout of the Encoder, the pose-independent projection, visibility and depth handling, atlas preparation, and coding. When the offload is disabled, the IPN still classifies and relays traffic but omits these scans, and the Encoder recomputes the same sequence before its unchanged CUDA branch. The paired runs gauge the headroom released under an identical topology.

1.5.2 Experimental Campaign

Chapter 5 details the evaluation protocol, and only its essentials appear here. The source material is the **Loot** sequence of the 8iVFB collection, converted offline into

a fixed-width binary format so that acquisition cost does not depend on parsing overhead. A finite, reproducible input lets the same population be replayed across configurations, removing a major source of ambiguity when latency, per-node load, packet dynamics, and visual quality are compared. The nominal configuration consists of 300 frames emitted at 30 fps. The workload-driven Temporal loop stays active throughout the repeated baseline and the fidelity controls, yet the admitted skip factor holds at unity, because the Encoder never crosses its overload threshold. All 238,146,391 source points are delivered in each such execution. Frame admission is thus a property of the running system, not a convenience. One stress condition drives the regulator beyond that threshold and records complete escalation and recovery cycles, while a diagnostic variant switches the mechanism off to expose packet pressure without source throttling. As Figure 1.4 summarises, the campaign separates two concerns. **Runtime characterisation** exercises the ordinary streaming path and records cadence, packet and frame completion, node-local processing time, queue interaction, backpressure, Encoder workload, and Camera-to-User latency. The baseline is replicated so that run-to-run variation can be told from the impact of a modified setting. **Quality assessment**, by contrast, relies on capture-enabled runs, which keep diagnostic work out of the real-time budget. The fidelity of the geometry atlas is evaluated on its luma component through the **Mean Squared Error** (**MSE**), **Peak Signal-to-Noise Ratio** (**PSNR**), and the **Structural Similarity Index Measure** (**SSIM**), while reconstructed Point Clouds are set against the ground truth by spatial distances measured over the complete population of every frame that the run has delivered to the terminal [25, 67, 70].

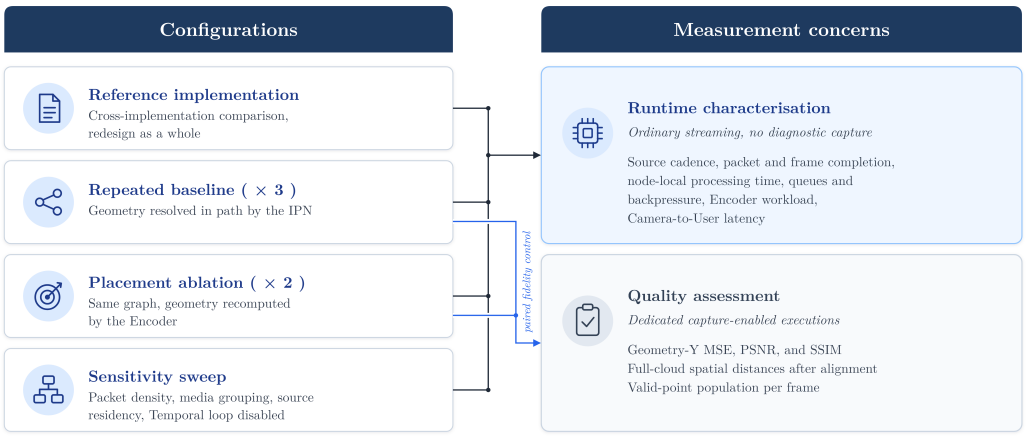


Figure 1.4. Overview of the experimental programme

Three kinds of evidence follow. Repeated baseline executions establish the stability of the nominal service. A paired **ablation** returns the geometric frontier to the

Encoder while keeping the rest of the graph intact, and constitutes the principal placement check. Sensitivity experiments alter individual factors, such as packet density, source-side pacing, or skip regulation, to reveal how overload surfaces and whether the feedback loop reacts before instability propagates. The nominal operating point already outlines the shape of the findings. Across three full baseline runs, the all-frame median Camera-to-User delay settles at 216.104 **ms**, whereas the reference implementation reaches 337.599 ms at the comparable reception point. The resulting reduction of about 36 % characterises the redesign as a whole and cannot be credited to in-network geometry alone. The ablation isolates the offload. With the frontier resolved in path, the first baseline run reports a median Encoder active-processing time of 17.577 ms and a median workload ratio of 0.163, with the other two runs remaining within 17.395 to 17.740 ms and 0.162 to 0.164. The two ablation repetitions raise these values to 46.821 and 25.601 ms and to 0.419 and 0.386, respectively. Terminal latency does not track this change proportionally, which is consistent with a pipelined service in which scheduling phase, stage overlap, and persistent codec behaviour all contribute. The reproducible outcome is therefore a loss of Encoder headroom when the offload is withdrawn, not a deterministic end-to-end gain when it is applied. The paired fidelity executions complete the picture, as both placements preserve the valid-point population of all 300 frames and the aggregate indicators remain effectively unchanged whichever site hosts the computation. The sensitivity experiments, in turn, indicate that the service reacts far more strongly to source-side residency than to packetisation parameters. Setting the payload to 84 points per datagram or enlarging the media grouping keeps the median delay within about 3 ms of the baseline, whereas reading the sequence from storage without prior mapping raises the median disk access time of the Camera from about 3.5 to 115.6 ms and roughly doubles the delay at the terminal, to about 449 ms. Halving the density to 40 points per datagram yields twice as many packets per frame, and with the Temporal loop active the regulator completes three escalation and recovery cycles, admitting 264 of the 300 frames. Its unregulated counterpart admits all the frames, but the packets that the Camera must resubmit under transmit backpressure grow from about 47.1 to 68.0 million over the run, a pressure that the Temporal loop otherwise absorbs by thinning the source.

1.6 Research Questions and Contributions

The investigation centres on the relationship between information dependencies, execution site, and observable pipeline behaviour. In-network computation is not presumed to be intrinsically advantageous. The thesis asks instead which functions

can leave their original host without violating their mathematical requirements, whether their displacement actually relieves the receiving component, and how the resulting system behaves under nominal and stressed conditions. Four research questions, listed below, organise the study and its evidence.

RQ1 Which operations admit a progressive or frame-completing evaluation in transit, and yield values that a downstream stage can reuse?

RQ2 How does moving the geometric frontier from the Encoder to an in-path node influence encoding load, processing headroom, queue dynamics, and Camera-to-User latency within an unchanged topology?

RQ3 Can the service graph sustain the nominal rate of 30 fps while preserving packet and frame integrity, and does overload surface as quantifiable backpressure and source adaptation rather than as uncontrolled frame loss?

RQ4 Does the decoded stream retain its fidelity when the frontier changes location, as judged by signal metrics on the geometry atlas and spatial distances on the rebuilt clouds within the adopted test setting?

Answering these questions yields six principal contributions.

1. A packet-oriented re-engineering of the reference workflow, in which generation, en-route computation, coding, reconstruction, and delivery become individually inspectable duties linked through direct user-space adjacencies.

2. A decomposition of coarse deployment blocks into seven placement units, which preserves end-to-end semantics while reassigning the Temporal and Pose control loops to the Encoder and the Decoder, respectively.

3. A dependency-based criterion for selective offload, which separates packet-progressive reductions and frame-completing quantities from tasks tied to complete representations or specialised hardware.

4. A geometry-aware transit module that accumulates coordinate statistics during traversal, resolves the centroid-dependent terms once every point has arrived, and exports the metadata for direct use by the Encoder.

5. An internal placement ablation in which a single responsibility alternates between the IPN and the Encoder without redundant computation while everything else is held fixed, so that the recorded effects can be attributed to its location rather than to the broader redesign.

6. An empirical validation that combines repeated runtime measurements, sensitivity tests driven towards saturation, completion indicators, workload analysis, terminal delay, and a separate objective quality campaign.

The work is architectural and experimental in nature. It does not propose a new compression scheme, nor a placement algorithm for the compute continuum, since the location of every unit is fixed and the question concerns the internal decomposition of a single function. The relocation it studies is, moreover, deliberately confined to the geometry that the traffic already exposes in transit. What it offers is a practical boundary at which part of that description can be derived earlier, carried alongside the stream, and withdrawn from the consuming application, while the rest of the chain continues to run on the resources best suited to it. The chapters that follow develop this argument, examining the legacy design, deriving the candidates for relocation, describing the platform around them, and assessing its responsiveness and accuracy against the four questions above.

1.7 Thesis Structure

The document is organised into several chapters, each devoted to a specific facet of the work and building on the results of the previous one.

1. **Introduction**: constructs the scientific and technical setting of the investigation, tracing the shift from planar media to volumetric services, the structural shortcomings of application-level pipelines whose functions are deployed as indivisible blocks, and the opportunity offered by a computationally active forwarding path. Particular emphasis is placed on the dependency criterion that separates what can be resolved in transit from what requires a complete frame or a dedicated accelerator, before the research questions, the contributions, and the outline of the thesis are stated in turn;

2. **Background**: reviews the foundations on which the proposed system rests, along three strands. It opens with Point Cloud streaming, covering three-dimensional representations and their trade-offs, the datasets adopted by the community as shared benchmarks, the standardised compression approaches together with the projection principle underlying the reference chain, the constraints of delivery, and the objective metrics used to assess reconstructed content. It then turns to In-Network Computing, discussing the motivations for binding computation to packet traversal, the domains in which this pairing has proved effective, and the conditions under which it has not. Finally, it introduces Data Plane programmability and the DPDK concepts underpinning

the software packet-processing environment employed here, together with the chaining abstractions used to describe the state of the path;

3. **Reference Pipeline**: reconstructs the system from which this study departs, module by module and in the order in which a frame traverses it, so that the comparison drawn later rests on a documented baseline rather than on a summary. It covers the acquisition that replays a stored sequence on an absolute clock, the temporal selection that decides which frames continue towards compression, the Encoder that turns each cloud into an ordinary picture through centring, orthogonal projection, visibility arbitration, and atlas assembly, before a video coder and a custom packetisation carry it onto the link, and the terminal that reverses those steps, filters what quantisation has blurred, and returns the commands of the viewer upstream. It closes with the instrumentation shared by the four modules and with the table against which the measurements of later chapters are compared;

4. **Methodology**: sets out the platform built for this study, unit by unit and in the order a frame meets them. It opens with the shape of the redistributed graph, with the contract every datagram carries, and with the compile-time switch that decides where the geometric frontier is settled. It then follows the source and its absolute schedule, the ingress node that reduces the samples while they travel and settles that frontier once the population is known, the hub that retains path state around the two functions which ignore it, the compression unit with its accelerator pipeline, its persistent coder and its load-driven regulator, the reconstruction unit that rebuilds the cloud and applies the requested stance, the egress boundary, and the terminal together with its presentation frontier. It closes with the auxiliary programs that prepare the host, convert the dataset and evaluate fidelity offline, and with the observables that every one of the units exports for later analysis;

5. **Performance Evaluation**: fixes the conditions under which the platform was exercised and the rules by which a trace may be read, before any quantity is quoted. It records the machine that produced every measurement, the processors withdrawn from ordinary scheduling and the unit assigned to each of them, the state the entry script establishes before the first container exists, and the generated code that binds the accelerated units to one graphics generation. It then separates the two families of execution, the interactive one and the one that arms the fidelity capture, states the operating point the whole campaign holds in common, and gives each of the fifteen executions with the single constant it displaces and the question that justified performing it.

It closes with the checks a trace must pass before it becomes evidence, the steady interval and the comparator applied to it, the limits past which none of these measurements may be carried, and the manifest that has to accompany a result for the campaign to be repeated elsewhere;

6. **Results**: reports what the archive contains, in the order the preceding chapter prescribes. It establishes how far the unchanged configuration itself varies, since nothing later can be read without that, then sets the chain against the application-level pipeline at the only boundary they share. The paired ablation follows, and it is the part of the evidence that sustains the case this thesis makes, since it shows where the arithmetic is paid, how much processing margin the compression unit regains, how the spacing of arrivals behaves along the route, and how far the difference survives to the terminal. The capture-enabled pair then asks whether the reconstruction changes when the frontier moves, at the level of the admitted population, of the coded stripe, and of the rebuilt cloud, with one frame retained for visual inspection on either side of the coder. The sensitivity conditions fix the limits of that point through packetisation, source residency, and datagram rate, the cost of an interaction closes the sequence, and the chapter ends by weighing the four research questions against what was actually measured, and by naming the three thresholds that settled each of those verdicts in the first place;

7. **Conclusions**: gathers what the campaign establishes, where its evidence stops, and what a subsequent study would attempt. It restates the four outcomes the measurements support, the decomposition of four coarse blocks into seven addressable units with the two reverse loops handed to the units that require them, the delay read at the single frontier the two architectures share, the processing margin the compression unit regains once the geometric frontier is settled in transit, and the reconstruction that does not change when it moves. It then draws the boundaries of the evidence, the single host with its fixed topology and generated device code, the single sequence and the single stream, the terminal effect whose direction is reproducible and whose magnitude is not, and the fidelity statement that depends on the assessment procedure it declares. It closes with the work that follows from those boundaries, a study of the constants that govern the regulator and a controller replacing its thresholds with an estimate, the sub-frame coding that would move the one synchronous point the chain retains, a temporal control at the terminal beside the one at the source, and the deployments, datasets, and resource measurements that would separate the architecture from this laboratory.

Chapter 2

Background

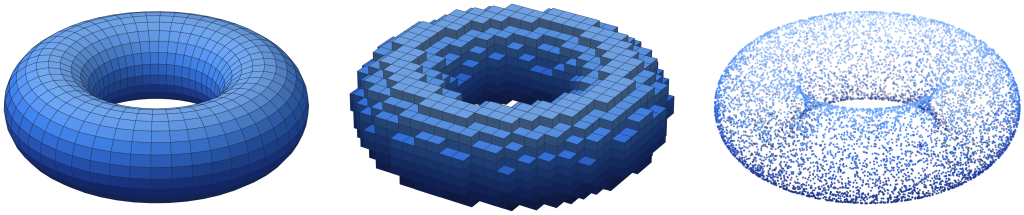
2.1 Foundations of Point Cloud Streaming

Every live volumetric service must settle five matters before a single frame reaches the viewer, namely the format that describes the scene, the manner in which that media model is reduced to a transmissible size, the processors that absorb the heaviest transformations, the mechanisms that carry the result across the network, and the measures that establish whether what arrives is faithful to what was captured. These choices are not independent, since each one narrows the options available to the next. The subsections below therefore address them in that order, drawing on the literature for general principles and on the reference implementation only where its specific design decisions bear on the present study.

2.1.1 Volumetric Representations

Three-dimensional content admits two broad modelling families. **Implicit representations**, such as **Signed Distance Functions** (**SDFs**) or **Neural Radiance Fields** (**NeRFs**), encode a scene indirectly through a function that must be queried to obtain geometry or appearance [10, 45]. They offer remarkable compactness, but the evaluation step they require remains too costly for the live and interactive regime considered here, and they are consequently excluded from this work. **Explicit representations**, in contrast, enumerate geometric primitives directly, and three of them dominate the field. A **polygonal mesh** describes a surface through vertices, edges, and faces, which renders smooth geometry and fine texture with high fidelity. Its connectivity, however, has to be rebuilt whenever the topology of a moving subject changes, so live capture and manipulation become computationally demanding. A **voxel grid** extends the notion of pixel to three dimensions by assigning a value to each cell of a uniform lattice. Indexing thus becomes trivial, yet memory grows

with the cube of the resolution, and fine detail vanishes whenever cells are coarser than the features they discretise. A **Point Cloud**, finally, lists individual samples in space, each defined by its position and possibly enriched with attributes such as **Red, Green, and Blue** (**RGB**) colour. It is the native output of **LiDAR** sensors and of **RGB-Depth** (**RGB-D**) rigs, it needs no connectivity, and it can be manipulated on the client with modest effort, although a faithful rendition may call for a very large population and hence for considerable storage and bandwidth. Figure 2.1 contrasts the three explicit forms on one object, rendered from a single viewpoint, so that their spatial organisations can be compared.



(a) Polygonal mesh **(b)** Voxel grid **(c)** Point Cloud

Figure 2.1. Explicit representations of a single object

When coordinates are quantised onto a regular lattice, the cloud is said to be **voxelised**. The outcome joins the flexibility of scattered samples with the ordered addressing of a grid, since every point can be located through integer indices and neighbouring positions are predictable. This regularity simplifies spatial queries, bounds the numeric range of each coordinate, and constitutes a widespread assumption among compression pipelines, the one adopted here included [72].

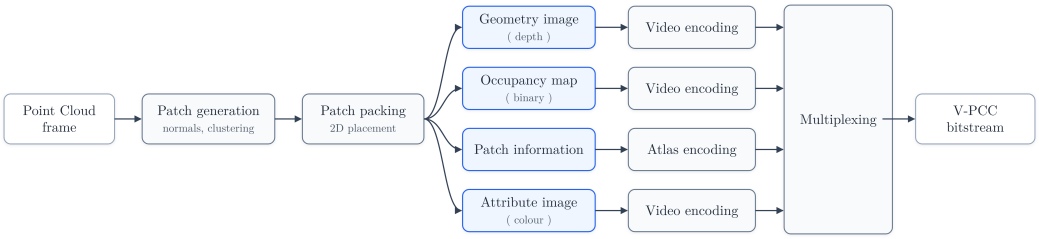
2.1.2 The 8iVFB Dataset

This **8iVFB** collection ranks among the reference benchmarks of volumetric research and forms part of the material with which the **Moving Picture Experts Group** (**MPEG**) evaluates its compression standards [11]. It comprises four dynamic captures of a full human body, named **Longdress**, **Loot**, **Redandblack**, and **Soldier**, each recorded for ten seconds at 30 frames per second by 42 RGB cameras grouped into 14 clusters, each behaving as one logical RGB-D sensor. Frames are voxelised on a $1024 \times 1024 \times 1024$ lattice, which corresponds to 10 bits per coordinate and is commonly labelled **vox10**, and each occupied cell carries an RGB triplet. Loot, the sequence employed throughout this thesis, spans 300 frames of roughly 0.8 million points each, for a total of 238,146,391 samples and an average of about 794 thousand per frame. The population of individual frames ranges from 770,822 to 835,458 points, with a standard deviation below 2 % of the mean, so the per-frame

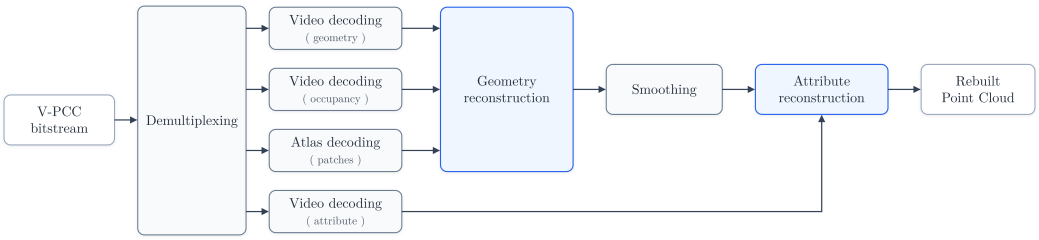
workload remains nearly constant and differences between runs are unlikely to stem from the content itself. Such a combination of density, surface-like distribution, and regular addressing is exactly the regime in which projection onto planar images yields its greatest benefit, as the following subsection clarifies.

2.1.3 MPEG Point Cloud Compression

The volume involved makes compression a precondition for any practical service. In the uncompressed layout adopted by the platform, which stores 16 bytes per point, one Loot frame occupies about 12.7 MB on average, and a stream at 30 frames per second would require roughly 3 Gbit / s, far beyond ordinary access links. Conventional video codecs cannot be applied directly either, since a Point Cloud forms an unordered collection devoid of the implicit neighbourhood of a pixel grid. MPEG issued a call for proposals in 2017, which eventually produced two complementary specifications, now published by the **International Organization for Standardization** (**ISO**) together with the **International Electrotechnical Commission** (**IEC**) in the 23090 family [21, 29, 59]. The effort distinguishes three categories of content. Static clouds include surveys of architecture or heritage objects, dynamic clouds capture a moving subject across consecutive frames, and dynamically acquired clouds are typically sparse LiDAR sweeps collected while the sensor itself moves. **V-PCC** targets the second category by turning geometry and attributes into ordinary video. Each frame is first segmented into **patches**, clusters of points sharing a similar surface orientation, estimated from their normals and projected onto one of six axis-aligned planes, a decomposition that avoids the self-occlusions a single projection would cause. The patches are subsequently arranged on a rectangular canvas, which yields a **geometry image** storing depth, an **attribute image** storing colour, and a binary **Occupancy Map** marking the pixels that hold valid samples, complemented by the patch descriptors required for reconstruction. Each of them records the plane onto which its patch was projected, the place it occupies on the canvas, and the offsets that return it to three-dimensional space, while the pixels left unoccupied are filled with values that smooth the transition between neighbouring patches and thereby lower the residual handed to the coder. The three images are compressed as distinct video sub-bitstreams, whereas those descriptors travel in a dedicated **atlas** sub-bitstream, and the whole is multiplexed into a single container. The decoder mirrors these steps, rebuilding positions from depth, occupancy, and patch data, smoothing the seams between adjacent patches, and restoring colour, as Figure 2.2 summarises for both directions of processing, from the frame delivered by the source to the cloud rebuilt at the far end of the chain.



(a) Encoding



(b) Decoding

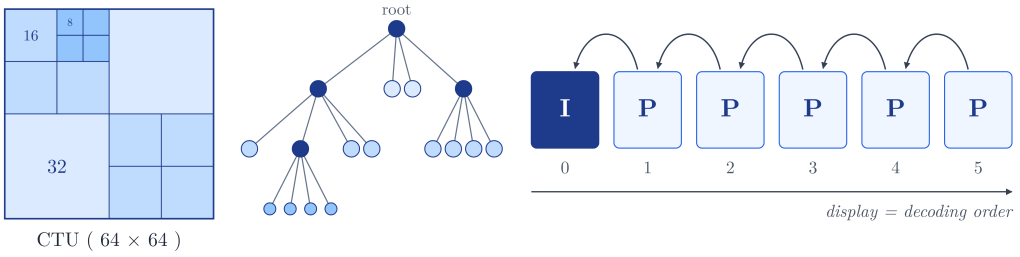
Figure 2.2. Processing stages of V-PCC

G-PCC, conceived for the first and third categories, operates directly in three-dimensional space. Geometry is represented through an octree, or alternatively through surface approximations and predictive trees, while attributes are coded with hierarchical schemes such as the **Region-Adaptive Hierarchical Transform** (**RAHT**) or with predicting and lifting transforms [27, 52]. For dense, body-shaped captures such as the 8iVFB subjects, the projection route is the natural fit, and it brings a practical advantage beyond efficiency. Once volumetric data become image sequences, the whole ecosystem of codec hardware opens up to the pipeline. The reference implementation keeps this principle, projecting each frame onto six orthogonal views in cross-shaped atlases for geometry, texture, and occupancy, without adopting the full set of normative tools or their syntax.

2.1.4 High Efficiency Video Coding

Since the projection route delegates the bulk of the payload to a video codec, the characteristics of that codec shape the whole pipeline. **HEVC**, finalised in 2013 by the **ITU Telecommunication Standardization Sector** (**ITU-T**) jointly with ISO / IEC as the successor of H.264, also known as **Advanced Video Coding** (**AVC**), attains roughly half the bitrate of its predecessor at comparable perceived quality [61, 71]. Instead of fixed macroblocks, it partitions each picture into **Coding Tree Units** (**CTUs**) whose size reaches 64×64 samples, which a quadtree recursively splits into **Coding Units** (**CUs**) down to 8×8. Each CU is in turn

partitioned into **Prediction Units** (**PUs**), which govern how a block is predicted, and **Transform Units** (**TUs**), which delimit the areas subjected to the residual transform. Large blocks are thus reserved for homogeneous regions and small ones for detail, as Figure 2.3a illustrates. Encoding proceeds through four conceptual stages. Intra- or inter-picture prediction first removes spatial or temporal redundancy. An integer approximation of the **Discrete Cosine Transform** (**DCT**) then converts the residual into frequency coefficients, with a **Discrete Sine Transform** (**DST**) variant reserved for small intra **luma** blocks. Quantisation, the only lossy step, is governed by the **Quantisation Parameter** (**QP**), and entropy coding, in the form of **Context-Adaptive Binary Arithmetic Coding** (**CABAC**), compacts the result without further loss. In-loop deblocking and **Sample Adaptive Offset** (**SAO**) filters refine the decoded pictures before they serve as references. Along the time axis, frames are organised in a **Group of Pictures** (**GOP**), which mixes intra-coded I-frames acting as random-access points, predicted P-frames relying on earlier pictures, and bi-predicted B-frames that also reference later ones at the cost of an extra reordering delay at the encoder and the decoder.



(a) Quadtree partitioning of a CTU **(b)** Low-delay GOP

Figure 2.3. Spatial and temporal organisation in HEVC

Interactive services therefore favour low-delay structures made of an initial I-frame followed only by P-frames, shown in Figure 2.3b, which trade some efficiency for the absence of reordering. Two consequences bear directly on a projection-based chain. Atlas faces are best aligned with the CTU grid, so that no coding block straddles two views, and the rate-control mode, the GOP length, and the **placement** of I-frames influence both the size of each encoded frame and the delay before it becomes decodable, aspects that Chapter 3 takes up in detail.

2.1.5 Hardware-Accelerated Processing

Reducing volumetric compression to video coding moves the heaviest workload onto operations for which dedicated silicon exists. Recent **NVIDIA GPUs** embed fixed-function engines, made accessible by the **Video Codec SDK** as **NVENC**

for encoding and **NVDEC** for decoding, which operate independently of the programmable cores and handle HEVC streams with high throughput and modest latency. General-purpose multimedia toolkits, **FFmpeg** among them, wrap these engines as regular codecs and allow presets tuned for low delay to be selected. Projection and reconstruction, by contrast, belong to the volumetric domain and lie beyond the capabilities of a codec engine. Being data-parallel, since the same arithmetic applies independently to each point or pixel, they map well onto general-purpose programming through **CUDA**, whose kernels execute across thousands of threads and allow intermediate buffers to remain in device memory, thereby avoiding repeated transfers across the host interconnect. The reference implementation employs both forms, CUDA kernels for projection and reconstruction and hardware engines for coding, and provides a processor-only variant of the geometric stages so that the gain attributable to the accelerator can be quantified [18, 38, 46, 47]. The per-frame statistics that parameterise the projection are computed on the host before the first kernel runs, and it is this step that the proposed approach relocates.

2.1.6 Transfer of Volumetric Content

Transmission techniques for volumetric media fall into three families, frequently combined. **Viewport-adaptive delivery** exploits the fact that only part of the scene lies within the observer's view at any instant. By predicting where the viewer will look, and by discarding occluded or distant content, the sender reserves full quality for the visible portion, as ViVo demonstrates on mobile devices. Implementations commonly divide each frame into independently decodable tiles, so that only the portion the viewer can reach is put on the wire. **Layered delivery** encodes a base layer with the essential information and adds enhancement layers only when capacity allows. **Super-resolution-based delivery** transmits a sparser or coarser cloud and restores density at the receiver through learned models, trading bandwidth for local computation, as in YuZu [23, 73]. Adaptive bitrate streaming supplies the transport on which these techniques usually rest. MPEG-DASH segments the content into chunks available at several quality levels and lets each client select the level suited to its viewport and throughput, while scaling through standard **Hypertext Transfer Protocol** (**HTTP**) distribution infrastructure. Although traditionally associated with segments lasting several seconds, low-latency variants built on chunked transfers approach the delay of stacks based on the **Real-time Transport Protocol** (**RTP**), such as **Web Real-Time Communication** (**WebRTC**). **Jansen** et al. exploit this property for multiparty volumetric conferencing and identify the point count of each frame as a primary determinant of both latency and frame rate [1, 28, 31, 43, 58]. Rendering, lastly, may occur on the terminal itself or be delegated to edge or

cloud infrastructure, according to the capability at hand and the latency budget. The pipeline considered in this thesis reconstructs the cloud in a dedicated decoding stage before the terminal presents it in a web viewer, and relies on datagram transport rather than HTTP objects, because a live pipeline measured at frame granularity must avoid the buffering that segmented streaming entails.

2.1.7 Quality Assessment

Fidelity is assessed at two levels, both employed in this work. At the **signal level**, the decoded projection is contrasted with its uncompressed counterpart before any reconstruction takes place. For an image of N samples x_i and its decoded version $\hat{x}_i$, the **MSE** and the **PSNR** of the pair are defined as

$$\mathrm{MSE} = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{x}_i)^2\,, \qquad \mathrm{PSNR} = 10\log_{10}\frac{L^2}{\mathrm{MSE}}, \tag{2.1}$$

where L is the largest representable value, equal to 255 for 8-bit samples. Because a purely pixel-wise measure ignores how distortions are perceived, **SSIM** complements it by comparing local **luminance**, contrast, and structure over sliding windows:

$$\mathrm{SSIM}(x,y) = \frac{(2\mu_x\mu_y + C_1)\,(2\sigma_{xy} + C_2)}{\left(\mu_x^2 + \mu_y^2 + C_1\right)\left(\sigma_x^2 + \sigma_y^2 + C_2\right)}, \tag{2.2}$$

with μ, σ^2, and σ_{xy} denoting local means, variances, and covariance, and C_1, C_2 small stabilising constants. The index approaches unity as the two windows become indistinguishable. For a projected atlas, only the luma plane of the geometry region enters the comparison, since **chroma** carries no depth information. At the **geometric level**, the rebuilt cloud Q is set against the reference P in three dimensions. The building block is the point-to-cloud distance $d(q,P) = \min_{p\in P}\|q - p\|_2$, from which the aggregate indicators adopted in this work derive:

$$e_{\mathrm{mean}} = \frac{1}{|Q|}\sum_{q\in Q} d(q,P), \qquad e_{\mathrm{RMSE}} = \sqrt{\frac{\sum_{q\in Q} d(q,P)^2 + \sum_{p\in P} d(p,Q)^2}{|Q| + |P|}}, \tag{2.3}$$

$$d_{\mathrm{C}} = \frac{1}{|Q|}\sum_{q\in Q} d(q,P) + \frac{1}{|P|}\sum_{p\in P} d(p,Q), \qquad d_{\mathrm{H}} = \max\left\{\max_{q\in Q} d(q,P), \max_{p\in P} d(p,Q)\right\}. \tag{2.4}$$

The directed mean describes how far reconstructed points lie from the reference surface. The symmetric **Root Mean Square Error** (**RMSE**) penalises large deviations more severely, while the **Chamfer distance** d_{C} adds the opposite direction and thereby also accounts for reference regions that the reconstruction failed to cover. The **Hausdorff distance** d_{H} reports the worst deviation. MPEG evaluates codecs through closely related point-to-point and point-to-plane measures,

commonly expressed as PSNR values normalised by the resolution of the cloud [67]. All these quantities presuppose that Q and P share one coordinate frame. When a pipeline normalises and re-poses the content, a rigid registration is therefore required first, typically through the **Iterative Closest Point** (**ICP**) algorithm, which alternates between matching each point with its nearest counterpart and estimating the rotation and translation that minimise the sum of their squared offsets. The resulting distances therefore quantify reconstruction error net of any global rigid displacement, which the alignment absorbs by design. Since spurious points can bias that estimate, robust variants discard statistical outliers from the subset used for registration while still computing the reported distances on the complete cloud. Chapter 5 specifies how these choices are fixed, together with the correspondence radius and the outlier criterion adopted there [3, 56].

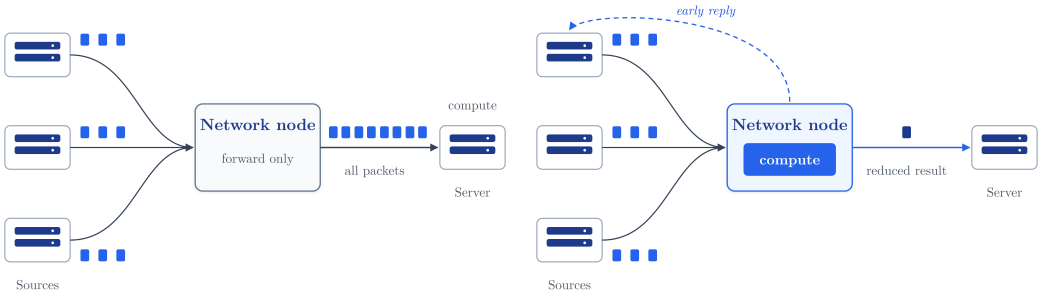
2.2 Computation Bound to Packet Traversal

The previous section treated the network as the medium across which volumetric content travels. **INC** questions precisely this assumption. Its relevance to the present work lies less in any single application than in the reasoning that decides when an operation belongs to the path, and the subsections below retrace that argument from its origins to the questions it still leaves open for media workloads such as the volumetric streaming pipeline examined in this thesis.

2.2.1 Origins and Definition

The idea that forwarding elements might execute user-defined logic predates current hardware by more than two decades. **Active networks**, proposed in the mid-1990s, envisaged packets that carried their own code, or chose among routines already resident, to be executed by the routers they traversed, an ambition that stalled on concerns of safety, performance, and manageability. **Software-Defined Networking** (**SDN**) later separated control from forwarding and, through interfaces such as **OpenFlow**, made forwarding behaviour externally configurable, while the set of header fields and actions remained fixed by the device [36, 42, 64]. **PDPs** lifted this last restriction, and INC emerged as the systematic use of that freedom for application purposes. In this thesis, the term denotes the execution of part of a distributed service on a device located along the path between its endpoints, where traffic is inspected, transformed, aggregated, or answered as it passes. Two features distinguish this notion from plain **offloading** onto an edge server. The host is selected because the data already traverse it, not because it offers spare capacity, and the computation is organised around the unit of transit, namely the packet,

rather than around the complete message. Figure 2.4 contrasts the two models of placement for traffic converging on one receiver from several sources.



(a) End-host model **(b)** In-network model

Figure 2.4. Location of computation within the data path

In the end-host model, every datagram crosses the intermediate node unchanged and the target host bears the entire effort. In the in-network variant, by contrast, the node absorbs part of that effort, forwarding a condensed result or replying on behalf of the receiver, so that bytes, hops, or processing cycles are saved before the traffic reaches its destination. The second model does not abolish the first, since the server remains responsible for whatever the node cannot complete, and the practical question becomes how to divide the work between them.

2.2.2 Motivations and Expected Benefits

Programmable switches process billions of packets per second with sub-microsecond latency, and every packet of a distributed application already crosses them. Four benefits are usually invoked in favour of exploiting this position. The first is **data reduction**, obtained when partial results are merged before they reach their destination. The second is **latency reduction**, achieved when a request can be satisfied without travelling to a remote server. The third is **load relief**, since duties withdrawn from end hosts free processing capacity for other tasks, and the fourth is **energy efficiency**, because specialised pipelines may perform certain operations at a lower cost per bit than general-purpose servers. These gains are neither automatic nor independent. Ports and Nelson argue that each placement must be justified on its own merits, weighing the expected advantage against the restricted primitives, the limited state, and the operational complexity that network devices entail, and they caution against migrating logic merely because the hardware permits it [50]. The same stance informs the present work, which treats the forwarding path as one candidate site among several and accepts it only where the benefit can be measured under the load conditions of a live service. In concrete terms, a relocation is retained

only if it lowers the processing time or the workload of the component it relieves, leaves the content of every frame intact, and does not lengthen the end-to-end delay perceived by the viewer. A gain confined to a synthetic benchmark, or one that merely shifts the burden onto the forwarding node without a clear benefit for the service, is not regarded here as sufficient ground for the move.

2.2.3 Representative Systems

The literature is commonly organised by application domain, and a handful of prototypes illustrate the recurring patterns, summarised in Table 2.1. **Aggregation** is exemplified by **SwitchML**, which sums gradient vectors produced by many workers inside a top-of-rack switch during distributed training, reducing the traffic each worker must exchange and accelerating the procedure by up to 5.5 times on the reported benchmarks. Since a switch cannot buffer a complete gradient, workers stream it in fixed-size chunks that occupy a pool of aggregation slots, shadowed so that loss does not corrupt the partial sums. **Caching** is represented by **NetCache**, which keeps the most popular key-value pairs in on-chip memory so that skewed workloads are partly served by the network itself, raising throughput three- to tenfold while balancing demand across storage servers. The **Data Plane** also counts queries per key, letting a controller refresh the cached set as popularity drifts. **Coordination** services such as **NetChain** implement chain replication within the fabric and complete agreement in less than a server round trip. **Monitoring** takes the form of **INT**, in which packets collect queue occupancy, latency, and path information at every hop, exposing network state per packet without dedicated probes. **Load balancing** appears in **SilkRoad**, which tracks state for millions of flows in switch memory and replaces software balancers at line rate, sparing the servers it displaces [32, 33, 35, 44, 57]. Its connection table stores digests rather than full identifiers, and a transitional structure keeps consistency as the pool changes. These five examples also differ in how deeply they must look into a packet. The annotation carried out by INT and the address mapping of SilkRoad touch header fields alone, whereas aggregation, caching, and replication interpret values that the application placed in the payload, and therefore demand a parser reaching past the protocols a forwarding chip natively recognises. All of them, however, act on elastic traffic, since a gradient chunk, a read request, or the opening of a session is released when its producer is ready and loses little by arriving slightly later, a latitude that content generated at a fixed cadence does not enjoy. Each was moreover evaluated inside one data centre, where the topology is known in advance and a single authority owns every element involved. The gains quoted above belong to that setting, and they describe what the placement yields on a short and homogeneous path free of

competing tenants, not a property that would survive once the route lengthens or crosses more than one administrative domain.

Pattern	System	Operation in transit	State on the device
Aggregation	SwitchML	Summation of gradient chunks sent by the workers	Aggregation slots with shadow copies for recovery
Caching	NetCache	Answers to read requests for frequently accessed keys	Cached key-value pairs and per-key query counters
Coordination	NetChain	Chain replication of stored values across switches	Replicated values and per-key sequence numbers
Monitoring	INT	Insertion of per-hop metadata into the packets	None beyond the metadata carried by each packet
Load balancing	SilkRoad	Mapping of each connection to a server instance	Compact digest table for millions of connections

Table 2.1. Representative in-network computing systems

What these systems share is structural rather than functional. Each operation is simple with respect to the capabilities of the device, consumes information already carried by the traffic, and produces a benefit that grows with the volume flowing through the node. None of them, moreover, requires the switch to reconstruct an entire application message, since every packet contributes a self-contained fragment to a state that remains valid after each update. The geometric processing examined in this thesis borrows from two of these patterns. Like SwitchML, it merges what many packets carry into a compact state, and, like INT, it appends the result to the relayed packets as metadata destined for a downstream consumer, although its unit of accumulation is a Point Cloud frame rather than a training step or a single hop, so that part of its work must await the last datagram.

2.2.4 Constraints of the Execution Substrate

These systems in turn reveal the boundaries of the approach. Switching pipelines offer a small number of match-action stages, each able to touch a bounded amount of state per packet, typically without loops, dynamic allocation, or floating-point arithmetic, and with a total on-chip memory in the order of tens of megabytes. SwitchML, whose conversion of gradients into fixed-point integers was recalled in the first chapter, also leaves normalisation to the end hosts, since division is unavailable in the pipeline. State is organised per packet rather than per message, so any computation spanning many datagrams has to be decomposed into incremental updates and must tolerate loss, duplication, and reordering unless the application restores these guarantees end to end. Correctness therefore relies on explicit fallbacks, whereby the endpoint recomputes or retransmits whatever the device could not

complete. **Tokusashi** et al. further show that the advantage depends on load. A network device performing application work proves more efficient than a server under heavy traffic, but not necessarily when utilisation is low, which motivates activating in-network logic on demand [68]. INC is thus a placement decision subject to verification, not a universal improvement, and the substrate decides what may be admitted there, whatever the appeal of moving it.

2.2.5 Placement within the Compute Continuum

This view connects INC with the orchestration literature on that same **continuum**, where the functions of a service are distributed across devices, edge nodes, and cloud resources. Early formulations relied on **Virtual Network Embedding** (**VNE**), which maps each function to a single node and each stream to a single path, and therefore cannot represent streams that are replicated or shared among several consumers, a trait that characterises immersive media. The **Cloud Network Flow** (**CNF**) model overcomes this limitation by casting placement, routing, and resource allocation as a single information-flow problem over a network augmented with computing locations [6, 7, 40, 48]. Subsequent work has derived approximation algorithms for information-aware service graphs and dual-timescale control systems that pair centralised planning with distributed reactions to queue dynamics. These frameworks treat each function as an opaque component with given requirements and leave its internal realisation unspecified. INC extends the set of candidate hosts to the forwarding elements themselves, but only for functions whose internal structure is compatible with them, and it is exactly this compatibility that the orchestration models take for granted without examining it in any detail.

2.2.6 Implications for Volumetric Pipelines

Media payloads pose a particular challenge in this respect. The unit of meaning, a frame, is scattered across a large number of datagrams, close to ten thousand per Loot frame at the nominal packetisation of the platform, and most processing stages expect that unit to be fully assembled before they begin. Comparatively few studies have examined which parts of a volumetric pipeline satisfy the conditions discussed above. The criteria distilled from the literature nevertheless apply directly. A candidate function has to be decomposable into per-packet updates, hold a bounded state per frame, tolerate the absence of a whole message until its final step, and yield a result that downstream stages can consume in place of their own. The classification of operations introduced in the first chapter translates these criteria into the vocabulary of Point Cloud processing. The space requirement is the most demanding of the four for geometric work. Retaining the coordinates of a Loot

frame until the radius can be resolved occupies close to ten megabytes, an amount that exceeds by orders of magnitude the memory a switching pipeline can offer to a single function. Which substrates can host a function meeting these criteria, and at what price in expressiveness, is the question taken up next.

2.3 Data Plane Programmability

In-network functions require a forwarding path whose behaviour can be defined by software rather than fixed in silicon. This section examines how that requirement is met, moving from the pipeline abstraction adopted by programmable switches to the range of hardware and software targets, then detailing the framework on which the proposed platform is built and the chaining abstractions it borrows to coordinate several in-path elements, with particular attention to what each option allows a function operating in transit to compute and retain. Three properties recur throughout the comparison, namely the arithmetic a target supports, the amount of state it can preserve across packets, and the visibility it grants over its own queues, since together they decide whether a geometric reduction can be hosted at all and whether its effect can then be measured reliably. The last of them is often overshadowed by raw throughput, yet it weighs heavily here, since the conclusions rest on attributing delay and load to individual units.

2.3.1 Programmable Switching Pipelines

As recalled in the previous section, OpenFlow confined matching to a predefined collection of header fields, so that supporting a new protocol required revising the specification and, eventually, the silicon. The **Reconfigurable Match Tables** (**RMT**) architecture showed that a switching chip could instead expose a programmable parser and a sequence of match-action stages operating on arbitrary header fields while sustaining hundreds of gigabits per second. **P4** then provided a language for such targets. A P4 program declares the headers to be recognised and a parser that extracts them, a set of tables that match on parsed values or metadata and trigger actions capable of rewriting fields, updating registers, or selecting an output port, and a deparser that serialises the modified packet. A separate runtime interface lets the **Control Plane** populate the tables while the pipeline runs. A traffic manager separates the ingress from the egress half, queueing packets, scheduling their departure, and replicating them when a copy must reach more than one port. Figure 2.5 depicts the corresponding abstract pipeline, commonly referred to in the literature as the **Protocol-Independent Switch Architecture** (**PISA**) and adopted, with variations, by most programmable targets [4, 5].

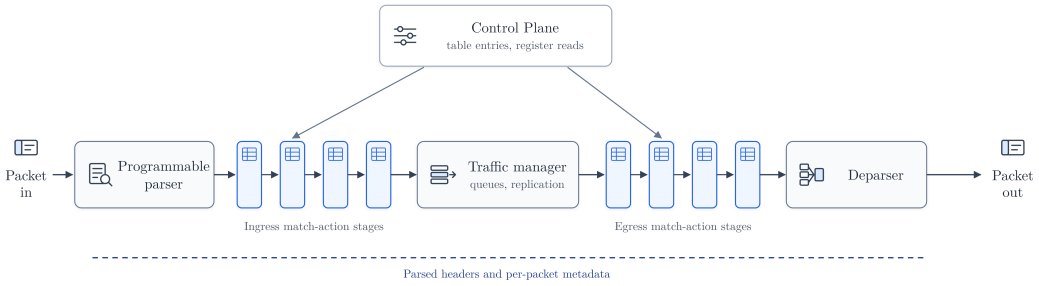


Figure 2.5. Abstract pipeline of a programmable switch

The model is intentionally restrictive. Every packet traverses a bounded number of stages in constant time, and each stage can access a given memory region only once per packet, which guarantees line-rate operation regardless of the traffic mix. The price is paid in expressiveness, because a program is admitted only if the compiler can map every table, register, and action onto the physical resources of the stages, so iteration, dynamic allocation, and elaborate arithmetic are excluded by construction. Stateful elements such as registers and counters persist across packets, but each can only be updated through a simple read-modify-write operation completed within a single stage. Arithmetic is generally limited to integer addition, subtraction, shifts, and bitwise logic, while matching memory is divided between **Static Random-Access Memory** (**SRAM**) for exact lookups and **Ternary Content-Addressable Memory** (**TCAM**) for ternary and longest-prefix lookups, both scarce and statically partitioned among the tables at compile time. The parser, in turn, inspects only a bounded portion of each packet, so that data carried deep in the payload remain largely opaque to the pipeline, a restriction of direct relevance for datagrams that transport geometric samples rather than protocol fields. Recirculating a packet through the pipeline offers a partial remedy, although every additional pass consumes a share of the bandwidth otherwise available to new traffic. These properties explain why the systems surveyed in the previous section decompose their logic into small per-packet steps, and they define the target against which any function proposed for hardware offload must eventually be tested.

2.3.2 Hardware and Software Targets

Programmable behaviour can be deployed on devices with very different trade-offs. Switching **ASICs** offer the highest aggregate throughput together with the tightest constraints. **SmartNICs** place multicore processors, programmable pipelines, or both on the network adapter, relieving the host of selected tasks, whereas **FPGAs** provide custom logic with greater flexibility at the price of longer development cycles. Software targets run on commodity servers and give up raw speed in exchange for

general-purpose development, floating-point arithmetic, and memory measured in gigabytes. Their performance hinges on bypassing the overheads of the conventional kernel stack, namely per-packet interrupts, system calls, data copies between kernel and user memory, and generic buffer management. Three frameworks illustrate the main strategies. **netmap** maps adapter rings into **user space** through a thin kernel module, and **DPDK** moves the driver itself out of the kernel and polls the device. The **eXpress Data Path** (**XDP**) takes the opposite route, running programs written for the **extended Berkeley Packet Filter** (**eBPF**) inside the kernel at the earliest point of reception, after a verifier has proved them safe [24, 41, 54]. The three strategies differ in how much of the host they surrender. netmap and XDP leave the adapter under kernel control, so conventional tools and protocols keep working alongside the accelerated path. DPDK, by contrast, detaches the device from the kernel altogether, a price it repays with complete control over buffers, queues, and the order in which packets are processed, which instrumentation at packet level requires. Comparative studies show that such frameworks reach similar throughput once batching, polling, and memory pre-allocation are applied, with differences stemming mostly from their abstractions and their integration with the host. Software switches such as **Open vSwitch**, whose datapath can itself run on DPDK, further demonstrate that virtualised deployments can forward traffic between containers or virtual machines entirely in user space [2, 19, 49]. The user-space target adopted here, whose rationale the first chapter anticipated, rests on precisely these properties, and its framework is examined next.

2.3.3 The Data Plane Development Kit

DPDK is a collection of libraries and drivers for fast packet processing in user space, maintained as an open-source project [13, 65]. Its **Environment Abstraction Layer** (**EAL**) initialises the runtime, reserving memory from **hugepages**, large pages that reduce address-translation overhead and remain resident in physical memory, binding each **lcore**, the DPDK name for an execution thread, to a specific processor core, and isolating concurrent instances on the same host through distinct file prefixes. Packets are stored in **mbufs**, fixed-size buffers drawn from pre-allocated **mempools**, which carry both metadata and a reserved **headroom** where headers can be prepended without copying the payload. Lockless **rings** provide bounded queues through which lcores exchange references to such buffers. **PMDs** access device queues directly, and applications move traffic through burst calls that receive or transmit up to a given number of mbufs at a time, amortising per-packet costs. Transmission is explicitly non-blocking. A burst call returns the number of descriptors actually accepted, possibly fewer than requested, and the program decides whether

to retry, buffer, or drop the remainder, which exposes **backpressure** as a quantity that the application can observe, log, and react to. Designs typically follow a **run-to-completion** model, in which one lcore receives, processes, and transmits each burst, or a **pipeline** model, in which stages on different lcores communicate through rings. Polling keeps a core fully busy even when no traffic arrives, so deployments dedicate cores to lcores and often withdraw them from the general-purpose scheduler through the `isolcpus` kernel parameter, protecting timing from unrelated activity. Besides physical adapters, DPDK offers virtual devices. The **virtio-** and **vhost-user** drivers implement the two ends of a virtio channel entirely in user space. The peers negotiate shared regions over a **Unix** socket and then exchange descriptors through virtqueues placed in hugepage memory, so two containerised applications can communicate at user-space speed without any adapter or kernel involvement [55, 63]. Figure 2.6 summarises these blocks for two containers linked in this way, including the descriptor rings and the hugepage region the peers share.

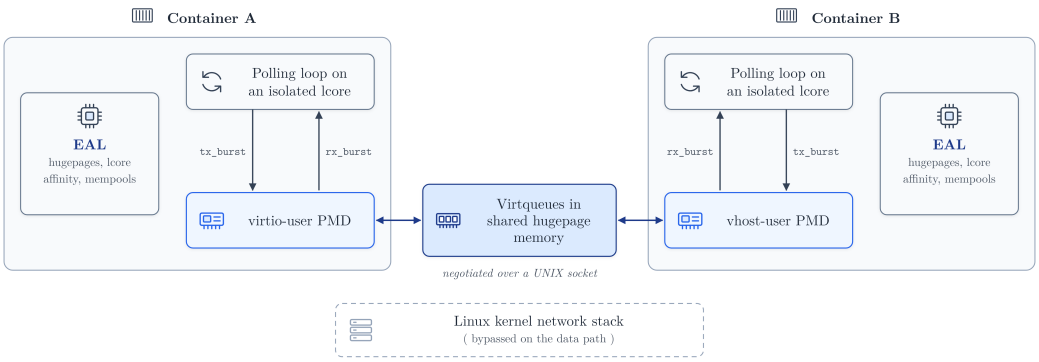


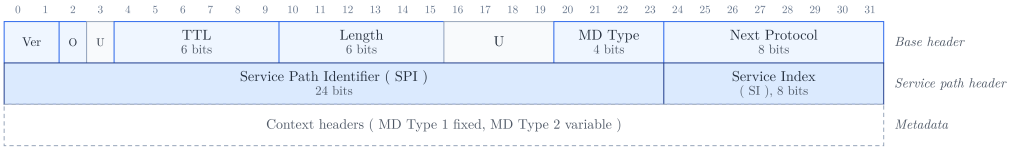
Figure 2.6. DPDK components for two linked containers

This configuration is the one adopted by the platform described in the following chapters. Every unit initialises its own EAL instance under a distinct file prefix, reserving 256 MB of hugepage memory without probing any physical adapter, so that each hop between two units corresponds to one shared-memory channel whose occupancy and transmission outcome can be observed directly.

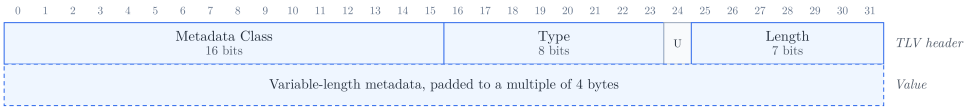
2.3.4 Service Function Chaining and the NSH

Once several in-path elements cooperate on the same traffic, their order and shared context must be expressed explicitly. The **SFC** architecture defines the relevant roles. A **classifier** assigns traffic to a service path, **Service Function Forwarders** (**SFFs**) steer packets along that path, and **Service Functions** (**SFs**) perform the processing itself. An **SFC proxy**, finally, acts for functions unaware of the chaining encapsulation, removing it before delivery and restoring it afterwards [22, 53]. The

NSH carries the corresponding state in band, as shown in Figure 2.7. Its base header specifies a version, a time-to-live decremented at each hop, the total length in four-byte words, the metadata type, and the protocol that follows. The service path header holds a 24-bit **Service Path Identifier** (**SPI**), which names the chain, and an 8-bit **Service Index** (**SI**), decremented as functions are traversed so that the current position is always known. Context headers then carry metadata. With **MD Type 1** they occupy a fixed size of sixteen bytes, whereas with **MD Type 2** they consist of variable-length elements, each identified by a metadata class, a type, and a length, and padded to four-byte boundaries, so that word alignment is preserved from one end of the header structure to the other.



(a) Base and service path headers



(b) Context header of MD Type 2

Figure 2.7. Format of the NSH

These notions supply a vocabulary for describing classification, path position, proxying, and in-band metadata, and they map onto the proposed platform as follows. The **IPN** classifies the traffic of the **Camera** and imposes an MD Type 2 header whose metadata class belongs to the experimental range, the **TPN** acts as proxy for the Encoder and the Decoder, and the **EPN** removes the encapsulation before delivery to the **User**. Three distinct service paths separate the forward flow from the **Temporal** and **Pose** feedback loops. On the forward path, the IPN assigns the initial Service Index, and each return from the Encoder and then from the Decoder, mediated by the TPN, lowers it by one, so that the index observed at the EPN reflects the functions already applied to the frame. Feedback travels under its own identifiers. Pose commands issued by the User receive their header at the EPN, whereas temporal regulation messages directed towards the source terminate at the IPN, which removes the encapsulation before handing them to the Camera. Throughout the chain, the context header conveys the values computed in transit to the downstream units, which no longer need to derive them.

Chapter 3

Reference Pipeline

3.1 Architecture and Interfaces

The system from which this work departs delivers a stored sequence of **Point Cloud** frames to an interactive viewer through four processing modules arranged in a line. A **Camera** reads the source material and injects it into the chain, a **Flow Controller** decides which frames continue towards compression, an **Encoder** turns each admitted frame into a video picture and compresses it, and a **Client** reverses that transformation, rebuilds the cloud, and presents it to the viewer [38]. Two properties shape the whole design. The media travel in one direction only, whereas the commands that govern orientation and temporal sampling return along channels of their own, so that interaction never competes with the stream it modifies. The terminal, moreover, is not a passive receiver. It chooses the viewpoint to be rendered and the cadence at which the sequence advances, and communicates both decisions upstream while the stream continues to flow. Figure 3.1 shows the resulting arrangement together with the transports that join them. The sections that follow examine each module in the order in which the data traverse it, and close with the instrumentation that all four share, the common ground on which every measurement reported later rests, whichever of them produced it.

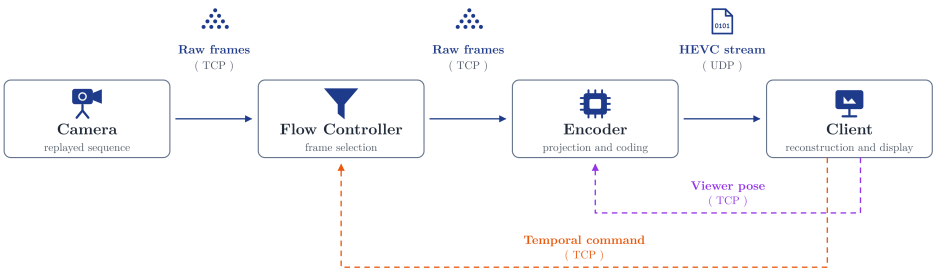


Figure 3.1. Modules of the reference system

The choice of transport follows the nature of each link. Raw frames travel from the Camera to the Encoder, across the Flow Controller, over the **Transmission Control Protocol** (**TCP**), because a truncated or reordered cloud cannot be interpreted downstream. The compressed video instead reaches the Client over the **User Datagram Protocol** (**UDP**), where a late retransmission would be worth less than the delay it costs [14, 51]. The two control channels are again reliable connections, one carrying yaw, pitch, and zoom towards the Encoder, where they steer the projection of the next frame, and the other carrying relative temporal commands towards the Flow Controller, which raise, lower, or restore the sampling rate. Both remain open for the whole session, so that a command never pays the price of a new handshake. All four modules run as containerised services on a single emulated network, where each addresses the others by name, and only the two that drive the video hardware require the graphics processor.

3.2 Acquisition and Isochronous Injection

The Camera stands in for a volumetric sensor. Rather than capturing a live subject, it replays a stored sequence, the **Loot** capture of the **8iVFB** collection, at the nominal rate of a real device. What the substitution reproduces is the cadence of such an instrument, not its physics, since no acquisition noise, no exposure interval, and no latency of its own enters the stream. The frames that depart are exactly those the dataset holds, and the timeline they carry opens when the session does rather than when a scene was lit. Its behaviour falls into three phases, shown in Figure 3.2, namely an offline preparation of the material, a transmission loop governed by an absolute clock, and a background recording of what it observes.

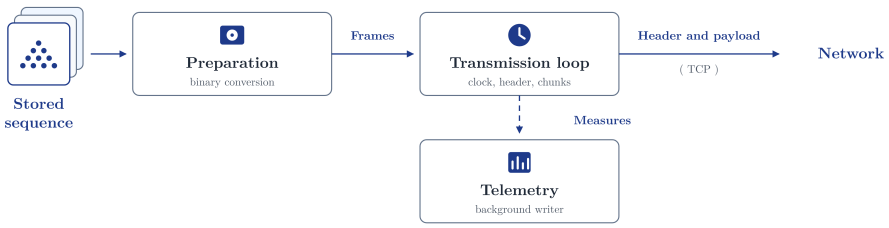


Figure 3.2. Stages of the Camera

Preparation converts the original text-based **PLY** files into a binary layout once and for all. The gain is twofold, since coordinates no longer have to be parsed from characters at every frame, and each value keeps the exact single-precision pattern of the **IEEE 754** standard instead of being rewritten through a decimal

representation [11, 38]. The conversion also shrinks each frame by about 26 % and moves the residual cost of acquisition from the processor to the storage interface. Because the source directory is resolved before transmission begins, the loop itself contains no format decision and performs a plain sequential read from a location that has already been validated. Several input modes are available, differing in the origin of the binary material and in when it is prepared.

3.2.1 Pacing and the Synchronisation Header

Injection is **isochronous**. A master clock is fixed when the session opens, every frame receives a target instant computed from that origin rather than from its predecessor, and any thread that finishes early waits until its instant arrives. Drift therefore cannot accumulate, and the departures preserve the cadence a physical sensor would impose. Each payload is preceded by a fixed header of 32 bytes, written to the socket as a single unit, which carries the length of what follows, the monotonic identifier of the frame, the instant of capture that every later node uses to compute the delay perceived at the terminal, the baseline orientation and magnification of the source, namely null angles and unit zoom, the temporal step in effect, which this module leaves at one, and how many points the frame carries. Two measures protect the departure of large payloads. The small-packet coalescing of the transport is disabled through the `TCP_NODELAY` option, so that bytes reach the interface as soon as they are written, and the payload is pushed in chunks of 128 KB instead of one blocking call, which keeps the send window busy and the context switches few, without letting the buffers of the sender grow without bound.

3.3 Temporal Selection at the Flow Controller

The second module regulates how often a frame goes on. Its role is deliberately narrower than that of the stages that transform geometry or media, since it performs a frame-granular admission decision before projection and coding take place, without interpreting the point payload of an admitted frame. **Temporal** adaptation therefore remains separate from the operations that alter the representation itself, and its effect appears as a controlled change in the cadence presented to the downstream chain. Figure 3.3 sets its three concerns side by side, namely the channel along which the commands of the viewer arrive, the test applied to each frame that reaches the node, and the quantities that either outcome contributes to the record of the session for later analysis of the resulting behaviour.

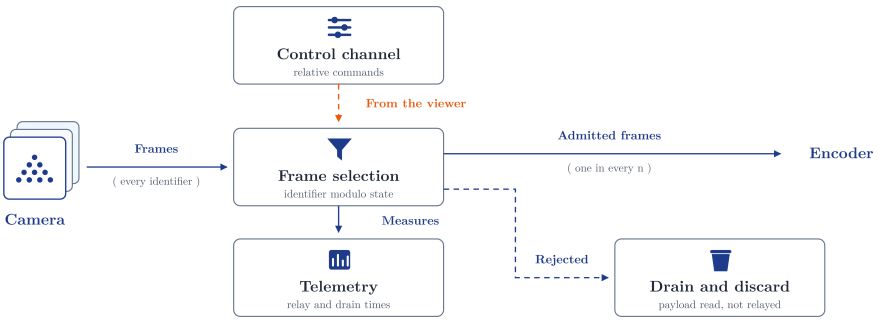


Figure 3.3. Organisation of the Flow Controller

The node holds an integer that the viewer raises, lowers, or resets through relative commands, protected by a lock, so that no arrival is tested while this setting is being updated, and a frame proceeds only when its identifier is divisible by the value in force. Unity therefore lets everything through, whereas a step of n thins by the same factor the cadence that reaches the Encoder. An admitted frame does not keep the header it arrived with. The module rebuilds it, preserving identifier, instant of capture, orientation, and point count, and replacing the temporal field with the value that actually governed the decision, so that the stages downstream know the cadence in force rather than the one the source assumed. The orientation fields cross the node without being read, since the only stage that acts upon them is the normalisation carried out inside the Encoder, and they are therefore relayed verbatim. Discarding, finally, is not free. The payload of a rejected frame must still be drawn from the socket, otherwise the next header would fall out of step with the byte stream, and the time spent draining it is charged to the node, so that the relief obtained further along never looks larger than it is.

3.4 Encoder: from Geometry to Video

The Encoder is where the volumetric representation becomes an ordinary picture. It therefore marks the boundary between operations that still depend on the spatial organisation of the cloud and those that can proceed through a conventional media-processing chain. Crossing that boundary requires the geometry to be expressed in a form that preserves the information needed downstream while exposing a regular image structure to the codec. Its chain, summarised in Figure 3.4, receives the binary frames over the persistent connection opened by the upstream module, brings the cloud into a reference frame of its own, observes it from the faces of a box, packs the resulting maps into a single canvas, and hands it to a video encoder that has no notion of what the samples mean. The reasoning behind this route was presented in

Chapter 2, and the subsections below describe its realisation here, from the arrival of the points to the departure of the datagrams.

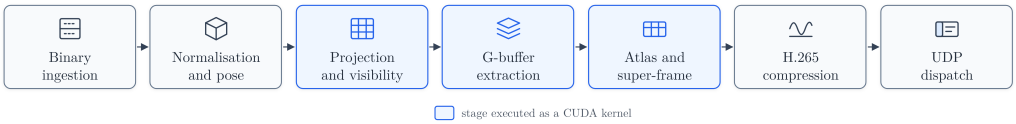


Figure 3.4. Processing steps of the Encoder

The maps shown further on were dumped from the implementation developed for this thesis, which reproduces the layout and the numerical conventions of the reference, and they all belong to a single frame of the sequence. Appearance is rendered through its **luminance** component alone, because the dump facility writes the two colour-difference planes apart from it. This module is also the heaviest of the four, and the only one in which the volumetric and the video description of the same frame coexist, so that the whole cost of the conversion falls inside the budget granted to that frame. Each stage described below is therefore examined for what it computes and for the price it charges to the interval of a frame.

3.4.1 Ingestion and Object-Centred Framing

A frame is read from the connection that the upstream module keeps open, the header that precedes it announces how many bytes follow, and the payload is turned into numerical arrays of positions and colours without any inspection of the stream itself. Those positions refer to the acquisition volume, a frame of reference that suits neither projection nor temporal prediction, since a subject drifting through the scene would make the video encoder track a global displacement instead of a local change of surface. A system of axes proper to the object is therefore established within one traversal. The **barycentre** of a cloud of N points comes first, as

$$C = \frac{1}{N} \sum_{i=1}^{N} p_i, \qquad p_i = (x_i, y_i, z_i), \tag{3.1}$$

and every point becomes $p'_i = p_i - C$, so that rotation and scaling later operate around that centre and the displacement between consecutive pictures reduces to what the surface itself does. The orientation asked for by the viewer is applied next to the centred cloud, as a rotation about the vertical axis for yaw and about the horizontal one for pitch, which keeps the subject framed at any angle. Those angles reach this stage through the channel dedicated to them, and they combine with the baseline that the source writes in every header, so that framing follows the viewer

picture by picture instead of being settled once. A scale comes next, derived from the largest distance between the centre and any point and from how far the virtual rig stands, modulated by the zoom in effect, so that the apparent size of the subject stays stable. A translation along the depth axis finally pushes it away from that rig, which fixes the interval over which depth is later quantised and keeps the whole subject in front of it. The four operations, taken together, compose into a single affine transform applied consistently across the complete frame,

$$p_i'' = s\, R_x(\varphi)\, R_y(\psi)\, (p_i - C) + t, \qquad t = (0, 0, z_0), \tag{3.2}$$

with ψ and φ the angles in force, s the scale just derived, and z_0 the distance to the rig. Because that transform is invertible, the terminal can recover the original coordinates exactly from the handful of parameters that describe it, which is why they are measured here once and then travel with the frame, instead of being reconstructed by the terminal from the picture it receives. The arithmetic per point, moreover, is the same whatever the viewpoint asked for, so a session in which the viewer keeps turning the subject weighs on this stage neither more nor less than one in which the angles are left where the source originally put them.

3.4.2 Fitting the Projection Volume

The extent of the normalised cloud is measured along the three axes and enlarged by a margin of 10 %, which prevents points from touching the border of a view while the subject rotates. A scale is then derived from the most demanding axis relative to the resolution of a face, so that each image remains within its allotted area,

$$S_{\text{global}} = \max\left(\frac{E_x}{W}, \frac{E_y}{H}, \frac{E_z}{W}\right) \cdot \gamma, \qquad \gamma = 1.1, \tag{3.3}$$

where E_x, E_y, and E_z are the measured extents and W and H the pixel dimensions of a single view. A common value serves all six of them. Faces are therefore filled rather than padded with empty bands, at the price of a mild anisotropy between the horizontal and the vertical direction, which is identical on every view and is undone at reconstruction. The value travels with the stream, because without it the receiver could not return from pixels to metric coordinates. In the reference execution order, this fitting stage remains on the host. Once the complete frame is available, the geometric quantities required to delimit the projection domain are resolved before the accelerated projection is launched, so that the kernel receives a region whose centre, extent, and global scale have already been established. The host-side preparation forms a frame-level dependency immediately upstream of the data-parallel projection. This boundary is revisited in Chapter 4, where the information required to derive

the same metadata is accumulated while the frame traverses the network and the completed values are supplied to the Encoder directly.

3.4.3 Orthogonal Views and Depth Arbitration

With the domain settled, the module behaves as a rig of six cameras placed on the faces of that volume. Rays are parallel rather than convergent, so that metric distances survive and no foreshortening appears. Written in homogeneous coordinates, a centred point $p = [\,x, y, z, 1\,]^{\mathsf{T}}$ reaches the plane of the first two axes through

$$P = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}, \qquad P\,p = [\,x, y, 0, 1\,]^{\mathsf{T}}, \tag{3.4}$$

which cancels one axis and leaves the other two as pixel coordinates. What the matrix cancels is not lost, since it is precisely the depth d that the first map will store. Permuting the axes yields the remaining views. The front and the back keep the third coordinate as depth, the lateral pair sets aside the first, and the upper and the lower view reserve the second for the same purpose, while the two coordinates left over address the picture. A cloud is porous, so several points frequently fall on the same pixel, and only the one nearest to the face may survive. A buffer of depths, one per view and reset below the admissible range at the start of every frame, arbitrates that competition sample by sample. Since the comparison is expressed on the complement of the depth for each candidate,

$$D(u, v) = \max\left\{1 - d_i \mid \mathrm{proj}(p_i) = (u, v)\right\}, \tag{3.5}$$

the test reduces to a maximum, which a large number of threads may evaluate on the six faces at once without any ordering among their writes, and every sample that wins it rewrites the attributes of that pixel [8]. The surface obtained in this way carries none of the interior points, whose appearance and disappearance from one picture to the next would unsettle the prediction of the codec.

3.4.4 The Three Map Layers

Each surviving sample feeds three layers, known together as the **G-buffer**, which share the same coordinates and differ in what they hold, shown in Figure 3.5 for a single face of the volume. This organisation converts the irregular point set into aligned raster data without merging attributes that carry different meanings. The resulting layers can therefore be handled as ordinary images while preserving the

correspondence needed to recover the projected surface. Depth already lies on the unit interval, and eight bits carry it into the **Geometry Map** as

$$Y(u, v) = \lfloor 255\, d \rfloor \,, \tag{3.6}$$

where the local smoothness of physical surfaces yields a signal that a video encoder compresses efficiently. Colour reaches the **Texture Map** through a conversion performed during the projection itself rather than in a separate pass over the frame, with the coefficients of the **BT.601** recommendation,

$$\begin{aligned} Y &= 0.299\, R + 0.587\, G + 0.114\, B, \\ U &= -0.169\, R - 0.331\, G + 0.500\, B + 128, \\ V &= 0.500\, R - 0.419\, G - 0.081\, B + 128, \end{aligned} \tag{3.7}$$

which hands the codec the components it already expects, computed inside the loop that is in any case visiting every sample [30]. Appearance, unlike depth, changes abruptly from one pixel to the next, and the two therefore stress the coder in opposite ways. A binary **Occupancy Map** finally records where a sample exists,

$$O(u, v) = \begin{cases} 255 & \text{if a point projects onto } (u, v) \\ 0 & \text{otherwise} \end{cases} \tag{3.8}$$

which separates the subject from the background and keeps the receiver from rebuilding points in regions that hold no sample at all. The explicit mask avoids reserving a particular depth or colour value to signal the absence of geometry, which would make validity depend on the numerical range of another layer.



(a) Geometry **(b)** Texture **(c)** Occupancy

Figure 3.5. G-buffer layers of the face that captures the subject

All three layers are produced together, so that a sample is written once to each of them while its attributes are still in registers, and the cost of the stage follows the count of visible elements rather than the population of the frame. Whether a

location contains a projected point can therefore be established without consulting any of the attributes stored at the same coordinates.

3.4.5 Atlas and Super-Frame Assembly

The six views of each layer are then arranged on a common canvas. Each face is enlarged beforehand to the next multiple of the largest coding block, as

$$W' = \left\lceil \frac{W}{64} \right\rceil \cdot 64, \qquad H' = \left\lceil \frac{H}{64} \right\rceil \cdot 64, \tag{3.9}$$

which keeps every **CTU** of the video encoder inside a single view, in the manner anticipated in Chapter 2, and the canvas inherits that alignment without any padding of its own. Positions are fixed once and for all in a grid of three rows and four columns, as Figure 3.6 shows, the central row carrying front, right, back, and left in that order, with the upper and the lower view in the second column above and below it. Cells that no view occupies are written black. Because the assignment of views to cells never changes, a given surface always lands in the same area of the canvas, which favours prediction from the previous picture. The correspondence is not exact to the picture element, however, since the projection is recomputed for each frame from the extent measured at that moment, so a motionless point may shift slightly when that extent changes. The redundancy actually available between pictures therefore follows the amount of movement in the scene, and not the stability of the layout alone. Half of the twelve cells hold no view at all, a void that costs the coder almost nothing, since a uniform region is described in a handful of bits. Schemes that pack patches wherever room allows would fill the canvas more tightly, and would gain in occupancy what they give up in predictability, which for a real-time chain counts for more. The waste is therefore deliberate, and it is the price at which the position of every surface is known in advance.

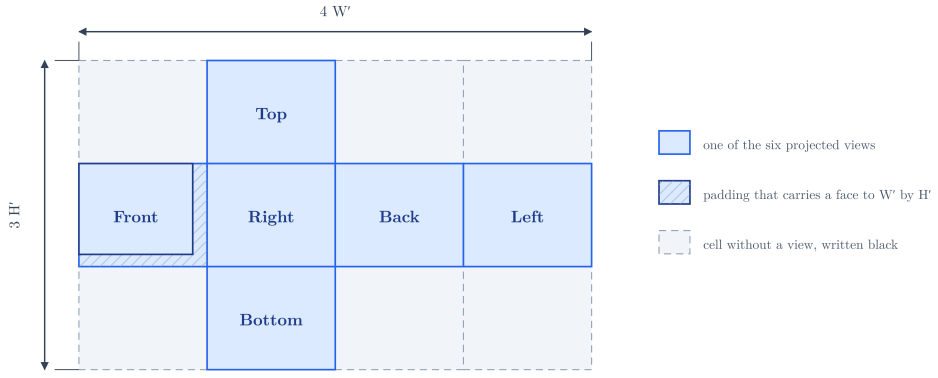


Figure 3.6. Cross-shaped layout of the six views

The three canvases are finally stacked into the single picture that reaches the video encoder, which Figure 3.7 lays out plane by plane. The luminance plane carries geometry, appearance, and occupancy one above the other, whereas the two colour-difference planes are halved in each direction by the **I420** layout, so that

$$W_Y = 4W', \quad H_Y = 9H', \qquad W_{UV} = \tfrac{1}{2}W_Y, \quad H_{UV} = \tfrac{1}{2}H_Y, \tag{3.10}$$

and only the band that holds appearance receives meaningful colour-difference values, the two remaining bands being filled with the neutral level, the value 128 that stands for the absence of any colour difference. The three planes follow one another in memory in that order, each keeping its own dimensions, and what crosses into the video encoder is a view over that memory rather than a copy, so that no duplication of several megabytes burdens the path that each frame must complete within the interval allotted to it by the source. Because depth and occupancy travel in luminance, neither of them is touched by the halving that the colour-difference planes undergo, which is the reason for gathering them there instead of spreading the three layers over the three planes of the buffer. They are written in full even where they carry no information, since the format admits no partial plane, and a constant fill leaves the coder almost nothing to describe over the regions that geometry and occupancy occupy in the luminance above them. The boundaries between the three bands obey the same alignment as the faces, since each band is three padded faces tall, so no coding block ever mixes depth with appearance either. The buffer has the same size for every frame, whatever the population of the cloud, so the pipe into the video encoder always carries the same number of bytes.

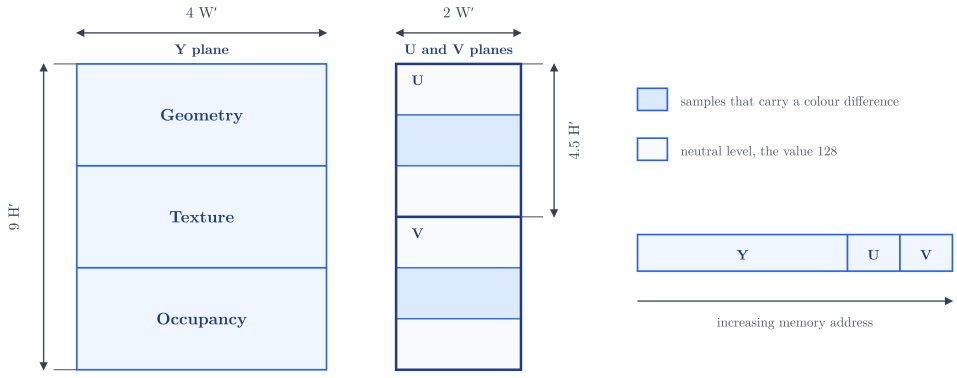


Figure 3.7. Arrangement of the transmitted picture

As far as the codec is concerned the result is a conventional video signal, although its luminance conveys depth and occupancy while its colour-difference samples convey appearance, and that resemblance to an ordinary picture is precisely what allows an

unmodified encoder, and the hardware behind it, to be employed. The interpretation of those channels therefore lies outside the coding stage.

3.4.6 Coding and Dispatch

Compression is entrusted to a separate **FFmpeg** process fed through a pipe, which produces an **HEVC** bitstream. Rate control is a single **Constant Bitrate** (**CBR**) target applied to the whole picture, so the three bands share one **QP** budget, none protected from the others, while the structure of the sequence is the low-delay one of Chapter 2, so that no picture waits for a later one before it can be decoded. The compressed stream is wrapped in an **MPEG Transport Stream** (**MPEG-TS**) container and written to a second pipe, from which the application reads it [18, 26]. Network transmission remains outside that process, which keeps congestion on the link from propagating back into the Encoder and allows the instant of departure of every datagram to be recorded exactly. A thread of its own samples the output pipe at the size configured for the session, so the length of a block is settled by the reader and not by the coder, and one and the same picture may therefore leave as a different number of them depending on that setting alone. Each outgoing block is prefixed by a header of 48 bytes carrying an absolute sequence number, the identifier of the frame being transmitted, the global scale and the centre of the bounding volume, the scale and the angles applied during normalisation, and the barycentre removed at the beginning, which together allow the receiver to undo every transformation. That identifier serves a second purpose besides reconstruction, since it lets a trace taken on the wire be aligned with the record kept inside the node, so a block that never arrived can be told apart from one that arrived and was thrown away. Four payload sizes are available, of 1000, 1316, and 5000 bytes and one left to the stream itself. The second of them deserves a remark, since $1316 = 7 \times 188$ extracts whole container packets, and the resulting datagram of 1392 bytes, once the custom header and the network headers are counted, stays below the 1500-byte limit of the link. The smallest size breaks that alignment on purpose and leaves the receiver to reassemble the pieces, whereas the largest reaches some 5076 bytes and forces the interface to fragment, which is how the cost of fragmentation and the loss it invites enter the measurements, one missing fragment at a time.

3.5 Reconstruction and Control at the Client

The terminal undoes what the previous module built and, at the same time, decides what the chain will do next. Figure 3.8 follows a frame from the arrival of its datagrams to the display, and shows the two channels through which the viewer

reaches back into the modules upstream. Both duties share the same loop, so an interval spent on reconstruction is one in which a command cannot yet be acted upon, which is why the stages below are separated by function and measured one at a time. The last of them is also the only stage in the chain whose output leaves the pipeline towards a person rather than towards another node. Unlike the modules that precede it, the terminal has nobody to hand its work on to, and whatever it fails to finish inside the interval of a frame reaches the eye late or not at all, which makes its budget the tightest of the four, and the one on which the impression of an unbroken motion finally depends, whatever happens upstream.

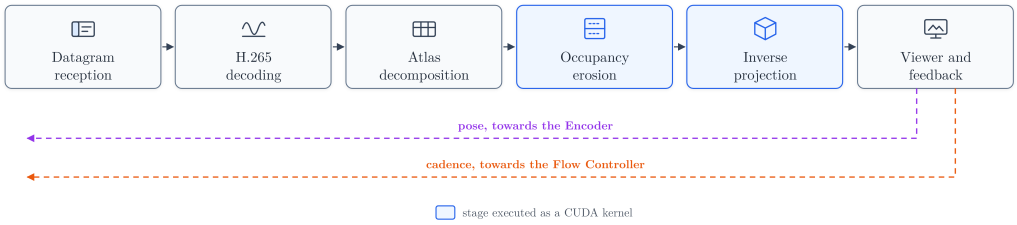


Figure 3.8. Stages and feedback pathways of the Client

3.5.1 Arrival and Decoding

Reception is divided between two threads. The first does nothing but drain the socket at the pace the operating system delivers, since what is left waiting there nobody will send again, and the second strips the custom header, keeps the metadata it carries, and passes the compressed payload to a hardware-assisted decoder through a pipe. A momentary slowdown of the second therefore never reaches the first. That stage runs outside the application, so a fluctuation in the time it takes propagates neither towards the socket nor from it. The thread that feeds it also collects the pictures that come back, joins each recovered picture with the metadata of the frame it belongs to, and places the pair on a queue, from which reconstruction draws. What emerges from the buffer is what the Encoder assembled, apart from the loss that compression introduced between the two ends of the link.

3.5.2 Unpacking the Composed Picture

The phase that follows stops treating the picture as an image. The luminance plane is cut into three equal bands, whose boundaries derive from the padded dimensions of a face alone, and each band is read according to what it holds. Depth and occupancy come from the outer two and need nothing else, since their colour-difference samples were never given a meaning, and ignoring those planes spares the memory traffic that

reading them would cost. Appearance instead requires the central band together with the two subsampled planes, which are addressed at halved coordinates. All three retain the arrangement of six views described earlier, so a face is located by arithmetic rather than by search, and the margin that alignment added around each view is discarded at this point, so that no point is ever rebuilt from a region the Encoder filled with black. What a frame yields is therefore bounded by the active area of the six views and not by the size of the picture.

3.5.3 Occupancy Filtering

Quantisation leaves the silhouette of the subject surrounded by a halo, where pixels that the Encoder had left empty acquire intermediate values, as the profile in Figure 3.9 illustrates. Such an edge is a step of the largest amplitude the map can carry, and the transform that represents it spreads part of that energy over the neighbouring samples, so the coder returns a gradient where the source held a sharp discontinuity. Rebuilding those pixels would scatter isolated points around the surface, so the outline is shrunk by one pixel with a 2×2 morphological filter applied to the map before anything is reconstructed [20]. The filtered mask then becomes the validity reference used to decide which decoded locations are allowed to produce geometry. This confines the correction to the affected contour while leaving the interior of the occupied region available for reconstruction.

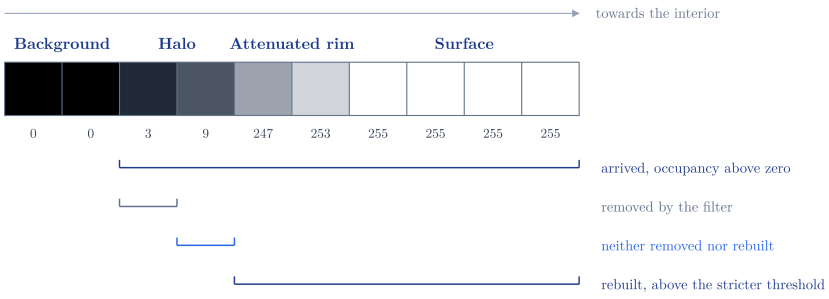


Figure 3.9. Occupancy across the border of the subject

Three counts follow the population of a frame through this stage. The arrived points are those whose occupancy exceeds zero in the map as received, the points removed by the filter are the difference between that count and the one obtained from the eroded map under the same loose test, and the valid points are those that reconstruction admits under a stricter threshold, tuned so that the halo is excluded while genuine surface survives. Because the last two counts submit the same filtered map to different tests, a residual close to 5 % of the arrived population

is neither removed nor rebuilt, and this fraction follows from applying two thresholds in sequence rather than from any error of measurement in the node.

3.5.4 Inverse Projection and Restitution

A pixel earns a point when its occupancy clears the stricter threshold, one that sits below the value the Encoder wrote, because compression no longer returns it intact, and insisting on the original level would discard surface that is perfectly sound. The attenuated rim of Figure 3.9 gathers precisely the pixels that such a requirement would throw away, and they belong to the outline of the reconstructed region rather than to the halo beyond it. The depth of the pixel is read from the geometry band and returned to the unit interval by a division,

$$d = \frac{Y_{\mathrm{geo}}(m, n)}{255}, \tag{3.11}$$

after which the position on the face supplies the two remaining coordinates and the metadata carried by the datagrams restore the metric frame,

$$\begin{aligned} X &= \left(n_x - \tfrac{1}{2}\right) M \, S_{\mathrm{global}} + C_x, \\ Y &= \left(n_y - \tfrac{1}{2}\right) N \, S_{\mathrm{global}} + C_y, \\ Z &= \left(n_z - \tfrac{1}{2}\right) M \, S_{\mathrm{global}} + C_z, \end{aligned} \tag{3.12}$$

with M and N the dimensions of a face and with the scale and the centre taken from the header of the datagram that delivered the frame. Appearance is restored beside geometry, reading luminance from the central band at full resolution and the two colour-difference values at half that, from the planes that follow it, and returning to the components in which the source had expressed colour, through the inverse of the conversion that the Encoder carried out upstream,

$$\begin{aligned} R &= Y + 1.402\,(V - 128), \\ G &= Y - 0.344\,(U - 128) - 0.714\,(V - 128), \\ B &= Y + 1.772\,(U - 128). \end{aligned} \tag{3.13}$$

Position and appearance therefore emerge together, and the rebuilt cloud matches the scale and the look of the original capture, net of what the coder discarded. The work is done per pixel and without dependence between pixels, so the stage grows with the number of survivors and not with the size of the picture. Colour returns coarser than it left, since the two planes were halved in each direction before transport, so neighbouring points may inherit one and the same pair of values, a loss that geometry, carried at full resolution, never suffers. Reconstruction keeps validity

apart from attribute recovery, since occupancy decides whether a point exists, while geometry and colour settle its location and appearance.

3.5.5 Interaction and Display

The terminal keeps two reliable connections open for the whole session, one towards the Encoder and one towards the Flow Controller, and a listener that never blocks the reconstruction loop turns keystrokes into commands for either of them. Pitch, yaw, and zoom travel along the first and reach the projection of the next frame, whereas the keys that step through the sequence or restore its nominal cadence take the second. Before presentation, a light transformation removes the framing introduced upstream, namely the push along the depth axis and the automatic scaling, while preserving the orientation the viewer asked for, so that the cloud appears at its natural size in a generic viewer. The result is handed to a background server that publishes it to a browser page over a **WebSocket** connection, and only the most recent frame is ever sent, older ones being dropped rather than queued [17]. A **backlog** would otherwise distort both what the viewer sees and the delay attributed to the chain, and the page therefore remains a channel for inspection rather than a stage that the measurements of the node have to account for.

3.6 Instrumentation and Metrics

Every module is observed in the same way, and that uniformity is what makes the four of them comparable at all. Measurements are taken inside the processing loop but never written from it, being handed to a background writer that accumulates them for the whole session and turns them into a single table once streaming has ended, so that recording cannot perturb the timing it describes.

3.6.1 Quantities Valid Across the Chain

Four definitions hold everywhere. Residency is the interval between the entry and the exit of a frame, and summing it along the route gives the delay perceived at the terminal. Efficiency is the share of that duration spent on useful work rather than on waiting, with the operations that count as such depending on what the module does. Throughput is obtained frame by frame, as the bytes received divided by the time actually spent receiving them, rather than as an average over the whole session. Scheduling delay compares the moment at which a frame is taken up with the one the cadence upstream prescribed, and it is defined only where a schedule is inherited rather than set, which leaves it without meaning at the Camera. None of the four

describes what the frame looks like once rebuilt, which the counts of the terminal and the measures of the evaluation establish instead.

3.6.2 Measurements Bound to a Single Role

Each module adds what its own duties make meaningful. The Camera records the latency of every read from storage, the size of each payload, the time spent putting it on the link, and the rate that results, and when the binary material is prepared during the session rather than beforehand, a separate record keeps the cost of each conversion. The Flow Controller separates the wait for a header from the work that follows it, reports an effective rate that falls as admitted frames become rarer, and logs the irregularity of the arrivals together with the delay of the link that feeds it, which describes the stream it receives before any decision of its own alters it. The Encoder isolates normalisation and projection from the compression that follows, so that the cost of geometry and that of coding never appear as one figure, and when the comparison is enabled it weighs each picture against an uncompressed reference through **PSNR** and **SSIM**. The Client divides its cycle into decoding, unpacking, filtering, and reconstruction, and measures the interval between a command and the first frame that visibly obeys it, which spans the control channel, the projection recomputed upstream, and the return of the video, and is the only quantity in the chain that reflects a round trip rather than one direction of travel.

3.6.3 Fusion and Reading of the Records

Once the session is over, the logs of the four modules are joined on the identifier of the frame into a single table, and nothing measured is discarded on the way, so that the life of a frame can be followed from the read at the source to the point rebuilt at the terminal. Two precautions govern how that table is then read. Intervals that run in sequence are distinguished from intervals that overlap, so that service times are never inflated by work carried out in parallel, and statistics are reported over explicitly delimited windows that leave out the warm-up and the isolated stalls, with the median, the **Interquartile Range** (**IQR**), and the percentiles beside the averages, so that the distortion a window would otherwise hide remains visible. Read in this way, the table separates what a module computes from what the link costs, and it sets the delay against the budget the nominal cadence allows, which is the threshold on which the **QoE** of an interactive service depends. That same table is the ground truth against which the platform described in Chapter 4 is compared, quantity by quantity, node by node, and frame by frame.

Chapter 4

Methodology

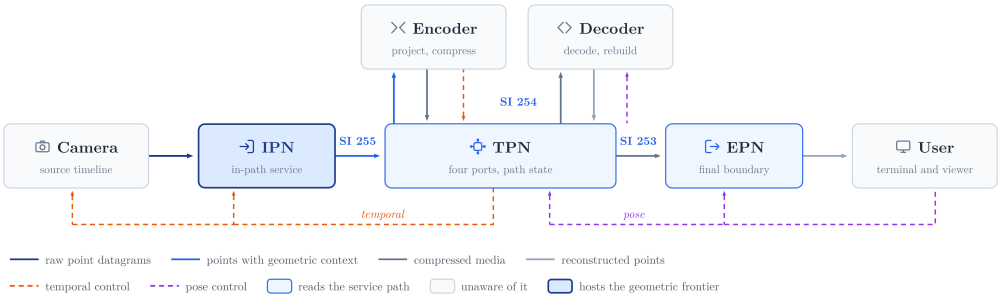
4.1 Anatomy of the Redistributed Graph

The chain reconstructed in Chapter 3 performs every operation inside four application processes, and the network between them carries bytes without inspecting them. This chapter describes the platform built for the present study, in which the same volumetric workflow is redistributed over seven independently placed units joined by a software forwarding path that reads, augments, and rewrites the traffic it relays. The account proceeds unit by unit, following the order in which a frame encounters them, and states for each one the data it accepts, the state it keeps, the arithmetic it performs, the parameters that govern it, and the observables it exports. Two properties of the redesign are established first, because every later section depends on them. The first is the shape of the graph itself, namely which functions became separate placement units and how they are joined. The second is the contract that travels with each datagram, which fixes how a unit recognises the traffic it must serve and how the path records the progress already made. Sources, build recipes, and the specification on which the present account rests are available in the public repository of the project [9]. The measurements that exercise this platform, together with the conditions under which they were gathered, belong to Chapter 5 and are kept outside an account concerned with the design alone.

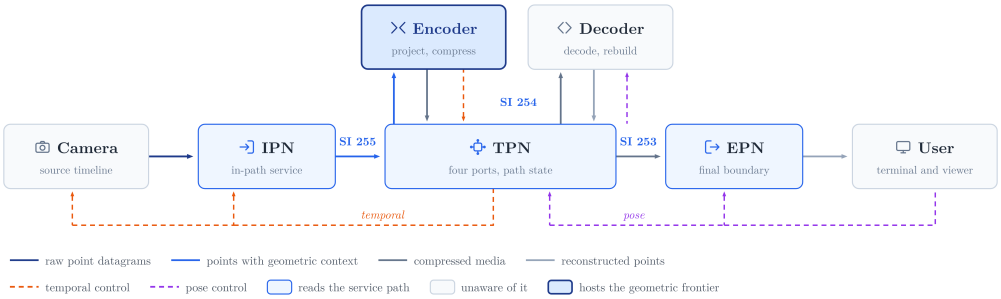
4.1.1 Placement Units and Their Adjacencies

Seven processes replace the four services of the reference. The **Camera** reads the prepared sequence and injects it on an absolute timeline. The **IPN** classifies that traffic, performs the geometric treatment examined in this work, and imposes the service envelope. The **TPN** is a four-port hub holding the path state of the two application units that ignore it. The **Encoder** assembles complete frames and

produces compressed media, the **Decoder** restores the cloud and orients it as the viewer asked, the **EPN** closes the forward route and opens the reverse one, and the **User** validates the terminal stream and drives the presentation. The three names that designate a forwarding unit are labels of placement rather than assertions that a process corresponds to one abstract role of the chaining architecture. The ingress unit combines classification, forwarding, and a service function of its own that reads the path, the hub joins forwarding to the proxy duty owed to the two unaware programs, and the egress unit terminates the forward route while classifying the commands that open the reverse one. A forwarding path realised in software invites that co-location, and the borrowed vocabulary is retained because it describes the placement accurately, not because process boundaries have to mirror those of the abstraction. Every unit executes inside its own **Docker** container and owns a private **DPDK** runtime, initialised through an **EAL** instance whose file prefix isolates its **hugepage** files from those of its neighbours, with 256 MB of hugepages reserved in a single backing store and no probe of any physical adapter, in the form that Section 4.9.4 reproduces [12, 13, 65]. Adjacency is therefore purely virtual. Each link is a **Unix**-domain socket on which one endpoint instantiates a `net_vhost` device with a single queue pair, while its peer attaches a `net_virtio_user` device whose virtqueue holds 4096 entries and whose hardware address is fixed [55, 63]. The four application units invariably take the second of those positions, the TPN the first on each of its four ports, and the remaining pair of forwarding units one of each. A hop is thus a shared-memory channel whose acceptance behaviour the sender observes directly, and six of them form the topology with no switching tier in between. The count of seven is not arbitrary. Each of the four application concerns of the reference, namely production, compression, reconstruction, and presentation, survives as one process, while the three units placed around them answer to the three moments at which the redesign decides something about a packet instead of about a frame, these being the entry to the path, its suspension, and its exit. A finer decomposition would have created units with nothing to decide, and a coarser one would have buried a decision inside a program that also performs arithmetic. Such a hop costs one copy and one descriptor exchange, with no adapter, no driver interrupt, and no kernel traversal. That property is deliberate. It holds the transport term near a floor rather than making it representative of a datacentre fabric, which leaves the arithmetic of each unit as the dominant contribution and keeps a change of placement from being masked by time on the wire. Figure 4.1 shows the resulting graph in its two principal build states, distinguishing the units that work on the packet as it passes from those that work on the assembled frame, and marking the five classes of traffic its links carry, the media and the two return loops.



(a) Geometry resolved in path



(b) Geometry resolved at the Encoder

Figure 4.1. Placement graph of the proposed platform

The address plan is a closed testbed contract rather than a routed deployment, and Table 4.1 records it in full. Every native unit builds and validates its own **Ethernet**, **Internet Protocol version 4** (**IPv4**), and **UDP** headers, recomputing the network-layer checksum through `rte_ipv4_cksum()` whenever a header is generated or rewritten. The transport checksum is deliberately left at zero, a convention that **RFC 768** permits and that the closed topology together with the explicit application validation renders acceptable here, although it would not be defensible on a public path [15, 51]. The addressing itself encodes the link rather than a host. The third octet of every address numbers the adjacency from one to six, the hardware address of each endpoint repeats that number in its fifth byte, and a value of 254 in the final octet marks the forwarding side of a link that faces an application, so a datagram captured anywhere can be placed on its segment by inspection alone. Forwarding decisions never consult a routing table. No address resolution runs anywhere in the topology either, since every unit carries the hardware address of its peer as a compile-time constant, and a frame whose link-layer type is not the one the network protocol in use declares is discarded before any further field is examined. A unit accepts a datagram only when the ingress port, the address pair, the port pair, and the payload length agree with the single contract defined for that adjacency, so an unexpected packet is released rather than forwarded.

Adjacency	Endpoints	Port	Socket
Camera and IPN	10.0.1.1, 10.0.1.254	5001, 6633	`/tmp/sfc-cam-sff1`
IPN and TPN	10.0.2.1, 10.0.2.2	6633, 6633	`/tmp/sfc-sff1-sff2`
TPN and Encoder	10.0.3.254, 10.0.3.1	6633, 7001	`/tmp/sfc-sff2-enc`
TPN and Decoder	10.0.4.254, 10.0.4.1	6633, 8001	`/tmp/sfc-sff2-dec`
TPN and EPN	10.0.5.2, 10.0.5.1	6633, 6633	`/tmp/sfc-sff2-sff3`
EPN and User	10.0.6.254, 10.0.6.1	6633, 9001	`/tmp/sfc-sff3-usr`

Table 4.1. Endpoint contract for the direct channels

Three of the seven units interpret the service envelope, and four ignore it. That asymmetry is intentional. Keeping the Encoder and the Decoder unaware of the chaining protocol prevents an application parser from acquiring routing duties, and it lets the same two programs be exercised with or without the surrounding path. The TPN absorbs the difference by removing the envelope before those units and restoring it afterwards, so the progress of a frame along the route remains recorded even while it is served by a function that cannot read it.

4.1.2 State Carried Along the Path

Three service paths are defined, each with its own identifier, and every datagram that crosses an aware boundary carries an eight-byte base header modelled on the **NSH** [22, 53]. Its leading sixteen-bit word packs a **Time To Live** (**TTL**) of six bits, set to 63 where the envelope is imposed and lowered at every aware hop, beside a length of six bits counted in four-byte units, while the positions of highest order, which the specification reserves for a version number and two flags, are left clear. One byte then names the kind of metadata that follows and one the protocol carried after it, which the experiment fixes at `0xFE`, and a concluding word joins a twenty-four-bit **SPI** to an eight-bit **SI**. The forward route uses `MAIN_SPI = 100`, and its index descends from 255 to 253 as the two unaware functions are traversed, so the value observed at the EPN states how much of the chain a frame has already crossed. The reverse **Temporal** path uses `TEMPORAL_SPI = 200` and the reverse **Pose** path `POSE_SPI = 300`, both at a constant index of 255, since neither traverses an intermediate function. On the forward route alone the base header is followed by a context descriptor of the **MD Type 2** form, whose metadata class is drawn from the experimental range at `0xFFF6`, whose type names the geometric payload, and whose seven-bit length announces the sixty-four bytes behind it, in which the IPN publishes the sixteen quantities it has resolved. The three structures amount to seventy-six bytes, a figure the base header advertises as nineteen words. Not every element of the record travels on the wire, however. Wherever the envelope has to

be withdrawn, the TPN retains one slot per frame identifier holding the index and the hop count observed at the instant of withdrawal, and it rebuilds the header from that slot when the traffic re-enters the aware portion of the chain. The two representations describe one progression from opposite sides, the header for the units able to read it and the slot for the interval in which none could. That table is dimensioned once from the declared length of the run, and a slot is written at most once per transition, so nothing accumulates while the session proceeds. Admission rests on agreement rather than on inference. A datagram is accepted only when the ingress port, the identifier pair expected there, the exact width of the payload, and the fields of the descriptor where one is present hold together, and whatever fails any of those tests is released instead of being repaired. Fixing the route, the shape of the packet, the admissible transition, and the criterion of completion in the source itself is what keeps the comparison of the later chapters interpretable, since none of those choices could be recovered afterwards from a capture of the traffic alone. The wire layout is a closed project protocol, and the vocabulary borrowed from the chaining architecture describes it without claiming interoperability, because no field it defines answers to any authority outside the project itself.

4.1.3 Protocol Stack and Datagram Budget

The payload assumes four distinct forms along the route, each with a header stack of its own, distributed over the six segments that Figure 4.2 sets out. A source datagram carries the forty-byte `cam_hdr` followed by at most `POINTS_PER_PACKET` records of sixteen bytes each, a capacity the campaign fixes at eighty. Once the IPN has served it, the same application bytes travel behind the base header, the context descriptor, and the geometric block. The TPN removes the first two before the Encoder but preserves the third, so the compression unit receives the geometry as ordinary application data. Compressed media leaves the Encoder as the `cam_hdr`, a forty-eight-byte `enc_hdr`, and seven transport-stream packets of 188 bytes, while the rebuilt cloud leaves the Decoder as a fifty-two-byte `dec_hdr` followed by point records again, which the hub carries behind the base header alone over the single segment that separates it from the egress unit. Every one of these forms is dimensioned against a conventional **Maximum Transmission Unit** (**MTU**) of 1500 bytes, and Table 4.2 gives the resulting sizes. No datagram approaches the limit, so no application message is ever fragmented by the network layer, and each unit checks its largest constructed packet against `NETWORK_MTU` at start-up rather than trusting the arithmetic to remain valid after a parameter change, which the check catches before the first datagram of a run is built.

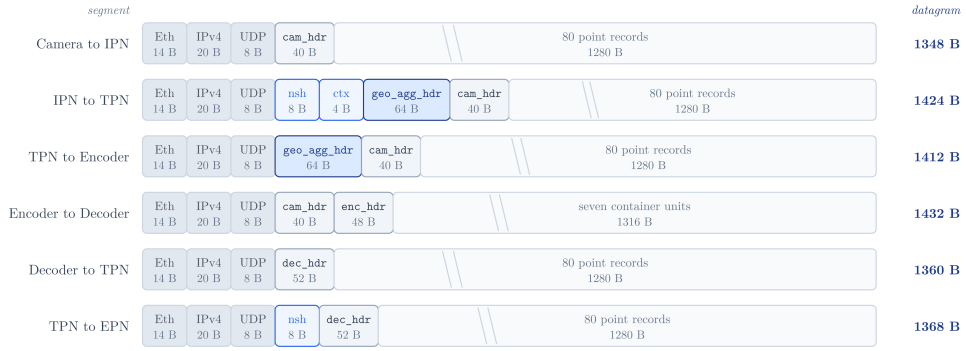


Figure 4.2. Protocol stacks of the transported forms

The two commands are far smaller and are deliberately fixed in width. A regulation message is a sixteen-byte structure holding a frame reference, a timestamp, and the requested factor, while a viewing message is a twenty-four-byte structure holding a timestamp and three angular or scaling components. Because each command occupies exactly one datagram of a known length, the aware nodes classify it from the ingress port, the payload length, and the protocol fields alone, without reassembling anything. That property is what makes a datagram service the natural carrier here, whereas the twelve bytes it spares against the smallest header a reliable transport could present weigh almost nothing at the rate these loops sustain. Both of them convey an absolute desired state rather than a sequence of increments, so the useful guarantee is that the most recent request eventually prevails, which the application obtains by re-presenting an unresolved request and by confirming it against the returning stream, and not by asking the transport to preserve an ordered history of intentions that a later message has already superseded.

Segment	Application bytes	Datagram	Margin
Camera to IPN	$40 + 1280$	1348	152
IPN to TPN	$76 + 40 + 1280$	1424	76
TPN to Encoder	$64 + 40 + 1280$	1412	88
Encoder to Decoder	$40 + 48 + 1316$	1432	68
Decoder to TPN	$52 + 1280$	1360	140
TPN to EPN	$8 + 52 + 1280$	1368	132

Table 4.2. Largest datagram of each segment, in bytes

Sizing the receive and transmit rings is a separate decision, and the platform normalises every call to `rte_eth_rx_queue_setup()` and `rte_eth_tx_queue_setup()` at 4096 descriptors, matching the virtqueue capacity declared on the socket. Each unit draws its buffers from a pool of 16383 elements with a per-core cache of 256,

whose data room sums the reserved **headroom**, the configured MTU, the link-layer header, and a further sixty-four bytes, so that the largest datagram the unit can build always fits in one segment. Traffic moves in bursts of thirty-two packets. A transmit call that accepts nothing is treated as local pressure rather than as loss, and the sender retries with a pause whose length grows with the streak, up to the ceiling of 2048 consecutive refusals that `MAX_ZERO_ACCEPTS` sets for one pending burst, after which the remaining buffers are released. Any positive acceptance resets the streak, so a frame may legitimately record a large cumulative count of refused attempts and still complete, which is why the tally means pressure, not loss.

4.1.4 Selecting the Site of the Geometric Frontier

The comparison at the centre of this work needs two builds of the same graph that differ in one respect only. Both are obtained at compile time, through one constant in the IPN and one in the Encoder, reproduced in Listing 4.1. With in-path processing enabled, the IPN performs the scans described in Section 4.3 and publishes the result, while the Encoder consumes it. With both constants disabled, the IPN keeps its classification, validation, envelope, relay, and accounting duties untouched, allocates no geometric workspace, and emits a context block whose active-point count is zero, which the Encoder reads as a declaration that nothing is available and answers by reconstructing the identical quantities locally, in the same order of operations. Nothing else changes between the two builds, neither the packet graph, nor the transport, nor the projection arithmetic, nor the regulation parameters. The frontier is therefore computed exactly once in either case, the definition applied is the same, and the only variable is the unit that evaluates it.

```
// IPN
#define NETWORK_PROCESSING_DISABLED 0
#define NETWORK_PROCESSING_ENABLED 1

#define NETWORK_PROCESSING NETWORK_PROCESSING_ENABLED

// Encoder
#define OFFLOAD_MODE_DISABLED 0
#define OFFLOAD_MODE_ENABLED 1

#define OFFLOAD_MODE OFFLOAD_MODE_ENABLED
```

Listing 4.1. Selection of the execution site

Making the choice at compile time rather than at run time is a deliberate measurement decision. A run-time branch would leave a test, and possibly a fallback probe, inside the per-packet path of both builds, so the two conditions would no longer differ

solely in the work performed. Because `OFFLOAD_MODE` is a constant, the stricter validation that the Encoder applies to an incoming context block is compiled away in the disabled build, which therefore accepts the structurally present but empty descriptor without paying for an additional check. The cost of that decision is that a placement change requires a rebuild, which the launch procedure performs for every unit before the process starts, so no stale binary survives.

4.2 The Source and Its Absolute Clock

The Camera is the only unit that creates work. Everything measured downstream is conditioned by the instant at which it releases a frame, by the number of datagrams into which that frame is cut, and by the residency policy applied to the material it reads, so its configuration is an experimental variable in its own right and not an implementation detail. It is also one of the two units whose behaviour a decision taken further along can alter while a run proceeds, and the only one where that change governs how much work the chain receives at all, since the factor that controls admission is derived by the compression unit and enforced here. Everything else it does is settled before the first frame leaves, by constants that the subsections below set out. It consumes the prepared sequence described in Section 4.9.5, a directory of header-less files in which each frame is a contiguous array of sixteen-byte records, opening `K_FRAMES` of them at start-up and refusing to proceed when one is missing or when its length is not a whole number of records, and it releases at a nominal `TARGET_FPS` of thirty those that survive the admission rule. Figure 4.3 follows one cycle of the resulting loop, from the poll that may change the admitted rate through to the last datagram that the local **ring** accepts.

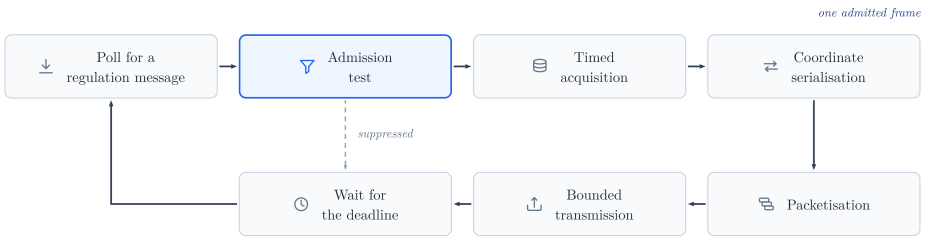


Figure 4.3. One cycle of the source loop

4.2.1 Residency of the Prepared Material

Where the bytes of a frame reside when the loop reaches it changes the cost of reading them, and the implementation exposes that choice explicitly. Under `CACHE_MODE_BEST` every frame is allocated, read, and converted to the wire representation

before streaming begins, so neither acquisition nor conversion survives inside the timed path, which then touches resident memory alone. Under `CACHE_MODE_MIDDLE` a single staging buffer, dimensioned on the largest frame of the sequence, is allocated in advance and reused, while the read of each frame stays inside the loop. Under `CACHE_MODE_WORST` both the allocation and the read occur per frame, and the release follows the transmission. A second constant governs page residency. With `WARM_MODE_ENABLED` every source file is mapped read-only and pinned before the measured sequence, the staging buffer being held beside them, yet that mapping is never the channel through which a frame is afterwards read, so the timed acquisition retains ordinary buffered semantics over a working set already present in memory. A fixed settling interval of five seconds then separates the end of the preparation from the first deadline, which keeps page pinning and device initialisation outside the schedule instead of inside its opening frames. The configuration adopted throughout this work pairs the middle mode with warm residency, which keeps the per-frame read boundary visible without letting physical storage latency dominate it. The three modes are distinct experimental conditions rather than alternative readings of one execution, so the first of them together with the last serve as sensitivity bounds rather than as the operating point of the campaign.

4.2.2 Deadlines and the Admitted Rate

A single origin is sampled once, before the first frame, and every subsequent deadline is derived from it rather than from the previous departure, so a late frame cannot displace the schedule of its successors. Writing T_0 for that origin, f_t for the nominal rate, and s for the factor currently in force, the source therefore obeys

$$T_\mathrm{ideal}(k) = T_0 + \frac{k}{f_\mathrm{t}}, \qquad \sigma(k) = \left[(k-1) \bmod s \equiv 0\right], \qquad f_\mathrm{eff} = \frac{f_\mathrm{t}}{s}, \tag{4.1}$$

where σ decides whether frame k is admitted and f_eff is the rate that admission implies. The instant so defined is the moment by which the iteration of that index must have closed rather than the moment at which its datagrams begin to leave, which is why the drift reported further along measures a completion against the timetable and not a release against it. Admitted and suppressed indices alike wait for their own deadline, so the session clock keeps its meaning whatever the factor, and an index the rule rejects still contributes a record stating that it was rejected. Both the deadline and the wait that honours it are counted in processor cycles, and that wait never yields the core. Before each decision the loop drains the reverse channel, discards whatever fails to match the addresses, the ports, and the declared lengths of the one contract defined for that adjacency, normalises a zero factor back

to unity, and adopts the request only when its frame reference is not older than the one already in force. That reference advances even where the requested factor equals the active one, whereas the delay between the issue of a command and its adoption is charged only where the factor actually moves, and the sentinel that closes the stream is never admitted as a reference at all. Suppression thus happens at the earliest useful boundary, since a frame that fails the test consumes neither acquisition, nor serialisation, nor packetisation, nor a single transmit descriptor. Listing 4.2 sets out the fragment that governs both the deadline and the admission test.

```
uint64_t frame_cycles = ( uint64_t )( timer_hz / TARGET_FPS );
uint64_t start_time = rte_get_timer_cycles();

for ( int frame = 0; frame < loaded_frames; frame++ ) {
    uint32_t frame_id = frame + 1;
    uint64_t expected_time = start_time + ( ( frame + 1 ) * frame_cycles );

    double frame_temporal_control_ms = process_temporal_control( timer_hz );

    uint16_t frame_temporal_skip = current_temporal_skip;

    if ( unlikely( frame_temporal_skip == 0 ) )
        frame_temporal_skip = 1;

    bool frame_selected = ( ( ( frame_id - 1 ) % frame_temporal_skip ) == 0 );

    if ( unlikely( !frame_selected ) ) {
        // Record of the suppressed index

        while ( rte_get_timer_cycles() < expected_time )
            rte_pause();

        continue;
    }

    // Acquisition, serialisation, packetisation, transmission
}
```

Listing 4.2. Absolute deadline and the admission rule

This arrangement places regulation upstream of the point at which the reference pipeline could act. There, the decision was taken after the source had already committed a frame to a reliable stream, and a rejected frame still had to be drained to preserve framing. Here the factor is applied before a single byte is packetised, so the relief is proportional to the suppressed population rather than to the fraction of the transfer that can be abandoned. No supplementary pacing heuristic is layered on top, because the absolute schedule already fixes the intended departure instants,

and the pressure that the local transmit path exerts is reported through dedicated counters instead of being hidden behind an adaptive delay.

4.2.3 Serialisation and Cutting of the Frame

The prepared files store coordinates as little-endian single-precision values, which is the natural host representation, whereas the wire contract requires the same bit patterns in network order. The conversion is performed in place over the staging buffer immediately before packetisation, one thirty-two-bit word per coordinate, while the three colour bytes and the trailing padding byte are byte-valued and therefore pass through unchanged. The anchor with which every later unit dates the frame is read between the two operations, once acquisition has finished and before the conversion begins, whereas the residence of the frame at this unit opens earlier still, so the two boundaries answer different questions and may not be treated as one. The frame is then cut into datagrams of at most `POINTS_PER_PACKET` records, which at sixteen bytes each yields 1280 payload bytes, and only the last datagram of a frame carries the natural remainder, for which alone the network and transport lengths and the header checksum are formed again, everything else being copied from a template prepared once. Every packet of one frame repeats the same forty-byte header, so any receiver can act on the first datagram it sees. That header announces the identifier of the frame, the sequence number of the packet, the source timestamp, a neutral stance, the factor in force, the declared population of the frame, the population of the current datagram, and, in the field retained for alignment, the configured capacity. The stance is neutral because the viewing command is honoured at the reconstruction unit, so the projected representation no longer depends on where the observer is looking, which is not how the chain of Chapter 3 was arranged. Advertising the capacity rather than assuming it lets a receiver derive the offset of a packet from its sequence number without a compiled-in constant, and a zero in that field is read only as a silent fallback to the value compiled locally.

4.2.4 Acceptance Pressure at the Local Ring

Datagrams are offered in groups, a group being submitted once it holds a full burst or once the frame has no further points to contribute. A transmit call reports how many descriptors were actually taken, which may be fewer than requested and may be none at all. The source does not treat a refusal as a loss, because nothing has left the host, and it does not treat a subsequent success as a retransmission. Only the buffers the interface refused are offered again, and none of them is rebuilt. It retries the pending group, pausing for a number of cycles that grows with the streak until

a ceiling of half a burst is reached, so that for a streak of R consecutive refusals the pause instructions that accumulate over the entire streak are then given by

$$N_{\mathrm{pause}}(R) = \sum_{r=1}^{R} \min\left(r, \frac{B}{2}\right), \tag{4.2}$$

with B denoting the burst width. A streak longer than `MAX_ZERO_ACCEPTS` abandons the pending buffers, and any positive acceptance resets the counter, which is why a frame can record a large total of refused attempts and still be delivered in full. Four counters separate the phenomena that are easily confused, namely the calls that accepted nothing, the calls that accepted less than requested, the re-presentation operations, and the packets those operations offered again. A fifth records failures to obtain a buffer from the pool, absent throughout the validated executions. Because roughly ten thousand datagrams leave the source for each frame, this instrumentation is what allows local producer pressure to be told apart from what the receiver dropped. One consequence deserves emphasis before any figure of this unit is read. Waiting for a descriptor occupies elapsed time without the processor being inside a transmit call, so the span of egress and the time spent within those calls are recorded apart, and the aggregate that this unit labels as its own work embraces the former. Unlike the equivalent aggregate reported downstream, what this one states is a budget of occupancy rather than a measure of arithmetic alone.

4.2.5 Closing the Sequence

When the configured population has been emitted, the source builds a full burst of header-only control packets whose frame identifier is the reserved sentinel, thirty-two of them submitted together under the same bounded retry policy, with no point payload, with both population fields cleared, and with the factor in force at the close recorded in its usual place. The repetition protects the terminal condition against a transient refusal at the local ring, since a single copy could be abandoned by that policy while the stream was otherwise complete. The marker is a protocol event rather than an additional frame, so no row of the per-frame table is ever opened for it, and every downstream unit uses it to release the residual state it holds, to drain whatever its own subsystems still owe, and only then to serialise the observations it had been accumulating ever since the sequence opened.

4.3 Geometry Resolved While Forwarding

The IPN is the unit in which the claim of this thesis is put to work. It sits on the only link that carries raw samples, it sees every one of them exactly once, and it is

therefore the earliest place at which a quantity derived from the whole population can begin to take shape. Its design answers a narrow question, namely how much of the preparation that the compression unit would otherwise perform can be completed on material that is still in motion, and it answers that question by splitting the work depending on when each result becomes mathematically available rather than according to which program traditionally owns it. The split is not a matter of convenience. A quantity that a later packet could still change is worthless once it has been published as final, and a quantity that no later packet can change costs nothing to publish early, so the boundary between the two regimes is dictated by the data rather than chosen. What follows describes the two regimes it alternates between, sketched in Figure 4.4, together with the envelope it builds and with the reverse control traffic whose aware path ends here.

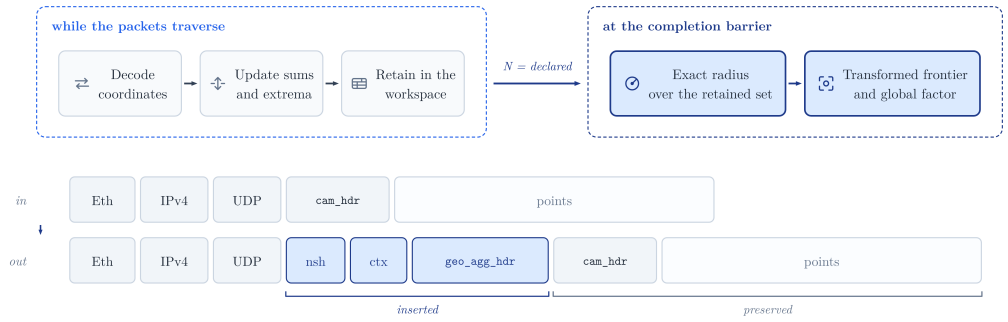


Figure 4.4. Regimes of the in-path service

4.3.1 Validation of a Source Datagram

Every arriving packet is examined before anything is derived from it. The link-layer type, the network-layer version and addresses, the transport ports, and the declared lengths are each compared against the single contract that the Camera adjacency admits, and the counters that record a rejection are kept apart by layer so that a malformed frame can be attributed rather than merely observed. Three further tests bind the application header to the bytes behind it, since the population announced for a datagram may not exceed the capacity the source advertises, the payload must measure exactly that population multiplied by the width of a record, and the advertised capacity may not change once a frame has begun. The header is then read, and the identifier it carries selects the frame-local state. That state describes one frame alone. An identifier other than the one in hand closes the frame that preceded it, releases whatever group of packets is still pending, and commits its record, after which the accumulators return to their initial values and the successor opens, so nothing retained here grows with the length of a session. Because the

header repeats in every datagram, the first packet of a frame suffices to open that state, to record the arrival instant used later as the residency origin, and to learn the declared population against which completion will be tested. Sequence numbers are expected to follow one another without a gap, and a frame counts as received in full only where the population the source declared, the number of datagrams it implies, and an unbroken sequence agree. Coordinates are recovered from network order one word at a time, and the colour bytes are left untouched, since the node has no use for them. The remaining work depends on the build, and the account that follows assumes that in-path processing has been left on.

4.3.2 Reduction Maintained Across Packets

Three running quantities are updated as points arrive, namely the component-wise sum, the component-wise minimum, and the component-wise maximum. Writing P_N for the prefix of N samples already observed, the node then maintains

$$S_N = \sum_{p \in P_N} p, \qquad p_N^{\min} = \min_{p \in P_N} p, \qquad p_N^{\max} = \max_{p \in P_N} p, \tag{4.3}$$

from which the axis-aligned **extent**, the **centroid**, and the **centre** of the raw **bounding box** follow immediately as $C_N = S_N / N$, as $E_N = p_N^{\max} - p_N^{\min}$, and as $B_N = (p_N^{\min} + p_N^{\max})/2$. Those three are formed again once every datagram has been absorbed rather than once at the end, so what a departing packet advertises is what the node holds at the instant it leaves. Extrema compose exactly under any grouping of the input. Sums do not, since finite-precision addition is not associative, so the accumulators are held in double precision and the packet order is treated as a reproducibility condition rather than assumed to be irrelevant. The wider accumulator is not a refinement, since a single-precision sum over a population of this size would lose its low-order bits long before the last packet arrived. Each decoded coordinate is also written into a workspace reserved once, before streaming, at the largest population the sequence contains, and touched at allocation so that no page fault can occur inside the measured path. That workspace occupies about ten megabytes, which is precisely the amount that places this function beyond a switching pipeline and within reach of a software unit. Two guards defend it, the first weighing the population a frame declares against the capacity reserved and the second weighing the running count together with the current datagram against the same bound, and either of them halts the process rather than allowing it to overrun. The inner loop that maintains those accumulators and fills the workspace, one datagram at a time, is the one reproduced in Listing 4.3.

```
for ( uint32_t point = 0; point < packet_points; point++ ) {
    float x = be_to_float( points[ point ].x );
    float y = be_to_float( points[ point ].y );
    float z = be_to_float( points[ point ].z );

    frame_geometry_points[ frame_point_count ].x = x;
    frame_geometry_points[ frame_point_count ].y = y;
    frame_geometry_points[ frame_point_count ].z = z;

    frame_sum_x += x;
    frame_sum_y += y;
    frame_sum_z += z;

    frame_point_count++;

    if ( x < min_x ) min_x = x;
    if ( x > max_x ) max_x = x;
    if ( y < min_y ) min_y = y;
    if ( y > max_y ) max_y = y;
    if ( z < min_z ) min_z = z;
    if ( z > max_z ) max_z = z;
}
```

Listing 4.3. Per-packet workspace retention

The cost of this pass is charged to a dedicated accumulator, so the time spent on geometry is separable from the time spent forwarding. Nothing here waits for the frame. Every datagram leaves the node with the best estimate available at that instant, which is why the descriptor attached to an early packet is a legitimate partial answer rather than a placeholder, and why the compression unit must select among the descriptors it receives instead of trusting the first.

4.3.3 Resolution at the Completion Barrier

Two quantities cannot be produced this way, and the reason is structural rather than technical. The radius that encloses the subject is measured from the centroid, and the centroid is not final until the last sample has arrived, so a single pass cannot deliver it. The same applies to the frontier of the transformed cloud, which depends on the scaling factor that the radius determines. When the received population equals the declared one, the node therefore performs two further passes over the coordinates it retained in transit. The first of the two passes evaluates

$$r_{\max} = \max_{p \in P} \left\| p - C \right\|_2 , \tag{4.4}$$

taking the square root per point rather than comparing squared distances, which keeps the numerical sequence aligned with the one the reference implementation follows. The second applies the resulting scale and reduces the transformed coordinates, so that with D denoting the fixed distance of the virtual observer, W and H for the width and the height of a projected face, and π for the fractional margin reserved around the subject, the node obtains the remaining quantities of the frontier through

$$f = \frac{0.2\,D}{r_{\max}}, \qquad p' = (p - C) f + (0, 0, D), \qquad g = (1+\pi)\max\left(\frac{E'_x}{W}, \frac{E'_y}{H}, \frac{E'_z}{W}\right), \tag{4.5}$$

together with the centre $B' = (p'^{\min} + p'^{\max})/2$ of the transformed bounding box. The constants are fixed at a distance of 1200 units, at a face of 640 by 480 samples, and at a margin of ten per cent. Listing 4.4 gives the reduction that produces the last two of these values. A non-finite or non-positive outcome falls back to unity, so a degenerate frame cannot propagate an unusable scale downstream. Only the radius is timed on its own, while the pass that follows joins the accumulator already carrying the per-packet work, and the quantity the measurement isolates is therefore the price of the dependency rather than of the placement.

```
float final_scale = 1.0f;

if ( isfinite( max_r ) && max_r > 0.0f )
    final_scale = ( CAMERA_DISTANCE * 0.2f ) / max_r;

for ( uint32_t point = 0; point < point_count; point++ ) {
    float tx = ( points[ point ].x - centroid_x ) * final_scale;
    float ty = ( points[ point ].y - centroid_y ) * final_scale;
    float tz = ( points[ point ].z - centroid_z ) * final_scale + CAMERA_DISTANCE;

    if ( tx < min_x ) min_x = tx;
    if ( tx > max_x ) max_x = tx;
    // Remaining components
}

float scale_x = ( max_x - min_x ) / ( float )WIDTH;
float scale_y = ( max_y - min_y ) / ( float )HEIGHT;
float scale_z = ( max_z - min_z ) / ( float )WIDTH;

float global_scale = fmaxf( ...( scale_x, scale_y ), ..._z ) * ( 1.0f + PADDING );
```

Listing 4.4. Frame-complete reduction of the transformed boundary

The choice to resolve the frontier here, and not merely the raw statistics, is what turns a diagnostic aggregation into an **offload**. Had the node published only sums and extrema, the compression unit would still have had to traverse its own copy of the

cloud twice, and the relocation would have saved one pass out of three. Publishing the scale, the transformed centre, and the global factor leaves the receiving unit with nothing to reduce, so the entire preparation collapses into reading sixteen fields from a header. What remains downstream is precisely what cannot be anticipated, namely the work that depends on the final reconstruction parameters, on visibility between points, on the requested stance, or on the coder. That the **barrier** exists at all is meanwhile a property of the arithmetic and not a shortcoming of the unit, since no layout of the packet format can make a centroid final before its last contributor has arrived. Producing the frontier offline and letting every datagram carry a value already known to be complete would have been feasible, yet what that arrangement measures is an offline result delivered by the network rather than one obtained on the path, so the deferred step is left where the dependency places it, and the platform is measured with that boundary kept in plain view throughout.

4.3.4 Replacement of the Envelope in Place

The node does not build a second packet. It removes the outer network headers of the arriving datagram, prepends its own link, network, and transport headers followed by the base header, the descriptor, and the sixty-four-byte block, and leaves the application bytes exactly where they were. The three inserted structures occupy seventy-six bytes, which the base header advertises as nineteen four-byte words, and the transport and network lengths are recomputed from the preserved inner length before the network checksum is cleared and generated again. The result is that service metadata travels with the payload it describes, on the same datagram and in the same order, instead of being sent as a separate message that a receiver would have to correlate. Every packet carries the descriptor current at the moment it left, so the sequence the compression unit observes is monotone in the population it reflects, and the last one of them is complete by construction. Groups are handed to the interface at thirty-two packets or as soon as a frame has been received in full, whichever falls first, so the closing datagram of a frame never waits behind one belonging to another. The marker that ends the stream crosses the unit in the same manner, dressed in the identical envelope with every field of the descriptor cleared, and its arrival is also what closes the record this unit keeps.

4.3.5 Termination of the Reverse Regulation Path

The same process is the last aware hop of the loop that travels the other way. A regulation message arrives from the TPN behind the base header of its own path, and the node admits it only where the version field reads zero, the flag reserved for maintenance traffic is clear, the advertised length is that of a bare base header, the

type of the metadata and the protocol behind it are the ones the experiment fixes, the transport length matches that header together with the sixteen bytes behind it, and the identifier pair is the one the regulation path owns. It then strips the envelope, rewrites the outer headers towards the Camera, and relays the structure untouched. The hop counter is read as part of that examination but never lowered, since the header holding it does not travel beyond this point. No decision is taken here either. The factor observed on the forward path is the one the source has already adopted, so within this node it is an item of context that qualifies the effective rate, never a second opportunity to discard a frame. Concentrating the decision at the unit that measures its own load, and the enforcement at the unit that owns the timeline, keeps the regulation loop free of intermediate policy and makes the returning header the only evidence that a request has genuinely taken effect.

4.3.6 The Node Without Its Computation

In the disabled build the process keeps every duty except the geometric one. It still validates, still classifies, still inserts the base header together with its descriptor, still forwards the application payload unchanged, still terminates the reverse path, and still exports its timing, integrity, and acceptance counters. What disappears is the per-packet decoding of coordinates, the running reduction, the retention of the workspace, which is not even allocated, and the two passes at the barrier. The descriptor is emitted with every field cleared, and its active-point count of zero is the agreed signal that no geometry accompanies the frame. This is the arrangement that makes the comparison fair, because the forwarding graph, the number of datagrams, the header sizes, and the service state are identical in both builds, and the only difference in the work performed is the presence or the absence of the scans whose placement the remaining chapters set out to examine.

4.4 The Hub Around the Unaware Functions

Two of the seven units are ordinary applications that know nothing of the path they belong to. Preserving that ignorance is a deliberate architectural choice, since it keeps the projection and reconstruction programs free of routing responsibilities and allows either of them to be replaced without touching the chaining logic. The consequence is that somebody must hold the state they cannot hold, and the TPN is that somebody. It owns four ports, serves three transitions of the forward route and the two reverse chains, and is the only unit that both removes and restores a service envelope. Figure 4.5 lays out the five flows it handles, the ports on which they arrive, and the index progression it maintains across them.

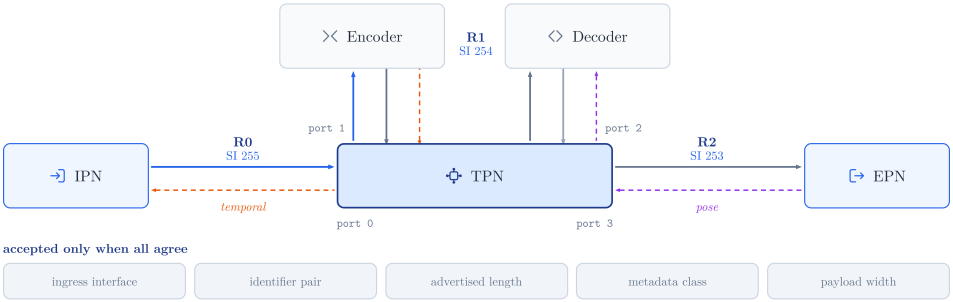


Figure 4.5. Flows and index progression at the transit hub

4.4.1 Ports, Routes, and Payload Semantics

The four interfaces face the IPN, the Encoder, the Decoder, and the EPN in that order, and three route identifiers name the forward transitions between them. The first carries point datagrams from the ingress node towards compression, the second conveys compressed media towards reconstruction, and the third carries rebuilt points towards the egress node. Each route interprets the application bytes differently, and the accounting follows that interpretation. On the first and third routes the node counts points and derives an integrity ratio against the declared population, whereas on the second it counts media bytes, because a compressed frame has no point population to compare. Timing anchors, cycle occupancy, arrival spacing, jitter, and acceptance counters are kept per route as well, so that three independent records describe what is physically the same process. Separating them is what allows a pressure observed at one transition to be attributed to that transition rather than to the node taken as a whole, which a single aggregated record covering all three of them could never have expressed on its own at all.

4.4.2 Classification by Ingress and Shape

No table lookup decides where a packet goes. The decision is made from the interface on which it arrived, the identifier pair it carries when one is present, and the exact length of the structure behind the transport header. A datagram entering from the ingress node is accepted on the forward route only when its identifier pair is the expected one, when the advertised header length matches the seventy-six bytes the geometric form requires, when the metadata class is taken from the experimental range, and when the descriptor announces a payload of exactly sixty-four bytes. A twenty-four-byte command entering from the egress node is accepted on the reverse viewing path only when the base header stands alone and the total payload equals the two lengths combined. Listing 4.5 reproduces the two tests. Anything that

satisfies neither is released without being forwarded. The hop counter is lowered once per frame and per transition rather than once for every packet, and a datagram belonging to a transition already applied leaves it untouched, so a repeated copy can neither advance the index twice nor spend a budget the frame has already spent, while a context down to its last hop refuses the transition outright.

```
if ( spi == MAIN_SPI && si == MAIN_SI_SFF1 && rx_port == PORT_SFF1 ) {
    if ( unlikely( nsh_length != MAIN_GEOMETRY_SIZE ) )
        return INVALID;

    struct nsh_md2_ctx_hdr *geo_ctx = ( struct nsh_md2_ctx_hdr * )( nsh + 1 );

    if ( unlikely( geo_ctx -> metadata_class != rte_cpu_to_be_16(
    MD_CLASS_EXPERIMENTAL ) || geo_ctx -> type != MD_TYPE_GEOMETRY || ( geo_ctx ->
    u_length & 0x7F ) != sizeof( struct geo_agg_hdr ) ) )
        return INVALID;

    type = MAIN_SFF1_ENCODER;
    target_tx_port = PORT_ENCODER;
}
else if ( spi == POSE_SPI && si == POSE_SI && rx_port == PORT_SFF3 ) {
    if ( unlikely( nsh_length != sizeof( struct nsh_hdr ) || udp_payload_length !=
    nsh_length + sizeof( struct pose_payload ) ) )
        return INVALID;

    type = POSE_SFF3_DECODER;
    target_tx_port = PORT_DECODER;
}
```

Listing 4.5. Validation rules of the aware boundary

A command travelling the other way is recognised with equal economy. A sixteen-byte payload arriving bare from the compression unit is the regulation message, and the node wraps it in the base header that its own path defines before relaying it towards the ingress node. Nothing else in the system produces a bare payload of that width on that interface, so the width itself is the discriminator. This is the property that a byte-stream transport would have destroyed, since the boundary between two commands would then have had to be recovered by a framing layer, or by closing the connection, before any classification could begin.

4.4.3 Retention of the Index Across a Function

On the first transition the node captures the service state, removes the base header and the context descriptor, and hands the compression unit a plain datagram whose application payload begins with the geometric block. The block is preserved on

purpose, because it is application information rather than routing information, and the receiving program reads it as such. When the compressed result returns, the node advances the index to 254 and forwards a plain datagram to the reconstruction unit. When the rebuilt points return, it advances the index to 253, rebuilds the eight-byte base header, and forwards the result to the egress node. The chain therefore records, at every aware boundary, how many of its functions a given frame has already crossed, even though the two functions responsible for the heaviest work never saw that record. Restoration is exact rather than approximate. The identifier of the path is a constant of the experiment and the index follows from the transition being served, whereas the hop count is the one observed when the envelope was withdrawn, held in the slot that the frame identifier selects, so what leaves an unaware function is the header that entered it, lowered by the hops consumed.

4.4.4 Ownership of Buffered Bursts

Packets leaving the node are grouped before submission, and a single egress interface can receive traffic belonging to different routes within the same window. Merging them would corrupt the accounting, because the acceptance outcome of a burst would then be charged to whichever record happened to own it last. The implementation therefore attaches a route and a record to each pending burst and flushes the group whenever the next packet would belong to a different one. Refused packets stay in the burst and are offered again under the same bounded policy used elsewhere in the chain, while a streak that exceeds the ceiling releases them explicitly instead of leaking the buffers. Four conditions release a group, namely a full burst, the completion of a frame on either of the two transitions that carry points, the terminal marker, and an iteration in which no interface delivered anything, which is what stops a partial group from waiting behind an idle poll. Under the nominal condition no forward transition of this node experiences acceptance pressure at all, which is expected of a unit whose per-packet work is confined to validation and header rewriting, and that absence is recorded rather than assumed.

4.5 The Compression Unit as a Hybrid Function

Neither of the two idioms that the rest of this chapter relies on describes the Encoder adequately. It is not a passive application that waits for a complete object, because it converts and places coordinates as the datagrams carrying them arrive, keeps its receive path serviced while the accelerator is busy, and decouples the coder behind a queue. Nor is it a packet-local function, because the six-view representation and the coder input are defined over a whole frame and cannot be produced before the

last sample has been placed. It is therefore a frame-aware unit with a data-plane front end, and its internal organisation, shown in Figure 4.6, is arranged so that the only blocking boundary is the one mathematics imposes.

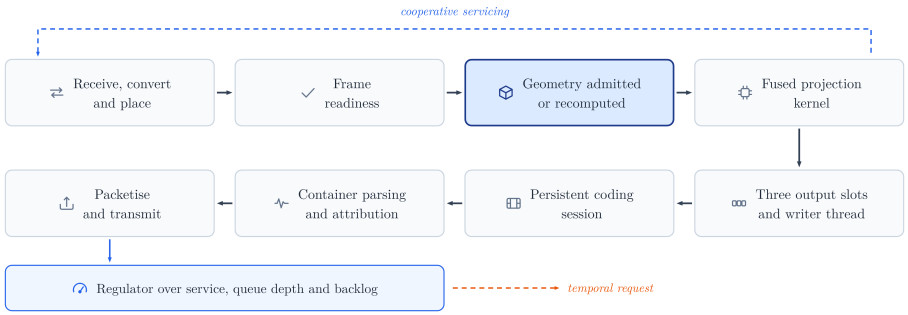


Figure 4.6. Structure of the compression unit

4.5.1 Assembly Conducted as Packets Arrive

Each datagram delivered by the hub begins with the geometric block, continues with the source header, and ends with its share of the cloud. The unit reads the identifier, selects the frame slot, and writes the converted coordinates at an offset obtained from the sequence number and the capacity the source advertises, so no reordering buffer and no second conversion pass are required. A bitmap records which datagrams have been seen, which distinguishes a duplicate from a fresh contribution and lets the population be counted without trusting the packet total. The first and last arrival instants are retained, since they delimit the interval against which the receive-side throughput of the frame is computed. A frame becomes eligible for projection only when the observed packet set and the received population both agree with the values the header declares, and an incomplete frame is carried forward until the terminal marker, at which point it may be compacted and processed rather than silently discarded. Work that does not depend on the whole frame is therefore already finished when that barrier is reached, so the wait covers only the operations that no rearrangement of the work could have started any earlier.

4.5.2 Admission of the Offloaded Frontier

Because the ingress node publishes a descriptor with every datagram, the unit receives a monotone sequence of partial answers and must decide which one, if any, describes the finished frame. It keeps the snapshot whose active-point count is the largest seen for that identifier, and at the barrier it admits that snapshot only when the count it declares equals the population actually assembled and when every

decoded component is finite and mutually consistent. A descriptor that announces more points than the frame contains is rejected at reception, before it can pollute the snapshot. This single equality test is what prevents an early partial answer from being mistaken for the final one, and it is also what makes the offload verifiable rather than assumed, since a frame whose descriptor fails it falls back to local computation and says so through its own timing fields. When the test succeeds, the unit adopts the centroid, the extent, the raw bounding-box centre, the radius, the scale, the global factor, and the projected centre without touching the point array again, and the two geometric timers of that frame consequently read zero.

4.5.3 The Geometry Without the Offload

In the disabled build the unit performs the work itself, and the implementation is deliberately the computational twin of the one described in Section 4.3 rather than a cheaper equivalent. It accumulates the coordinate sums in double precision together with the raw extrema in one pass, resolves the centroid, evaluates the radius with a square root per point against that centroid in a second pass, and applies the resulting scale in a third pass to obtain the transformed extrema, the projected centre, and the global factor. The order of the stages, the precision of the accumulators, and the use of a per-point square root all follow the formulation of Equations (4.3) to (4.5), so that the two builds evaluate the same function and differ only in where they do so. Both paths mark the result as ready and enter the projection stage through the same branch, which means that the accelerator receives an identical parameter set whichever unit produced it. A frame whose upstream descriptor is population-consistent but partial retains a compatibility route that reuses the supplied statistics and recomputes the radius over the compacted set, although no frame of the nominal executions takes it.

4.5.4 Device Memory and Cooperative Servicing

The projection stage never allocates. Device buffers for the point array, the depth arbitration grid, and the three attribute planes are reserved once, together with a dedicated stream and the event pair used for timing, and the host-side output slots are pinned and registered with the runtime so that transfers avoid an intermediate staging copy. Three such slots exist, which lets one frame be filled while another is being consumed by the coder writer and a third waits. The point array is moved to the device in chunks of `H2D_CHUNK_POINTS` records, and between two chunks the unit calls back into its own receive loop, so a transfer that would otherwise monopolise the core for the duration of a large frame is interleaved with packet servicing. The same callback is invoked repeatedly while the stream query reports outstanding

work. Cooperative polling of this kind does not shorten the accelerator interval, and it is not presented as an optimisation of it. Its purpose is to prevent the arrival of the next frame, or of a command travelling upstream, from being delayed merely because the current frame happens to be in the middle of its transfer.

4.5.5 The Fused Projection Kernel

One launch per frame performs the whole of the point-parallel work. A thread takes one sample, applies the object-centred transform that the ingress unit performs, converts its colour through the **luminance** and **chrominance** relations recalled in Equation (3.7), and then projects the result onto the six faces of the enclosing volume in turn [30]. The normalised coordinates that address a face follow from the point, the projected bounding-box centre, and the factor established earlier, as

$$n_x = \frac{t_x - B'_x}{W g} + \frac{1}{2}, \qquad n_y = \frac{t_y - B'_y}{H g} + \frac{1}{2}, \qquad n_z = \frac{t_z - B'_z}{W g} + \frac{1}{2}, \tag{4.6}$$

after which each face selects two of the three components as image coordinates and the remaining one as depth, with the appropriate reflections. Visibility is arbitrated in an integer grid rather than a floating-point one, because the resolution must be atomic. The depth of a candidate is quantised as $\delta = \lfloor (1 - d) \cdot 10^6 \rfloor$ and offered to the grid through an atomic maximum, and the thread writes the geometry, occupancy, and texture samples only when its own candidate is at least as near as the value already there. The arbitration is therefore indivisible while the three attribute stores that follow it are not, so two threads competing for one pixel are not assumed to produce a fixed write order, even though the aggregate fidelity observed across the validated executions is stable. A second kernel then packs the five planes into the single frame the coder expects, and the result is copied back into one of the three pinned slots from which the writer thread later draws it.

4.5.6 Layout of the Composed Picture

The dimensions follow the convention of Chapter 3 and are repeated here only to fix the numbers this platform uses. A face is projected at 640 by 480 samples and padded to 640 by 512, the six faces occupy a four-by-three arrangement of 2560 by 1536, and three such arrangements are stacked vertically for geometry, texture, and occupancy, giving a picture of 2560 by 4608. In the chosen sampling format this amounts to 17,694,720 bytes per submitted frame, of which two thirds are luminance. The padding to a multiple of sixty-four keeps every face aligned with the block grid of the coder, so that a coding block never spans two of them and the discontinuity at a face boundary is not spread across the residual of a neighbouring

one. The only substantive departure from the reference chain is that the stance requested by the viewer does not enter here. The source header carries a neutral orientation throughout, the projection geometry of a frame is therefore independent of any interaction, and the same compressed representation serves whatever view the terminal later chooses to present to the person watching.

4.5.7 A Coding Session That Outlives the Frame

The hardware coder is treated as a subsystem with its own state rather than as a per-frame invocation. One child process is started with the experiment and receives every picture through a pipe, which removes process creation, parameter parsing, and session warm-up from the measured interval, at the cost of an output whose timing is no longer synchronous with the input. Listing 4.6 gives the invocation. The rate control is constant at 10 Mbit / s with a buffer of twice that size, the picture group is fixed at fifteen with a forced refresh at its head, the preset and tuning select an ultra-low-latency configuration for encoding, and the two remaining options reduce the depth the coder retains and ask the container muxer to release material promptly [18, 26, 46]. No further latency switch is enabled, because none of the others produced a reproducible improvement when tested in isolation. A writer thread owns the pipe, so the unit never blocks inside a write while packets are arriving, and when all three output slots are committed it keeps servicing the network and the coder rather than waiting, until one becomes free.

```
ffmpeg -y -hide_banner -loglevel error \
       -f rawvideo -vcodec rawvideo -s 2560x4608 -pix_fmt yuv420p -r 30 -i - \
       -c:v hevc_nvenc -preset p2 -tune ull -delay 0 \
       -rc cbr -b:v 10M -maxrate 10M -bufsize 20M \
       -g 15 -forced-idr 1 -flush_packets 1 \
       -vstats_file /shared/log/encoder/ffmpeg.txt \
       -f mpegts -
```

Listing 4.6. Persistent hardware coding session

Before the first genuine frame, the unit submits private blank pictures in groups of fifteen under a reserved identifier, repeating the group until the reconstruction unit signals that its own session is ready. That traffic travels the ordinary media path and warms the coder, the container, the attribution logic, and the decoder in one operation, yet it never appears in the per-frame records because its identifier is excluded from them. The arrangement removes the startup transient from the measured population without pretending that no transient exists, and the number of private frames actually required is therefore an observation of the handshake rather than a quantity that the design fixes in advance. The check falls only at a group

boundary. That count therefore lands on a multiple of fifteen. The projection path is left cold, since blank material never reaches the accelerator.

4.5.8 Attribution of an Asynchronous Bitstream

A pipe does not preserve picture boundaries, so the unit rebuilds the container structure itself. Bytes are accumulated into complete units of 188 bytes, and the start of a video **Packetised Elementary Stream** (**PES**) unit marks the point at which the coder has begun to emit a new picture. The oldest frame still awaiting output is then associated with everything produced since the previous such marker, which yields the interval between the submission of a picture and the first evidence of its compressed form. That duration is reported as a combined coding, scheduling, muxing, and delivery frontier, never as an isolated measure of the hardware coder, and it must not be added to the active processing time of the same frame as though the two were disjoint. The parser does not hold a whole compressed picture. Every complete group of seven container units already accumulated is released at once, and only the residual fragment below that size waits for the next marker to confirm that the preceding picture has ended. Release and attribution are thus separate decisions, which is what prevents the pipeline from acquiring a second full-frame buffer immediately in front of the network stage.

4.5.9 Regulation Driven by Measured Load

The only feedback loop that can reduce the amount of work entering the chain is owned by this unit, because it is the unit that can observe whether the work is being absorbed. For each eligible frame it samples a service interval, smooths it with an **Exponentially Weighted Moving Average** (**EWMA**), and compares the result against the time the current factor allows, so that with f_{t} the nominal rate, s the factor in force, and α the smoothing weight, the controller maintains

$$T_{\mathrm{base}} = \frac{1000}{f_{\mathrm{t}}}, \qquad E_n = \alpha\, T_n + (1-\alpha) E_{n-1}, \qquad \rho_n = \frac{E_n}{s\, T_{\mathrm{base}}}, \tag{4.7}$$

with T_{base} in milliseconds and ρ_n the fraction of the admitted budget consumed. The ratio alone does not decide anything. An increase is asserted when ρ_n reaches its threshold, or when the wait between readiness and service exceeds a quarter of the budget, or when two frames are pending at once, or when the depth of the coder queue is growing. A decrease demands more, namely that the projected ratio at the next lower factor stay below half, that the wait fall within a tenth of the budget, that no frame be pending, and that the coder depth not be increasing. Table 4.3 lists the constants. Both predicates must hold for a consecutive run of observations

before the factor moves, and the run required to recover is far longer than the one required to escalate, which biases the loop towards keeping the source thinned rather than towards oscillating around the threshold it applies.

Constant	Value	Role
EWMA_ALPHA	0.25	Weight of the newest service sample
OVERLOAD_RATIO	0.75	Budget fraction above which an increase is asserted
RECOVERY_RATIO	0.50	Projected fraction at the lower factor required to release
OVERLOAD_FRACTION	0.25	Share of the budget the readiness wait may occupy
RECOVERY_FRACTION	0.10	Stricter share required before releasing
OVERLOAD_STREAK	2	Consecutive assertions before the factor rises
RECOVERY_STREAK	9	Consecutive releases before the factor falls
MIN_FRAMES	3	Observations before the loop may arm
STABLE_STREAK	3	Stable observations that arm the loop
MAX_FRAMES	15	Observations after which it arms regardless
RETRY_FRAMES	3	Interval between two presentations of one request
MAX_SKIP	9	Largest factor the loop may request

Table 4.3. Constants that govern the temporal factor

A warm-up gate protects the loop from its own transient. Samples are recorded from the first frame but are excluded from any decision until the measured service has been stable for three consecutive observations, or until fifteen have elapsed, whichever comes first, and the state of the gate is written into the per-frame record so that an armed decision can never be confused with a suppressed one. Once active, the loop also distinguishes a request that has been issued from a request that has taken effect. The factor is considered adopted only when the headers returning from the source carry it, and while the two disagree the loop suspends transitions and presents the same request again every three observations. Transitions are single-valued as well. The factor moves by one step in either direction, never further, and stays between unity and MAX_SKIP, so a transient cannot make the loop discard eight frames out of nine in one decision. A transition also clears both streaks and re-establishes the baseline against which the coder **backlog** is judged. Figure 4.7 draws the resulting behaviour. Convergence therefore rests on the returning application state rather than on delivery of a datagram, which lets a command be lost without ever leaving the system in a state that nobody had ever intended.

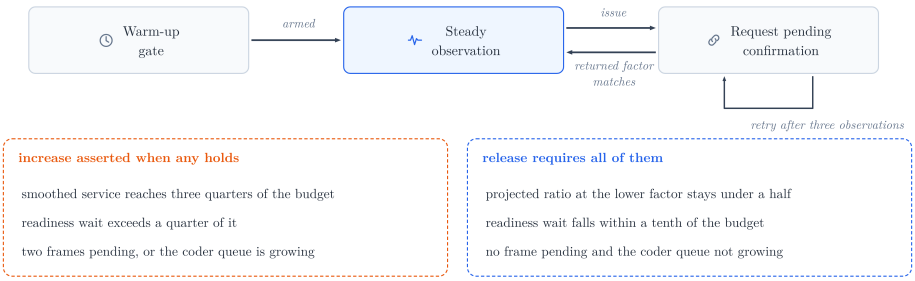


Figure 4.7. States and predicates of the regulator

4.5.10 Output, Capture, and the Ordered Drain

Compressed bytes leave the unit in groups of seven container units, preceded by the source header and by a forty-eight-byte block that restates the identifier, numbers the packet within its frame, and carries the scale, the projected centre, the neutral angles, and the centroid, so the reconstruction unit can invert the projection with the geometry that produced it. No service header is attached, since the hub owns that responsibility. When fidelity capture is requested, the unit additionally retains the luminance plane of every genuine picture and a byte-preserved copy of the attributed compressed stream in bounded memory regions, writing both to disk only after the sequence has ended and then evaluating the indicators over the geometry stripe alone through two isolated filter passes. The streaming path thus performs bounded copies and nothing else. Closure follows a fixed order, in which the residual frame is finalised, the writer and the parser are drained, the coder input is sealed, the remaining output is flushed, and only then are the records serialised, because compressed bytes routinely outlive the last submission and the terminal frame legitimately shows a longer residence than the steady state.

4.6 Rebuilding the Cloud and Serving the Viewer

The reconstruction unit is the second function that ignores the service path, and it is also the place where the interactive loop of this platform terminates. Both properties follow from the same decision. Since the stance requested by the viewer is applied here rather than at the source of the projection, a change of view never travels through projection, coding, transport, and decoding before it becomes visible, and the compressed representation of a frame stays independent of who is watching it. Figure 4.8 shows the stages the unit runs, together with the point at which the orientation requested by the viewer enters the sequence.

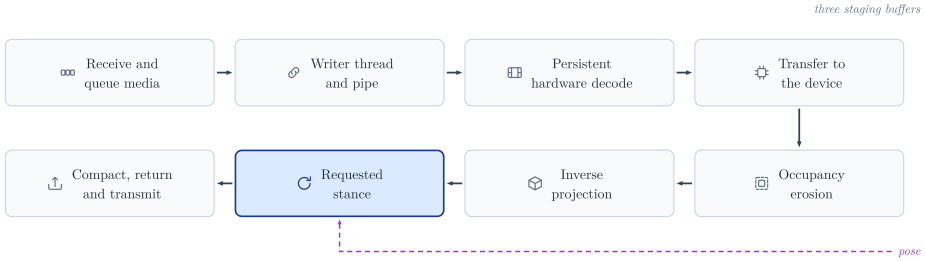


Figure 4.8. Stages of the reconstruction unit

4.6.1 Ingestion and the Bounded Media Queue

Datagrams arrive plain, with the source header, the compression block, and the container units behind it. The unit associates the bytes with the frame the blocks declare, appends them to a bounded queue of jobs, and lets a separate thread own the pipe towards the decoding session, so the polling loop is never suspended inside a write. The depth of that queue is recorded per frame as the maximum occupancy observed while the frame was being received, which exposes pressure between ingestion and the coder without requiring either side to be instrumented at every call. A job that cannot be admitted is counted rather than silently dropped. Submission towards the pipe is batched, and three staging buffers hold decoded pictures so that one can be reconstructed while another is being produced, which mirrors the arrangement adopted on the compression side.

4.6.2 A Decoding Session Held Open

The child process is started once, with hardware acceleration and the container demuxer configured for the smallest inspection window the stream permits, as Listing 4.7 shows. Its readiness is published through a file that the compression unit waits for before it admits the source, which is what turns the warm-up handshake into an explicit dependency instead of a race. The output side of that session is deliberately unadorned, since the child returns the decoded planes as raw samples in the layout the compression unit submitted, and a whole picture is accumulated before the reconstruction stage is offered anything. Nothing is rescaled, reordered, or re-timed on the way out, so the only transformation standing between the two accelerated units is the coding itself. The low-delay flag is retained, whereas the prompt-release option used on the compression side is not, because dedicated comparisons showed no repeatable gain once it had been added here. As on the other side, the chronology the coder itself reports is a diagnostic record and is not a one-to-one index of application frames, since the private pictures of the handshake

and of the drain appear in it. Attribution to a frame identifier therefore remains the responsibility of the native per-frame records and of those alone.

```
ffmpeg -hide_banner -loglevel error \
       -probesize 32768 -analyzeduration 0 -flags low_delay \
       -hwaccel cuda -c:v hevc_cuvid \
       -vstats_file /shared/log/decoder/ffmpeg.txt \
       -f mpegts -i - \
       -f rawvideo -pix_fmt yuv420p -
```

Listing 4.7. Persistent hardware decoding session

Once a picture is available it is moved to the device, and from that moment the work is entirely point-parallel. Three kernels run in sequence over the decoded planes, and the host measures each of them separately together with the transfers that surround them, so the residue that belongs to orchestration rather than to arithmetic is reported as such instead of being absorbed into a stage.

4.6.3 Suppression of Boundary Artefacts

Quantisation blurs the silhouette of a projected face, and the occupancy plane consequently acquires a fringe of weakly marked samples around the true surface. Admitting them would create a halo of points floating slightly off the reconstructed body. A morphological erosion over a two-by-two window removes it, and the window is anchored at the sample being written rather than centred, so that for the occupancy plane o of the **atlas** the result at each sample of the plane obeys

$$e(u,v) = \min\left\{o(u,v),\, o(u-1,v),\, o(u,v-1),\, o(u-1,v-1)\right\}, \tag{4.8}$$

with samples outside the plane read as fully occupied so that the border of the atlas is never eroded against emptiness [20]. Two populations are counted at this stage under a permissive test, namely the samples marked before the operation and those still marked after it, and a third is counted later under the stricter test that actually admits a point. The three quantities separate what the representation carried, what the filter preserved, and what the reconstruction accepted, which keeps a reduction caused by the coder apart from anything the network lost.

4.6.4 Return to Three Dimensions

A thread of the reconstruction kernel owns one sample of one face. It reads the eroded occupancy at the corresponding position of the atlas, abandons the sample unless the value reaches the admission threshold of 140, and otherwise recovers the depth from the geometry stripe and the two image coordinates from its own position

within the face. The inverse of the face mapping restores the normalised triple, each face contributing the component it had discarded and the reflections it had applied, and the denormalisation of Equation (3.12) then returns the point to the coordinate frame that the projection defined, using the projected centre and the factor that the compression block carries. Colour is read from the texture stripe, with the chrominance samples addressed at half resolution in both directions, and converted through the inverse relations of Equation (3.13). Admitted points are compacted into a contiguous array through an atomic counter, so the output contains no gaps and its length is known as soon as the kernel completes. The capacity of that array is fixed at six times the area of a face, which is the largest population that the representation is able to produce for any single frame.

4.6.5 Honouring the Requested Stance

A viewing command arrives as a bare twenty-four-byte payload and is admitted only if its timestamp postdates the one already in force, when the three components are finite, and when the scaling component is strictly positive. A repeated copy of the current command is therefore harmless, and a delayed copy of an older one cannot roll the view backwards. The accepted state is snapshotted before reconstruction begins, so a command arriving mid-frame never splits a cloud between two orientations. A third kernel then transforms the compacted result. Writing D for the observer distance, f for the scale the compression unit reported, R_{e} for the rotation used at projection, R_{d} for the requested rotation, and ζ for the requested scaling factor, every admitted point of the reconstructed cloud obeys the pair of relations

$$q = \frac{p - (0, 0, D)}{f}, \qquad p'' = \zeta \, R_{\mathrm{d}} \, R_{\mathrm{e}}^{\mathsf{T}} \, q, \tag{4.9}$$

where the first step undoes the framing that projection imposed and the second installs the orientation the viewer asked for. Since the source stance is neutral throughout, R_{e} reduces to the identity in every execution reported here, and the expression is kept general only because the header field that carries it remains defined. The delay between the timestamp of an accepted command and the completion of the transformation that first honours it is recorded whenever such a transition occurs, which gives the reverse loop a measurement of its own, one that depends neither on the browser nor on anything the browser itself reports.

4.6.6 Emission and Closure

The rebuilt cloud is cut into datagrams of at most the same capacity used upstream, each carrying a fifty-two-byte header that restores the frame identity and the source

timeline, restates the factor in force, reports the three populations counted during reconstruction, announces the stance actually applied, and advertises the packet capacity so the terminal can validate offsets without a compiled-in assumption. A frame that admits no point is still emitted, as a header without payload, so its completion remains observable. On the terminal marker the unit closes the coder input, drains the queued material, consumes the pictures still in flight, publishes the capture it may have been asked to retain, raises the marker that releases the compression unit from its own drain, forwards the terminal packet downstream, and serialises its records only once all of that has completed.

4.7 The Last Aware Boundary

The EPN is the smallest of the three path-aware units and the only one that serves traffic in both directions without ever holding a frame. Forward, it is the point at which the chain ends and the terminal receives an ordinary datagram. Backward, it is the place at which an interaction becomes a chain of its own. Figure 4.9 shows the two duties side by side and the state each one expects.

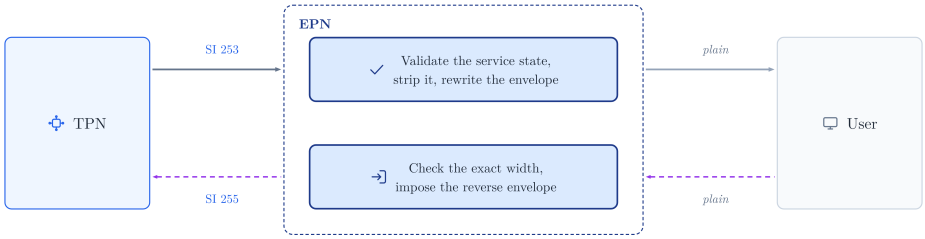


Figure 4.9. Duties of the egress node in both directions

4.7.1 Termination of the Forward Route

Traffic reaching this node comes from the hub and from nowhere else, and it is expected to carry the identifier of the main path together with the index that the last transition installed. The node validates the link, network, and transport headers, then the service state, and then the application consistency of the datagram, comparing the sequence number and the declared populations against what the frame has already delivered. It removes the eight-byte base header, rewrites the local headers for the terminal adjacency, and forwards the reconstruction header and its points untouched. Nothing of the application payload is interpreted beyond the fields needed for that check, which keeps the boundary cheap while still refusing a packet that would leave the terminal with an inconsistent frame. Receive completion

and forward completion are recorded separately, a distinction that proved necessary during development, because a frame fully received and only partly accepted by the local ring is a different failure from one that never arrived.

4.7.2 Classification of the Returning Command

The terminal-facing interface also accepts a bare twenty-four-byte payload, which is the viewing command. The node checks the exact length and the plausibility of the fields, then builds the reverse envelope with its own identifier and a constant index, and preserves the command byte for byte behind it so that the hub can strip the envelope and deliver the original payload to the reconstruction unit without ever interpreting a stance. Classification here is the mirror image of the one performed at the source end of the chain, and it completes the symmetry of the design, since every command in the system enters an aware path at the boundary nearest its origin and leaves it at the boundary nearest its destination. The node shares the descriptor depth, burst width, and bounded retry policy of the rest of the chain, and its record follows the schema of the other two aware units.

4.8 The Terminal and Its Presentation Frontier

The last unit closes three loops at once. It completes the forward route, it originates the viewing commands, and it feeds the browser that displays the result. Keeping those three concerns in one process would have coupled the polling loop to the pace of a rendering client, so the design separates them through a region of shared memory and a bridge of its own, as Figure 4.10 illustrates.

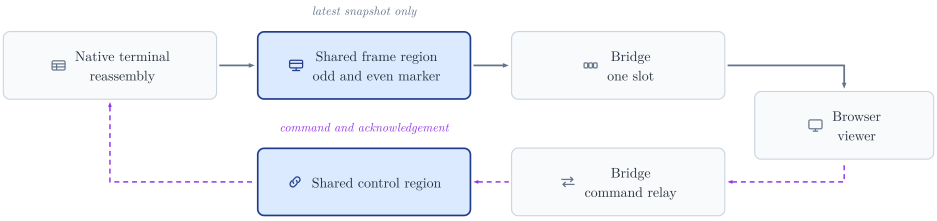


Figure 4.10. Delivery boundary between the terminal and the browser

4.8.1 Reassembly and Validation

Arriving datagrams are checked against the terminal adjacency and against the reconstruction header they carry. The sequence number selects the destination offset, derived from the advertised capacity, and the converted coordinates are written

straight into the shared region rather than into a private buffer that would later have to be copied. A frame is declared complete only when the packet count, the point count, and the population announced by the header all agree, and the three reconstruction populations it carries are retained so that the terminal can express what survived each stage as a fraction of the original cloud. Because the source timestamp has travelled unchanged from the Camera through every unit, the delay from acquisition to terminal availability is obtained here directly, and it is recorded twice, once at the instant at which the frame becomes complete natively and once at the instant at which it becomes visible to the bridge.

4.8.2 Publication Without a Lock

Two shared objects connect the native process to the bridge. One holds a seventy-two-byte descriptor followed by the point array of the current frame, the other a fifty-six-byte structure carrying the command exchange and the acknowledgement of the browser. That second object is halved along the same principle. One half carries the identifier, the kind, and the three components of a command, the other the frame and the command the browser has acknowledged together with the interval it measured, and each half owns a counter of its own, so a half caught mid-update is never read as a fresh one. Publication uses an odd and even sequence marker rather than a mutex. The value is odd while a frame is being written and becomes even when the descriptor is final, and the transition is a release store preceded by a barrier, so a reader that observes an even value is guaranteed to observe the payload that belongs to it. Listing 4.8 shows the closing sequence.

```
struct web_hdr *hdr = ( struct web_hdr * )web_frame_map;
uint64_t even_seq = web_seq + 2;

hdr -> frame_id = state -> frame_id;
hdr -> point_count = state -> valid_points;
// Remaining descriptor fields

__atomic_thread_fence( __ATOMIC_RELEASE );
__atomic_store_n( &hdr -> seq, even_seq, __ATOMIC_RELEASE );

web_seq = even_seq;
```

Listing 4.8. Publication of a completed frame

A reader that finds an odd value simply waits for the next one, and it can never directly block the writer, even under sustained polling pressure, which is the property that keeps the polling loop insulated from the browser. When fidelity capture is requested the publication is disabled altogether, since no browser participates in

that condition, while the reassembly logic stays identical, so that the two modes continue to share one and the same application contract.

4.8.3 Issuing a Stance and Confirming It

A command generated by the viewer receives a monotonically increasing identifier and a dispatch timestamp, and the native process records the requested orientation together with the first frame observed afterwards and the first frame whose returned stance actually matches. Dispatch does not wait for anything. If the requested state has not appeared in the returning stream, the same command, with its original timestamp preserved, is presented again every three completed frames, which is what allows a lost datagram to be survived without a transport acknowledgement. A newer command from the viewer supersedes an unresolved older one, since the loop conveys a desired state rather than a history of gestures. The two delays that result are kept apart, one measuring the first frame to arrive after the request whatever its content, the other that of the earliest frame which honours it, and their difference separates the transport of a command from its effect.

4.8.4 The Asynchronous Bridge

A separate process, written in a scripting language and running beside the polling loop, exposes the shared region to the browser over a **WebSocket** connection and serves the viewer page over **HTTP** from a thread of its own [17]. Its structure is what prevents a slow client from accumulating a backlog. Each connected peer owns a queue of one element and an acknowledgement gate. One coroutine watches the sequence marker and, when it changes, enqueues a token without copying anything, while another waits until the browser has confirmed the previous picture, then copies whichever completed snapshot is the most recent at that moment. A frame that becomes stale while the client is still rendering is therefore never sent, and the queue can never grow. Compression is disabled on the connection and the message size is left unbounded, because the payload is already a compact binary array and an additional transformation would add latency to every frame without reducing the transfer by any appreciable amount at these sizes.

4.8.5 The Browser Client

The page builds a point primitive whose storage grows in blocks, holds coordinates as single-precision triples and colour as normalised bytes, and redraws only when a frame or an interaction requires it rather than on a continuous animation loop. Orientation is controlled from the keyboard in steps of five degrees for the two angles

and by a multiplicative factor for the scale, with a reset that restores the neutral stance, and a gate prevents any command from being issued before the first genuine frame has established a baseline, with a pending request dispatched immediately afterwards. The moment a command is generated is recorded locally, and when a later frame comes back bearing the matching identifier, the page measures the interval that ends when the draw call returns, then acknowledges the frame with that measurement attached. The acknowledgement serves two purposes at once, since it releases the single-frame gate of the bridge and supplies the terminal with a presentation-side measurement. Its frontier is the completion of the draw call and not the illumination of a pixel, so the quantity is a render-completion proxy that excludes compositing, synchronisation with the display, and panel response. Figure 4.11 shows the interface during an ordinary session.

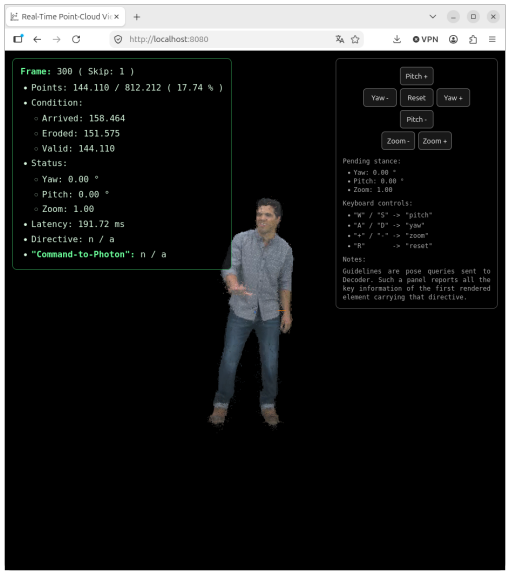


Figure 4.11. Web client during a streaming session

4.8.6 Capture of the Terminal Cloud

Under the fidelity condition this process retains every completed frame in a bounded region of its own address space and writes the whole collection after the sequence has ended, using a record made of a short descriptor and the point array it announces. This representation preserves the original frame boundaries and allows each captured cloud to be recovered independently during subsequent offline processing. The reconstruction unit performs the same operation one stage earlier, before erosion, and publishes its file atomically before it forwards the terminal marker, so the offline assessment can never observe a partially written capture. The process then writes its own file, serialises its records, and finally raises the marker that starts the assessment

described in Section 4.9.6. This ordering is what allows the objective evaluation to run on the same host without ever competing with the streaming path, and it also fixes the two stages at which fidelity is defined.

4.9 Applications That Surround the Chain

Seven programs carry the traffic, and a handful of smaller ones decide the conditions under which they run. Describing them is not a courtesy to reproducibility alone, because two of them determine quantities that appear in the measurements, namely the residency of the source material and the objective fidelity of the reconstruction. The others fix the state of the host, declare the topology, order the start-up, and rebuild every binary immediately before it executes.

4.9.1 Bringing the Host into Condition

A single entry script places the machine into the state the experiment expects. It synchronises pending writes, drops the page, inode, and directory caches, and reserves 1024 pages of two mebibytes each, for a total of two gibibytes of hugepages, which is the memory from which every unit later carves its packet pools. Only then does it declare the topology and start the units. The complementary script reverses the sequence, removing the seven containers, clearing the stale sockets and the compiled binaries left under the shared temporary directory, deleting the per-unit hugepage backing files, releasing the reservation, and dropping the caches again so that a subsequent run begins from the same condition. Neither script touches anything that requires a reboot, which keeps the two tiers of the configuration separate and forces that state to be reported for every run.

4.9.2 Withdrawal of Cores from the Scheduler

A separate pair of helpers installs and removes a boot parameter that withdraws all but one logical processor from the general-purpose scheduler, by rewriting the bootloader configuration and regenerating it. The change takes effect only after a restart, and the presence of the parameter must be verified on the kernel command line before results are collected. Establishing this condition at boot time also keeps the set of scheduler-visible processors invariant throughout the measurement session. This avoids run-to-run changes in scheduler eligibility and makes the processor environment reproducible across consecutive executions performed under the same boot configuration. Neither the preparation script nor the teardown script alters this state, precisely because it cannot be altered without a restart, so isolation is treated as a property of the session rather than of the run. The concrete assignment

of units to processors, together with the reasoning that produced it, belongs to the experimental configuration and is stated in Chapter 5.

4.9.3 Wiring and Start-Up Order

The topology script performs a deliberately minimal operation. It removes the endpoints left by a previous execution and declares the six adjacencies, and nothing else exists for it to do, since there is no bridge to build, no forwarding rule to install, and no switching process to reserve a processor for. The launcher then starts the units in dependency order, converting the availability of a socket or of a readiness file into an explicit precondition instead of relying on an arbitrary delay. The hub comes first, and its four service-side sockets must appear before anything attaches to them. The reconstruction and compression units follow, and both must publish their readiness markers before the chain is extended. The egress node, the terminal, and the ingress node come next, each gated on the socket or marker that the previous step creates, and the Camera is started last, so that no datagram can be emitted towards a unit that has not finished initialising, as Listing 4.9 shows.

```
start_node "sff2" "sff2" "6" "false" "6"
wait_for_dpdk_init "/tmp/sfc-sff1-sff2"
wait_for_dpdk_init "/tmp/sfc-sff2-enc"
wait_for_dpdk_init "/tmp/sfc-sff2-dec"
wait_for_dpdk_init "/tmp/sfc-sff2-sff3"

start_node "decoder" "decoder" "2,4" "true" "4"
start_node "encoder" "encoder" "5,7" "true" "5"
wait_for_node_init "/tmp/sfc-encoder-ready"
wait_for_node_init "/tmp/sfc-decoder-ready"

start_node "sff3" "sff3" "7" "false" "7"
wait_for_dpdk_init "/tmp/sfc-sff3-usr"

start_node "user" "user" "$USER_GROUP" "false" "0"
wait_for_node_init "/tmp/sfc-user-ready"

start_node "sff1" "sff1" "3" "false" "3"
wait_for_dpdk_init "/tmp/sfc-cam-sff1"

start_node "camera" "camera" "1" "false" "1"
```

Listing 4.9. Dependency-ordered start of the seven units

The same launcher exposes the one switch that selects between the two families of executions. With the switch clear, the terminal receives a bridged network interface and two published ports so that a browser can reach it, the presentation bridge

is started, and the launcher pauses before the Camera until the viewer has been associated. With the switch set, none of that exists, the terminal keeps a private namespace, and the source starts as soon as the native readiness conditions are met. Each unit runs privileged, without any network interface of its own in the nominal case, and mounts the hugepage filesystem, the shared temporary directory, the shared data and log tree, and its own source directory. The accelerator is exposed only to the two units that make use of it, and to no other.

4.9.4 Entry Points and the Build Boundary

No unit executes a binary carried inside the image. Each entry point compiles its own sources from the mounted directory and only then replaces itself with the resulting program, so a change to a file cannot be silently ignored by a stale artefact. The four service units and the terminal are built directly with the optimiser at its highest ordinary level and with the link and include flags resolved from the packet-processing framework, while the two accelerated units invoke a **Makefile** that rebuilds from scratch and applies the host- and device-specific profiles recorded in Chapter 5. This arrangement keeps the image responsible only for the toolchain and its dependencies, while the executable state is derived afresh from the sources exposed to the experiment at the moment of execution. It therefore makes the build boundary explicit and keeps the deployed code aligned with the configuration under test. The runtime arguments follow immediately, and Listing 4.10 shows a representative invocation. The processor on which the polling loop runs is supplied by the launcher as an environment variable, the memory reservation and the prefix isolate the instance, the probe of physical devices is suppressed, and the virtual devices declare the sockets and the queue depth of the link.

```
/tmp/sff1_dpdk -l $DPDK_CORE -m 256 --file-prefix=sff1 \
  --single-file-segments --no-pci \
  --vdev=net_vhost0,iface=/tmp/sfc-cam-sff1,queues=1 \
  --vdev=net_virtio_user1,path=/tmp/sfc-sff1-sff2,queue_size=4096,mac
    =00:00:00:00:02:01 \
  --log-level=*:alert
```

Listing 4.10. Runtime arguments of a two-port service unit

The invocation therefore forms the concrete boundary between the launcher-level configuration and the execution context seen by the packet-processing framework. The terminal entry point carries two further responsibilities. Under the interactive condition it starts the presentation bridge at a reduced scheduling priority, so that the coder work sharing its processor keeps precedence. Under the fidelity condition it instead arms a background gate that waits for the marker raised at the end of

the stream, then runs the assessment with every numerical threading environment pinned to a single thread, which prevents the offline stage from quietly acquiring a larger share of the host than the measurement assumed.

4.9.5 Offline Preparation of the Sequence

The dataset is not consumed in its published form. A short program reads each source file, extracts the vertex array, applies a scale factor that the experiment keeps at unity, casts the coordinates to single precision, and writes a contiguous array of fixed-width records matching the wire layout exactly, with three coordinates, three colour bytes, and one byte of padding. Two consequences follow. The Camera never parses a textual format, never allocates per field, and never performs a conversion whose cost would depend on the encoding of the input, so the residency modes discussed earlier measure storage behaviour rather than parsing behaviour. The record width is also identical along the entire chain, from the prepared file to the reconstructed cloud and to the capture written for assessment, which removes a class of conversion errors at the cost of carrying one unused byte per sample. The conversion is strictly a preparation stage rather than a compression step, its own timing is retained separately for reproducibility, and it must never be combined with any interval measured during the streaming sequence itself.

4.9.6 Post-Stream Geometric Assessment

The objective evaluation of the reconstruction runs after the terminal has published its marker, serially, on material that both the reconstruction unit and the terminal captured in memory during the stream. It considers two reconstruction stages, namely the cloud before erosion and the cloud after it, and for each it evaluates two alignment definitions. The stored orientation is first undone, so that a cloud returns to the coordinate frame in which the reference was originally captured, through

$$q = \frac{1}{\zeta}\, R^{\mathsf{T}}(\psi, \varphi)\, p, \tag{4.10}$$

after which the rigid transform between the two clouds is recovered. The procedure alternates a nearest-neighbour association restricted to a physical search radius with a **Singular Value Decomposition** (**SVD**) of the resulting correspondence, for at most thirty iterations or until the mean association distance stops improving [3, 56]. The coupled definition fits the complete reconstructed cloud directly. The decoupled one first removes statistical outliers from the cloud used for the fit alone, keeping a point when the mean distance μ_i to its twenty nearest neighbours satisfies $\mu_i \le \bar{\mu} + 2\,\sigma_\mu$, and then applies the recovered transform to the complete unfiltered

cloud, so the filter influences the estimate alone, and never reduces the population over which the reported geometric distances are ultimately measured. The search radius is fixed at two hundred millimetres expressed in voxels, which bounds how far a correspondence may reach and prevents a distant outlier from dragging the estimate towards itself. The thirty repetitions bound the cost rather than fix it, and a step where fewer than three correspondences survive the radius abandons the fit rather than report a transform derived from almost none.

```
for _ in range( ICP_REPETITIONS ):
    distance, ids = ref_tree.query( current, k = 1, workers = 1 )
    mask = distance <= ICP_RADIUS
    if mask.sum() < 3:
        break

    step_rot, step_shift = best_rigid( current[ mask ], reference[ ids[ mask ] ] )
    current = current @ step_rot.T + step_shift
    rotation = step_rot @ rotation
    shift = shift @ step_rot.T + step_shift
    score = float( distance[ mask ].mean() )
    if previous is not None and abs( previous - score ) < 1e-5:
        break
    previous = score
```

Listing 4.11. Association step of the rigid registration

The four indicators evaluated on the aligned pair are the ones introduced in Chapter 2, namely the directed mean of Equation (2.3) together with the symmetric root mean square error, and the two symmetric measures of Equation (2.4). The coupled definition reports the inlier error of the final association, whereas the decoupled one reports the bidirectional value over the whole population, and every quantity is additionally expressed in millimetres through the voxel pitch of the source sequence. Neither alignment nor measurement subsamples anything, and every association query is confined to a single thread. A frame contributes a result only when both captures and the corresponding reference are present and the evaluation returns successfully, the merged table replaces the terminal record atomically, and the two captured files are deleted only once the population actually completed equals the population the assessment expected at the outset.

4.10 What Each Unit Records

Every conclusion drawn in the following chapters rests on per-frame records that the units themselves produce, so the meaning of each quantity must be fixed before any of them is read. Two principles govern the whole scheme. A record is indexed by

the identifier the Camera assigned, never by the order in which something happened to be observed, so the same frame can be followed across nine files. Table 4.4 names the resulting files and the number of fields each carries.

Producer	File	Fields
Offline converter	`telemetry_converter.csv`	8
Camera	`telemetry_camera.csv`	31
IPN	`telemetry_sff1.csv`	43
TPN, first transition	`telemetry_sff1_enc.csv`	43
TPN, second transition	`telemetry_enc_dec.csv`	43
TPN, third transition	`telemetry_dec_sff3.csv`	43
Encoder	`telemetry_encoder.csv`	69
Decoder	`telemetry_decoder.csv`	62
EPN	`telemetry_sff3.csv`	41
User	`telemetry_user.csv`	40, or 72 merged

Table 4.4. Per-frame outputs produced by the platform

Nothing is written to disk while the stream runs, since the values accumulate in bounded memory and are serialised once the terminal condition has resolved, which keeps file activity out of the measured interval. Nothing is sampled or averaged either, so every admitted frame contributes one row to every file that observes it, and a missing row is evidence of an incomplete frame rather than of a gap in the instrumentation. The values are held in fixed-size structures reserved before the first datagram arrives, so recording can never allocate memory at an inconvenient instant. Two chronologies fall outside the scheme and are reported apart from it, namely the diagnostic output of the two coder processes, which counts pictures rather than application frames, and the offline preparation of the dataset, whose own chronology precedes the stream and is never merged with it.

4.10.1 Anchors Common to Every Record

A handful of fields recur in nearly all files and carry one meaning throughout. `frame_id` is the source identifier, and `current_skip` the admitted factor under which that frame was handled. `rx_complete` and `tx_complete` are predicates rather than counts, the first asserting that the declared packet set and population were received in full, the second that the local ring accepted everything the unit offered. `camera_send_timestamp` is the source anchor propagated unchanged from acquisition, `recv_start_timestamp` the first arrival at the unit, and `node_exit_timestamp` the instant of the last accepted departure, from which `camera_node_ms` and `total_residency_ms` follow as the source-to-node delay and the residence of the frame. `active_process_`

ms aggregates the work the unit actually performed, active_tx_ms the time spent inside transmit calls alone, tx_duration_ms the wall-clock span of egress including any waiting, and node_efficiency_pct their ratio against residence. schedule_delay_ms is the cumulative drift against the ideal instant of the identifier, inter_arrival_ms the raw spacing between the first packets of two consecutive observed frames, and instant_jitter_ms together with desynced_jitter_ms the deviation of that spacing from the interval the identifier gap implies and its smoothed chronology. The first observation of a run reports a spacing of zero.

4.10.2 Integrity, Rate, and Local Pressure

A second group describes what crossed the unit and how hard it was to push it out. original_points is the population the source declared, rx_points the unique population received, and tx_points the population forwarded, from which data_integrity_pct is the ratio of the second to the first. rx_packets and tx_packets count valid datagrams in each direction, payload_bytes the application bytes involved, and reference_size_bytes the logical size against which the comparison quantities are formed. Two throughput fields, internal_throughput_mbs and reference_throughput_mbs, divide measured and reference bytes by the real receive span of the frame in decimal units, whereas logical_bitrate_mbps, network_bitrate_mbps, and reference_bitrate_mbps are rate formulations at the effective frame rate and must not be read as another spelling of throughput. Four counters isolate local acceptance behaviour, namely tx_zero_accepts, tx_partial_accepts, tx_resubmit_calls, and tx_resubmitted_packets, and a fifth, mbuf_starvation, records a failure to obtain a buffer and belongs to the units that allocate one. Four further counters, eth_errors, ipv4_errors, udp_errors, and nsh_errors, attribute a rejected datagram to the layer that refused it, and they exist only in the three units of the chain that are able to read the service path.

4.10.3 Admission, Acquisition, and the Service Loop

The Camera replaces the receive-side group with quantities that describe production. selected states whether the admission rule retained the frame, last_control_frame and temporal_control_ms the reference of the last accepted request and the cost of handling it, and tx_start_timestamp the boundary immediately preceding submission, which inter_departure_ms then uses to measure the spacing between two frames actually transmitted. disk_io_ms times the acquisition of the frame under the two modes in which acquisition happens inside the loop, serialization_ms the in-place conversion of its coordinates, and reference_efficiency_pct a second ratio whose numerator omits that conversion. The aware nodes instead add

loop observability through cycle_ms, the interval between two successive services, header_wait_ms, the wait for the first header of a frame, and cycle_occupancy_pct, the share of that interval which the residence of the frame occupied. Only the ingress node adds geometry_aggregation_ms and max_r_ms, the first covering the running reduction together with the frame-complete frontier pass and the second the radius evaluation alone, and only the hub adds rx_media_bytes and tx_media_bytes, which replace the point counts on the transition that carries compressed material, where a point population would have no meaning.

4.10.4 Projection, Coding, and Device Intervals

The compression unit adds the largest group. event names the state of the regulator, which is one of warm-up, idle, increase, decrease, retry, or invalid, while yaw, pitch, and zoom retain the neutral source stance. processed_points is the population projected and conversion_ms the accumulated cost of decoding and placing the incoming coordinates. geometry_aggregation_ms and max_r_ms carry the local fallback and read zero whenever a valid upstream descriptor is consumed, projection_ms the host-visible accelerator interval, and gpu_transfer_ms, gpu_kernel_ms, gpu_packing_ms, gpu_copyback_ms, and host_overhead_ms its decomposition into transfer, projection, packing, return, and residue. codec_write_ms times the submission of a picture to the coder, codec_exit_time marks the first attributed output, encode_service_ms is the interval used to drive the regulator, and encode_h265_ms the asynchronous frontier between submission and first compressed evidence. raw_queue_ms is the wait between readiness and service, render_queue_ms the wait between projection and submission, and frame_backlog together with codec_backlog the two depths the regulator inspects, alongside workload_ewma_ms and workload_ratio. mpeg_bytes_generated counts the compressed bytes attributed to the frame, ffmpeg_write_calls and ffmpeg_write_eagain the pipe submissions and their defensive retries, and mse_y, psnr_y, and ssim_y the coding indicators, which exist only under the fidelity condition and refer to the geometry stripe alone. Three timing aggregates complete the file, namely reference_process_ms and reference_residency_ms with the ratio reference_efficiency_pct, total_processing_ms, which unlike the active aggregate includes intervening waits, and e2e_latency_ms beside reference_jitter_ms, both carrying the definitions given for the corresponding fields of the other units along the chain.

4.10.5 Populations, Stance, and Presentation

The reconstruction unit reports rx_media_bytes for the compressed material attributed to a frame and then the three populations that its stages produce, namely

arrived_points before erosion, eroded_points after it, and valid_points after the admission threshold, with the second naming a surviving population rather than a removed one. erosion_ms, reconstruction_ms, and pose_ms time the three kernels, reconstruction_pipeline_ms the host-observed whole, and gpu_transfer_ms, gpu_copyback_ms, and host_overhead_ms the transfers and the residue. codec_queue_ms is the delay before compressed bytes begin their submission, frame_queue_ms the delay before a decoded picture is served, codec_backlog the maximum queue occupancy seen during reception, decode_service_ms the retrieval of a decoded picture, and decode_h265_ms the broader frontier that includes waiting. ffmpeg_write_calls, ffmpeg_write_failures, and codec_queue_drops describe the pipe and the bounded queue, pose_control_ms the delay from an accepted command to the transformation that honours it, and the reference aggregates repeat the pattern of the compression unit, with reference_process_ms deliberately excluding the stance kernel. The terminal expresses the same populations as fractions through arrival_pct, erosion_pct, and valid_pct, times the publication of a frame through web_publish_ms and the return of its acknowledgement through web_ack_ms, and reports e2e_latency_ms beside reference_e2e_ms, the latter ending when the frame becomes complete natively rather than when it becomes visible. Four fields close the interactive loop, namely cmd_id, reference_cmd_ms, cmd_apply_ms, and cmd_photon_ms, which name the command first observed on a returned frame, the arrival of the first frame after its dispatch, that of the earliest frame honouring it, and the browser-side proxy measured up to the return of the draw call. quality_save_ms times the retention of a capture and is absent from interactive runs, in which the terminal performs no operation of the kind at all.

4.10.6 Offline Columns and the Limits of Addition

The assessment appends thirty-two further fields to the terminal file, obtained as the product of two reconstruction stages, two alignment definitions, and eight indicators, the last four of which are the first four expressed in millimetres. Their names follow that product exactly, so a column such as post_robust_chamfer_mm states its stage, its alignment, its indicator, and its unit without further explanation. The offline converter contributes a separate file describing the preparation of the dataset, with filename and status, the population num_points, the two timed probes read_ascii_ms and write_bin_ms, the broader conversion_ms that also covers extraction, casting, and record formation, and the two sizes size_ascii_bytes and size_bin_bytes. One caution applies to the whole scheme. The intervals recorded by a unit overlap deliberately, because active work, residence, accelerator stages, coder latency, and source-to-node delay answer different questions, and adding them as though

they were disjoint serial stages would count concurrent work twice. The single frontier that spans the platform is the source anchor resolved at each terminal point, and even that one carries a mode-dependent boundary, since the anchor follows acquisition while the source residence precedes it. Taken together, the seven units and the records they keep define the object that the remaining chapters measure. The design fixes one variable and holds everything else constant, since the graph, the packet sizes, the service state, the coder configuration, and the regulation parameters are the same whether the frontier is resolved while the samples are in motion or after they have been assembled. The instrumentation then makes that variable observable one frame and one unit at a time, which allows a difference to be attributed to a placement instead of inferred from an aggregate. Two limits of the arrangement are better stated here than discovered later. The first is that a hop between units is a memory copy, so no conclusion about wire time reaches beyond this testbed. The second is that several of the intervals reported for one frame overlap by construction, so the platform supports statements about where work happens and about how long a stage occupies a unit, and not statements obtained by summing stages into one serial budget. Both limits are taken up in Chapter 5, together with the conditions that the present chapter has deliberately left to it, namely the assignment of units to logical processors, the families of executions through which the comparison is exercised, and the rules under which a trace becomes evidence.

Chapter 5

Performance Evaluation

5.1 The Questions the Campaign Has to Settle

The platform of the preceding chapter is a design, and a design decides nothing on its own. This chapter fixes the conditions under which it was exercised, states which executions were performed and why each of them was thought worth performing, and defines how a trace must be read before any quantity taken from it may be compared with another. The numbers themselves belong to Chapter 6 and are deliberately withheld here, because an experimental protocol that is written after its own outcome has already been seen is no longer a protocol. Three questions organise everything that follows. The first asks whether the redistributed chain behaves better than the application-level pipeline from which this work departs, at a frontier the two can both express. The second asks whether any advantage that appears is owed to the site at which the geometric frontier is resolved, rather than to the packet-processing substrate on which the new platform happens to stand. The third asks whether moving that site changes what the viewer eventually sees. Each question needs a control of its own, and no single execution among those described below answers more than one of them at a time.

5.1.1 One Independent Change at a Time

The campaign is differential and changes one condition per execution. Deployment, processor affinity, queue depths, the absolute source schedule, the coder invocation, and the service-path contract are identical everywhere, so that a trace differs from the nominal one in exactly the respect its label announces. That discipline is what makes the archive interpretable. A study that varied the packet capacity, the **placement** of the frontier, and the residency of the source together would still produce a table of numbers, and nothing in that table could be attributed to any one of the three.

The design also fixes what counts as evidence. A difference is admitted only where it exceeds the spread that the unchanged condition itself exhibits when it is repeated, which is why the nominal execution is repeated rather than measured once and the paired **ablation** is repeated as well. Everything that varies between two executions is recorded in the label of the run and in the **manifest** that accompanies its records, never inferred afterwards from the traces those executions produced.

5.1.2 Runtime and Fidelity Are Not Measured Together

A second separation runs through the whole campaign. An execution either measures the interactive system, with the presentation bridge attached and a browser rendering the result, or it measures objective fidelity, with the bridge absent and the two capture paths retaining material in memory for a post-stream assessment. The two cannot be combined without spoiling both. The capture condition performs sizeable copies inside the streaming path, one complete **luminance** reference per genuine frame, so its timings describe a system under instrumentation rather than the clean operating point. The interactive condition, conversely, leaves the fidelity columns unset precisely so that the latency experiment is not contaminated by work nobody would perform in service. The consequence is a rule that the later chapter applies without exception, that a coding indicator is read only from a capture-enabled execution and a terminal delay only from an interactive one.

5.2 The Host Held Still

Every measurement reported in this work was produced on one machine, within boot sessions configured identically, and the properties of that machine are part of the result rather than a footnote to it. The platform occupies the host completely. Seven native processes, two persistent coder children, a presentation bridge, and a browser share four physical cores, so nothing here behaves as it would on a machine with spare capacity, and the arrangement that follows was chosen to make that constraint explicit instead of letting the scheduler resolve it silently.

5.2.1 Processor Affinity and Where Each Unit Runs

The host exposes four physical cores and eight logical processors through **Simultaneous Multithreading** (**SMT**), paired as $\{0, 4\}$, $\{1, 5\}$, $\{2, 6\}$, and $\{3, 7\}$. Removing the virtual switch from the topology, as Chapter 4 described, released the processor that a switching poll loop would otherwise have consumed, and that capacity was returned to the application graph rather than left idle. Table 5.1 records the final assignment together with the reason for each entry. Two choices in

it deserve a word. The terminal polling loop keeps processor 0, the only one that the isolation parameter leaves under ordinary scheduling, which is a deliberate decision and not an oversight, since the offline assessment reuses that same processor once streaming has ended and never while it proceeds. The egress node shares a sibling with the coder input of the compression unit, which is admissible because that node does little more per packet than check a header and rewrite it, whereas its neighbour spends most of its time waiting on a pipe rather than computing.

Processor	Occupant of the core	Reason for the placement
Core 0	**User** polling loop, and the assessment that follows it	The one processor the isolation set leaves under ordinary scheduling, so the terminal stays reachable by housekeeping
Core 1	The **Camera**, which shares with no other unit at all	Source scheduling and packetisation, on the processor that the removed switch released
Core 2	**Decoder** coder child and writer, with the presentation bridge	Pipe work is pinned, and the bridge joins it only where no capture runs
Core 3	The **IPN**, and nothing beside it	A processor of its own for the in-path service, whose cost the ablation isolates
Core 4	Decoder polling loop and its reconstruction kernels	Native packet and kernel work kept apart from the coder pipe of the same unit
Core 5	**Encoder** polling loop and its projection stage	Assembly, projection, and regulation on a processor of their own
Core 6	The **TPN** and its four ports	Four ports, five flows, one polling loop
Core 7	The **EPN** with the Encoder coder child and writer	A light aware boundary beside asynchronous pipe work

Table 5.1. Assignment of units to logical processors

The container sets follow that table directly. The Camera receives `1`, the ingress node or IPN `3`, the hub or TPN `6`, and the egress node or EPN `7`, while the compression unit receives `5,7` and the reconstruction unit `2,4`, in each case with the polling loop on the first of the two and the coder child on the second. The terminal alone changes between the two families, receiving `0,2` where the bridge participates and `0` where it does not. Every native process is additionally told which processor its own loop must occupy, through the argument that Chapter 4 showed the launcher supplying, so the affinity of the loop is fixed twice over, once by the container and once by the packet-processing framework. The redundancy is intentional, since a set of two would leave the framework free to choose.

5.2.2 Removing Seven Processors from the Scheduler

The eight logical processors are not equal in the eyes of the operating system unless they are made so. A helper installs the boot parameter `isolcpus=1-7`, which

withdraws every processor except the first from the general-purpose scheduler, and a second helper removes it again [66]. Neither takes effect without a restart, and the kernel command line must be inspected before results are collected, since nothing in the per-frame records reveals which of the two states was in force. The effect of the parameter is narrow and worth stating precisely. It does not make a polling loop faster and it does not reserve memory bandwidth or cache. What it removes is the eligibility of the isolated processors for ordinary load balancing, so that no unrelated runnable task is migrated onto a processor that a polling loop already occupies, and a loop that never yields is therefore not interleaved with work that has nothing to do with the experiment. The quantities most exposed to that interleaving are exactly the ones this campaign sets beside one another, namely the service interval the regulator smooths, the arrival spacing observed at each boundary, and the delay accumulated from acquisition to the terminal. Holding the scheduler still does not make those quantities constant, since the units contend with one another regardless, but it removes a source of variation that would differ between two executions for reasons outside the design. Isolation belongs to the boot session and not to any individual run. The preparation and teardown scripts deliberately leave it alone, and for that reason the state of the kernel command line accompanies every archived result as an item written down when the machine was booted.

5.2.3 Memory Reserved Before Anything Starts

The entry script brings the machine into a known state before the first container exists. It forces outstanding writes to storage, empties the three kernel caches named in Chapter 4, and sets two gibibytes aside as large pages, out of which each unit later carves its own pool. **The order matters**. Dropping the caches first and declaring the topology afterwards means that the source begins from a cold file system in every execution, after which the residency mode of the Camera decides on its own terms how much of the sequence is resident when the schedule opens. Those two conditions are therefore independent variables and are reported as such, since a warm source on a freshly flushed host is not the same experiment as a warm source on a host whose caches were left as a previous run had shaped them, and the records they produced would not tell the two apart afterwards.

5.2.4 What the Build Nails Down

The image is common to the seven units and is built once from a **CUDA** development base on **Ubuntu** 22.04, carrying the compiler toolchain, the packet-processing framework at its long-term release, **FFmpeg**, and the interpreter with the numerical and connection libraries the bridge and the assessment require. Nothing prebuilt

inside it is ever run. Each entry point rebuilds its own sources before replacing itself with the result, so the code under test is always the code in the mounted tree. The two accelerated units carry a makefile whose principal flags are reproduced in Listing 5.1, and those flags are conditions of the experiment rather than incidental switches chosen for the convenience of the toolchain.

```
CPU_ARCH ?= native
CUDA_ARCH ?= sm_61

CXXFLAGS = -O3 -march=$(CPU_ARCH) -ffast-math -funroll-loops \
           $(DPDK_CFLAGS) -I./h -std=c++14 -Wall
NVCCFLAGS = -O3 -arch=$(CUDA_ARCH) -I./h -std=c++14 -Xcompiler -fPIC
```

Listing 5.1. Architecture selection of the accelerated units

The device target is the one that binds this campaign most tightly to its hardware. Setting `CUDA_ARCH` to `sm_61` compiles the projection and reconstruction kernels for **Compute Capability** 6.1, which is the capability of the **GeForce GTX 1060** that served this campaign, a device carrying three gigabytes of memory, and it produces device code for that generation alone rather than a portable intermediate form [47]. The host flag behaves similarly, since an optimiser told to target the machine it is running on will emit instructions that another machine may not possess. Neither choice is a defect. Both were made because the campaign compares two placements on one host, and fixing the generated code removes a degree of freedom that would otherwise sit between the two builds. What follows from them is a limit on portability rather than on validity, and Section 5.8 states which of the two kinds of conclusion survives a change of hardware. The remaining build-time constants of the accelerated units are fixed alongside those flags. The compression unit pins its coder child to processor 7 and the reconstruction unit pins its own to processor 2, through a constant in each header, not through the launcher, so that the affinity of a child process is recorded in the same place as the affinity of the loop that spawns it and keeps it supplied with the material it consumes.

5.3 Two Families of Execution

One switch, read by the launcher and passed to every unit that cares, selects which of the two families an execution belongs to. Its value is stated explicitly on the command line rather than inferred from the presence of a browser or any other circumstance, because an implicit condition can later be reconstructed incorrectly. Table 5.2 sets the two families side by side. The differences are not cosmetic. In one the terminal publishes into shared memory, a bridge relays the latest completed cloud to a browser, and the launcher waits for a viewer to attach before the source

starts. In the other the terminal is reachable by nobody, no viewer is awaited, and three capture paths retain material for a later assessment.

Element of the run	Interactive family	Capture family
Terminal processor set	Cores 0 and 2 together	Core 0 on its own
Presentation bridge	Started at a lowered priority	Not started at all
Published ports	Page and connection exposed	No port is published
Start of the source	After a viewer has attached	After native readiness alone
Coding indicators	Left unset in the records	On the geometry stripe alone
Retained clouds	None retained anywhere	Pre-erosion and terminal clouds
Offline assessment	Not armed at start-up	Serial, once the marker is up

Table 5.2. What the capture switch governs

Two consequences follow, and both constrain the chapter that reports the numbers. The browser-side measurement of a command exists only in the interactive family, because there is no browser in the other, so it can never be quoted beside a fidelity indicator taken from a capture-enabled trace. The coding and geometric indicators exist only in the capture family, so they can never be quoted as though they had been obtained while a viewer was interacting with the stream. The capture family nevertheless sustains the nominal cadence, which is itself worth recording, since it shows that the instrumentation is bounded rather than free. It remains an instrumented condition and does not replace the interactive family as the reference for anything measured in time, and the later chapter never uses it so.

5.4 Constants That Specify Each Run

A run is defined by the values its binaries were compiled with, not by the folder its records were written into. Table 5.3 lists the operating point shared by the whole campaign, and every execution described afterwards is stated as a departure from it. Three of these values are shared across more than one program and cannot be changed in one place alone. The capacity of a point datagram is compiled into all seven units, since every one of them either produces, validates, or interprets a payload whose width follows from it. The grouping of compressed material is shared by the compression unit, the hub, and the reconstruction unit. The nominal rate and the length of the sequence are read by the source and used by every consumer that derives a budget from them. A coherent rebuild of the affected binaries therefore precedes any execution in which one of the three moves, and an archive whose units disagreed about one of them would be a broken experiment rather than a new condition, whatever the records it produced appeared to contain.

Constant and value	What the constant governs
`CACHE_MODE`, set to `MIDDLE`	Where the bytes of a frame live when the loop arrives
`WARM_MODE`, set to `ENABLED`	Whether the sequence is pinned before the schedule opens
`NETWORK_PROCESSING`, `ENABLED`	Whether the ingress node resolves the geometric frontier
`OFFLOAD_MODE`, set to `ENABLED`	Whether the compression unit consumes that block
`TEMPORAL_ADAPTATION`, `ENABLED`	Whether the regulator may change the admitted rate
`TARGET_FPS`, held at 30	The nominal cadence of the absolute source schedule
`K_FRAMES`, held at 300	The length of the sequence the source streams
`POINTS_PER_PACKET`, held at 80	The capacity of one point datagram, in records
`NETWORK_MTU`, held at 1500	The bound every constructed packet is checked against
`MEDIA_PAYLOAD_SIZE`, at 1316	The grouping of compressed container units on the wire
`QUALITY_CAPTURE`, held at 0	Which of the two families the execution belongs to

Table 5.3. Nominal configuration for the campaign

A second group is held fixed everywhere and never varied, yet belongs in the record because any of its members would move the operating point if it were touched. The burst width is thirty-two packets, the bound on consecutive refusals is 2048, the receive and transmit rings and the declared virtqueue each hold 4096 descriptors, and the twelve constants of the regulator are the ones Chapter 4 tabulated. The pairing of the residency mode with warm pages likewise stays where it is for a methodological reason rather than an optimal one. The middle mode keeps the per-frame acquisition inside the timed loop, which is the property that makes the source comparable with the chain of Chapter 3, while warm pages prevent storage from dictating a cadence the experiment never meant to study.

5.5 The Conditions Evaluated

Fifteen executions constitute the campaign, and one further execution stands outside it. Table 5.4 names each of them, states the single departure from the common operating point, and gives the question it was performed to answer. The twelve rows correspond to fifteen executions, because two of the labels stand for a condition that was repeated rather than for one execution. Every departure listed there is a compile-time constant with a single exception, the capture switch, which the launcher reads from the environment, so all but one of the rows oblige the affected binaries to be rebuilt before the execution begins. The order of the table is the order in which the subsections take them, running from the conditions that carry the argument of this thesis to the ones that bound it. The subsections that follow explain the reasoning behind each family, because a label and a changed constant do not say why anyone should care about the result.

Run and its departure	The question it was performed to answer
`B0-R1..R3`, with nothing changed	How much does the unchanged system move
`O-OFF-R1..R2`, both switches disabled	What does the site of the frontier cost the chain
`BQ`, with the capture switch raised	What fidelity does the nominal point deliver
`OQ-OFF`, both disabled with capture raised	Does the change of placement alter that fidelity
`C-BEST`, `CACHE_MODE` at `BEST`	How the source behaves with no read in the loop
`C-M-COLD`, with `WARM_MODE` disabled	What page residency is worth to the schedule
`C-W-WARM`, `CACHE_MODE` at `WORST`	What per-frame allocation costs the source
`C-W-COLD`, at `WORST` with warm disabled	The pessimistic bound on source acquisition
`P40`, `POINTS_PER_PACKET` at 40	What twice the datagram rate does to the chain
`P40-NO-T`, the same rate with no regulator	The same pressure without source admission
`P84`, `POINTS_PER_PACKET` at 84	The largest aggregation the envelope permits
`M1000`, `MEDIA_PAYLOAD_SIZE` at 1000	A media grouping that no longer divides evenly

Table 5.4. Departures assessed in this campaign

5.5.1 The Baseline Repeated Three Times

The unchanged configuration was executed three times, and the three traces are kept apart rather than averaged into one. The reason is that a single execution of a system this tightly packed says nothing about how much of what it shows is the system and how much is the hour at which it ran. Three repetitions give an envelope, and the envelope is what licenses every later claim, because a difference smaller than the spread of the unchanged condition is not a difference at all. They also serve two further purposes. The first is that this configuration, and no other, is the one placed beside the application-level chain of Chapter 3. That comparison is made at the only frontier both systems express, namely the delay from the instant the source dates a frame to the instant the terminal holds it complete, and it is read as an observation about two whole architectures rather than as a measurement of any one mechanism inside them. The second is that the interactive loop lives here. A viewer attached to one of the three executions issued **Pose** commands throughout, another received a handful, and a third none at all, and that asymmetry is preserved rather than normalised away, since it is what allows the cost of an interaction to be separated from the cost of carrying the stream it modifies.

5.5.2 Moving the Frontier Back

The comparison at the heart of this work is not between the new platform and the old one. It is between two builds of the new platform that differ only in which unit resolves the geometric frontier, and that pair is the only experiment in the campaign capable of attributing anything to placement. Both switches move together.

Disabling the in-path computation alone would leave the compression unit consuming a descriptor that nobody filled, and disabling the consumer alone would leave the ingress node performing work that nobody read, so either half on its own would measure a broken configuration rather than an alternative one. With both disabled the frontier is still computed exactly once, over the same definition, in the same order of operations, from the same coordinates. Everything else is held. The graph, the number of datagrams, the widths of the headers, the service state, the coder invocation, the regulator constants, the source condition, and the interactive family are all unchanged, so the two builds differ in whether two passes over a cloud occur at one unit or at the other. What the ablation is expected to expose, and what the next chapter will either find or fail to find, is a compression unit that has to absorb the preparation it no longer receives. The quantities that should move are the aggregate of work that unit performs on a frame and the fraction of its admitted budget that the regulator observes being consumed, both of which are recorded per frame and neither of which depends on the terminal. The delay measured at the terminal should move as well, although it is the more fragile of the two, because it accumulates the phase relationship between several loops rather than the cost of one of them. That is why this condition, too, was repeated.

5.5.3 Two Captures for One Statement

Fidelity is controlled by two runs rather than by one execution, and together they match the geometric ablation exactly. One capture-enabled run uses the nominal location, and the other the disabled condition, with nothing else separating them. The purpose of arranging it that way is easy to state and easy to get wrong. Since the two builds evaluate one definition and differ only in the unit that evaluates it, they ought to deliver the same reconstruction to the viewer, and the same coding distortion on the way. The experiment therefore predicts an equality, not an improvement. An equality is a weak-looking claim and a strong control, because it is the one outcome that a badly posed **offload** could not produce. A frontier resolved from partial statistics, or admitted without the test that ties a descriptor to the population it describes, would reach the accelerator with different parameters and would change the projected **atlas**, the coded picture, and the cloud rebuilt from it. Comparing the indicators across both runs is what turns the neutrality of that handoff from an argument into a measurement. Three levels of evidence are drawn from the two runs. The final admitted population of each frame is compared directly, since both variants must first match at that boundary. The coding indicators are evaluated on the geometry stripe alone, cropped from the composed picture at its origin, which matches the boundary the reference methodology used and keeps texture

and occupancy out of a number that is meant to describe depth. The geometric assessment then contributes the thirty-two columns that Chapter 4 enumerated, covering both stages of the rebuild, both definitions of the fit, and eight indicators apiece. A divergence anywhere in that set would count against that location, and its absence is the only form the claim can take.

5.5.4 Bounds on the Source

Nothing enters the chain except through the Camera, so the policy by which it obtains a frame sits upstream of everything done afterwards. Four executions bound that policy from both sides. Two of them move the residency mode, one to the arrangement that prepares the whole sequence in the wire form before the schedule opens, and one that allocates and releases a buffer on every iteration. The other two disable page pinning under the middle and the pessimistic modes, so that the working set is no longer resident when the first deadline arrives. The optimistic bound is not an optimisation the campaign recommends. It removes acquisition and conversion from the timed path altogether and therefore describes a source that has already been read, which is a legitimate limit but not a streaming condition. The pessimistic bound is the mirror image, adding an allocation and a release to every iteration. Neither replaces the middle mode, which was chosen because it keeps a per-frame read boundary visible while leaving the rest of the chain to dominate the schedule. The two cold executions exist for a narrower reason. They are the evidence that page residency belongs in the manifest of every benchmark rather than being assumed, since a source whose pages are not resident cannot hold the nominal cadence at all, and a reader who found a lower cadence in an archive without that field would have no way to explain it. The optimistic mode admits no warm and cold distinction, because everything it touches is allocated, read, converted, and pinned well before the first deadline of the schedule arrives.

5.5.5 Datagram Rate and Media Grouping

Four further executions move the quantities that decide how much traffic a frame becomes. The forwarding path does not see frames. It sees datagrams, and the number of them a frame produces is set by the capacity of a point packet, so halving that capacity doubles the population the ingress node validates, the hub relays, and the compression unit places, without changing a single byte of the cloud itself. That execution is the stress condition of the campaign, and it is the one in which the **Temporal** regulator is expected to leave the idle state it occupies everywhere else. Its diagnostic twin repeats the same capacity with the regulator disabled, which separates two things that would otherwise be confounded, namely the pressure a

datagram rate creates and the admission decision the system takes in response to it. Without the twin, a trace showing fewer admitted frames could not be read as evidence that the loop had worked rather than as evidence that the chain had failed. The opposite direction is bounded as well. Raising the capacity to eighty-four points is the largest whole-point aggregation the service envelope admits inside the conventional transmission unit, and eighty-five would exceed it, so that execution establishes where the packet format actually ends rather than where the campaign happened to stop. The last of the four leaves the point path alone and changes the grouping of compressed material instead. The nominal grouping carries exactly seven container units of one hundred and eighty-eight bytes, and the changed one does not divide evenly, so the compression unit emits a residue that the reconstruction unit must accept. The container unit itself is untouched, which is what makes this a test of application-level grouping rather than of the transport format.

5.5.6 A Run Reserved for Looking

One execution in the archive is not a campaign condition. Both accelerated units carry a compile-time diagnostic that retains a single frame of the sequence, fixed at index 195, and writes it out once everything else has finished. Fixing the index in advance prevents the visual comparison from being shaped by a frame selected retrospectively because it happens to display a particular artefact more clearly. When it is enabled the compression unit reserves one picture-sized region before streaming and copies the composed atlas of that frame into it, while the reconstruction unit copies the decoded picture once, after its native evaluation and before the reusable slot is returned. Nothing is serialised at that moment. The files appear only after the stream has closed, the coder input has been sealed, the remaining output has been drained, the terminal marker has been forwarded, and the records have been written, so the selected frame carries one bounded memory copy in the live path and no file-system work whatever. What the diagnostic produces is a raw picture and three single-channel images on each side, showing the geometry, texture, and occupancy stripes of the atlas as the compression unit composed them and as the reconstruction unit received them. Because both observations are retained at the codec boundary, the pair of images isolates precisely what the compression stage altered, and deliberately keeps out everything that happened on either side of it within one run. Placing the two sets beside one another is the only direct way to see what the coder did to a projected surface, which no scalar indicator conveys. The run is nevertheless kept out of the performance archive and is never quoted as a condition, for the reason that governs the whole chapter. The host is fully allocated, one further copy per stream is not free even when its cost is bounded, and

a diagnostic that borrows from the measured path has no business appearing in a table whose purpose is to compare placements. Every archived campaign execution therefore carries the diagnostic disabled, as its manifest records.

5.6 Admitting and Reading a Run

A trace becomes evidence only after it has been checked, and the checks come before any statistic is computed. This section states them, together with the window and the **comparator** that the next chapter applies.

5.6.1 Checks a Trace Must Pass

Four conditions are verified on every archived execution. The first is completeness along the chain, meaning that the population the source declared reaches each boundary intact and that the terminal reports the frames the source admitted. The second is the absence of protocol rejections, since the three aware units count a refusal separately for each layer and any non-zero entry would mean the trace contains traffic the contract did not describe. The third is the absence of buffer starvation anywhere, which would indicate that a pool had been undersized rather than that a unit had been slow. The fourth ties the two coder chronologies to the application population, because each of them records a private handshake before the sequence and a drain after it, so the number of pictures a coder reports exceeds the number of application frames by a known and fixed amount. A trace that satisfies the four is admitted whatever it shows. A trace that fails any of them is either repaired or reported as a failure, and one condition in the campaign is retained precisely because it does not satisfy the first, since a stress execution that breaks completeness at a known boundary is a result and not an accident.

5.6.2 The Window and the Comparator

Statistics are computed over a **steady window** rather than over the whole sequence. The opening frames carry the warm-up of the regulator, the settling of the coder, and the first arrivals at every queue, while the closing frames carry the drain that the terminal marker starts, and neither describes the operating point. The interval therefore runs from the tenth admitted frame to the two hundred and eighty-fifth, which leaves the transient outside on both sides without trimming so much that a rare event would be excluded by construction. Reduction happens inside a run

before it happens across runs. Writing $x_{r,k}$ for a quantity observed on frame k of repetition r, and W for the steady window itself, the campaign compares

$$m_r = \operatorname{med}_{k \in W} x_{r,k}, \qquad M = \operatorname{med}_r m_r, \qquad \Delta = \left[\min_r m_r,\ \max_r m_r \right], \tag{5.1}$$

so that M is the median of the run-level medians and Δ is the envelope the repetitions expose. Reducing inside a run first is what prevents a single execution with more observations, or with a different phase, from dominating a pooled distribution. The envelope is reported beside the comparator rather than discarded, because a difference that falls inside it is not a finding. The median is preferred to the mean throughout, and not as a matter of taste. A stage that is normally quick and occasionally blocked produces a distribution with a long upper tail, in which an arithmetic mean describes neither the ordinary case nor the rare one, whereas the median states the condition the chain is in for most of its frames. Where the shape of that distribution matters rather than its centre, the quartiles and the extreme percentiles are reported beside it, as the instrumentation of Chapter 3 already required. Two further rules govern which quantities may be set beside which. Intervals recorded by one unit overlap by construction, as Chapter 4 stated, so a stage is compared with the same stage elsewhere and never added to a sibling that contains it. Logical payload, complete datagram, and reference size are three different denominators, so a throughput computed against one of them is never quoted against another, and the denominator is always named.

5.7 What These Measurements Are Not

Before the results are read, the boundaries of what they can support must be stated, because several of the quantities the campaign reports are more fragile than they look, and the next chapter would otherwise invite conclusions the archive cannot be asked to support, however plainly they seem to follow from it.

5.7.1 The Condition of the Machine

Everything measured in time on this platform is a property of this host during the session that produced it. The four cores are occupied completely, the two coder children share siblings with native loops, and the graphics processor serves two units, so the absolute value of a terminal delay, the instant at which the regulator first leaves its idle state, and the volume of acceptance pressure the source records are all outcomes of a particular arrangement running on one specific machine in a particular thermal and driver state. Another device will not reproduce them. A machine with more logical processors would dissolve the sibling sharing that the

placement table encodes, a different graphics generation would change the accelerator intervals and the behaviour of the hardware coder, and a different storage stack would change what page residency is worth. Even on this host, two executions of the unchanged configuration differ, which is why they were repeated and why the envelope is published. What does travel is the shape of the comparison rather than its magnitude. The ablation holds one host, one boot session, one graph, and one build apart from two constants, so the direction and the mechanism it exposes are properties of the placement rather than of the afternoon. A reader repeating the campaign elsewhere should therefore expect different numbers and the same structure, and should treat any claim of the first kind as local to the archive that carries it. The fidelity statements are the exception in one respect and not in another. They do not depend on the load of the machine, since they are computed after the stream from retained material, yet they do depend on the generated device code and on the hardware coder, so they are reproducible on the same hardware and comparable, but not bit-identical, on other hardware.

5.7.2 Frontiers That Do Not Compose

Three pairs of quantities are easily confused and are never combined. The delay from acquisition to the terminal is a native measurement that ends when a frame is complete at the last unit, whereas the browser-side quantity ends when a draw call returns, which is a proxy for presentation and not a measurement of a displayed pixel. They answer different questions and neither substitutes for the other. The asynchronous frontier of the coder spans submission, scheduling, muxing, and the first evidence of compressed output, so it overlaps with the active work of the same frame and must not be added to it. The reverse loops are two, not one, and the delay from a command to its effect at the reconstruction unit is measured natively, while the delay a viewer perceives includes the return path and the rendering, so the two are reported apart and never summed together.

5.7.3 The Two Levels of Comparison

The campaign supports two kinds of statement and they are not interchangeable. The comparison with the application-level chain is an observation about two complete architectures measured at a frontier both express, and it cannot be attributed to any one mechanism, because the two systems differ in their transport, their process graph, their control loops, and their deployment as well as in the place where the geometric frontier falls. For that reason only a quantity defined at a frontier that both architectures express can support a direct numerical comparison between them. The contrast between the two builds of this platform is the opposite kind of

statement, since everything except two constants is held, so what it finds belongs to the placement alone and is held firmly apart from the architectural differences that the first comparison necessarily carries with it. The chapter that follows keeps the two separated throughout, and any figure quoted from the first is accompanied by the caution that belongs to it, wherever in the text it appears. The distinction limits the claim that may be entered and not the measurement.

5.8 Repeating the Campaign Elsewhere

The archive is only half of a reproducible result. The other half is the set of conditions that produced it, and a reader who wishes to repeat the campaign on another machine has to change some of them deliberately rather than discover by accident that they were wrong for the machine in front of them. The distinction that matters is between what every host has to do in the same way and what each host has to settle for itself, and the two are kept separate here.

5.8.1 Bringing the Experiment Up

The sequence begins outside the platform. Three hundred consecutive frames of **Loot**, obtained from the **8iVFB** collection under its own licence, pass through the offline converter described in Chapter 4 before streaming begins [11]. The repository deliberately excludes that material, so preparation is a precondition rather than a convenience. Isolation comes next and requires a restart. The helper installs the boot parameter, the machine is rebooted, and the kernel command line is inspected before anything is collected, since otherwise the isolation state of an archived result cannot later be established. The entry script then reads the capture switch from the environment, resets the caches, reserves the hugepages, declares the six adjacencies, builds the image, and starts the seven units in the dependency order of Chapter 4. Under the interactive family a browser attaches to the exposed page before the launcher releases the source, and the viewer drives the orientation loop throughout the stream. The capture family omits that step. The teardown script removes the containers, endpoints, binaries, and **hugepage** files, leaving the machine ready for the next run with unchanged isolation state. It leaves the isolation parameter untouched, so another execution may follow without a restart.

5.8.2 Adapting the Build to Other Hardware

Four kinds of change are required on a different machine, and each redefines the experiment to some degree. The first concerns generated device code. The makefile targets one compute generation, so a graphics processor from another family may

reject the binary or lack the capability for which it was compiled, and the target must then be changed and recorded. The host target follows the same principle, since a binary optimised for the compiling machine is not portable across instruction sets. The second concerns the hardware coder. The compression unit selects a low-latency preset and a constant rate, both interpreted by a specific generation of the encoding engine and media framework, so different hardware may produce a different bitstream from identical input. This does not invalidate a runtime comparison within one machine, but prevents archives from being compared at the level of coded bytes. The third concerns placement. The assignment in Table 5.1 assumes eight logical processors with a particular sibling structure, and a host with a different topology cannot use it unchanged. Redrawing it changes the condition rather than merely adapting it, so an archive produced under another assignment remains internally valid but externally incomparable. The fourth concerns the isolation set, which must name the processors used by the new placement while leaving one outside for housekeeping. The two capture-enabled executions add one further dependency. The offline assessment confines each numerical library to one thread, preventing it from silently acquiring host resources outside the measurement budget, so omitting that restriction produces a faster assessment under a different condition.

5.8.3 The Manifest a Result Carries

No archive is complete without the conditions that produced it, and that information cannot be recovered from the records, since a per-frame table states what happened and not what was configured. Listing 5.2 gives the manifest, grouped by the moment at which each of its items is settled and not by subject.

```
# Kept across a restart
isolcpus from /proc/cmdline, kernel, CPU and SMT topology, GPU, driver

# Established by the entry script
hugepages, cache reset, cpusets, lcores, bridge pinning and nice, QUALITY_CAPTURE

# Compiled into the binaries
CPU_ARCH, CUDA_ARCH, compiler and device flags, toolchain versions
CACHE_MODE, WARM_MODE, NETWORK_PROCESSING, OFFLOAD_MODE
TEMPORAL_ADAPTATION, FFMPEG_CPU, TARGET_FPS, K_FRAMES
POINTS_PER_PACKET, NETWORK_MTU, MEDIA_PAYLOAD_SIZE, BURST_SIZE
MAX_ZERO_ACCEPTS, descriptor and queue depths, regulator constants
coder rate and picture group, DEBUG_VISUALS, DEBUG_FRAME_ID, gauge

# Produced by the execution itself
the nine per-frame tables, the two coder chronologies, their stderr
```

Listing 5.2. Conditions archived beside every execution

Two items in that list are worth singling out because they are invisible in the data. The isolation state appears nowhere in a per-frame table, so an archive that omits it cannot be placed beside another, and the diagnostic switch of the accelerated units likewise leaves no trace in the records although it changes what the selected frame does. Everything else in the manifest is a value somebody chose, and the discipline the campaign asks for is simply that the choice be written down at the moment it is made rather than reconstructed afterwards from a folder name. The grouping serves the same end, since an entry decided at boot and an entry compiled into a binary are not recovered in the same way by whoever repeats the campaign later, and the two are looked for in different places when a result is questioned.

5.9 What the Next Chapter Reports

The campaign is now fully specified. Fifteen executions were performed, three repeating one operating point unchanged, two repeating the placement ablation, two attaching the fidelity capture to either side of it, and eight each departing from that point in one stated respect. A sixteenth exists only to be looked at. The window is fixed, the comparator is defined, the checks that admit a trace are stated, and the boundaries beyond which none of it may be pushed have been set out before a single number has been quoted. Three things are worth carrying into that reading. The envelope of the repeated condition comes before any comparison, the two levels of comparison are never mixed, and every quantity measured in time belongs to the machine that produced it. Chapter 6 reads the archive in that order. It establishes the empirical envelope first, since nothing else can be interpreted without it, then places the nominal condition beside the chain of Chapter 3 at the one frontier both express, then turns to the paired ablation that carries the claim of this thesis, and finally reports what the two capture-enabled executions found. The sensitivity conditions follow, each answering its own narrow question.

Chapter 6

Results

6.1 The Traces Admitted to Analysis

Sixteen executions were performed under the protocol of Chapter 5, and every one of them was audited before a single quantity was extracted from it. Fifteen belong to the campaign proper and one exists only for visual inspection. The audit found no protocol rejection at any aware boundary, no buffer exhaustion at any unit, and no disagreement between the coder chronologies and the application population, so the archive is interpretable as a whole rather than trace by trace. Completeness along the route holds everywhere except in the two conditions that raise the datagram rate, where the terminal receives fewer frames than the source released. Those two are retained deliberately, because a break located at a known boundary is itself a measurement, and the later part of this chapter establishes exactly where it occurs. What follows is ordered as the preceding chapter announced. The variability of the repeated condition comes first, since no later difference can be read without it, after which the chain is set against the application-level reference at the only boundary the two have in common. The paired **ablation** then carries the argument of the thesis, the capture-enabled pair examines whether the reconstruction survives the move, and the sensitivity conditions bound the operating point from either side. Interaction closes the sequence, because it is the only quantity that travels the other way. Every figure below is drawn from that archive, apart from the value set by the application-level pipeline, taken from its own logs.

6.2 The Repeated Baseline

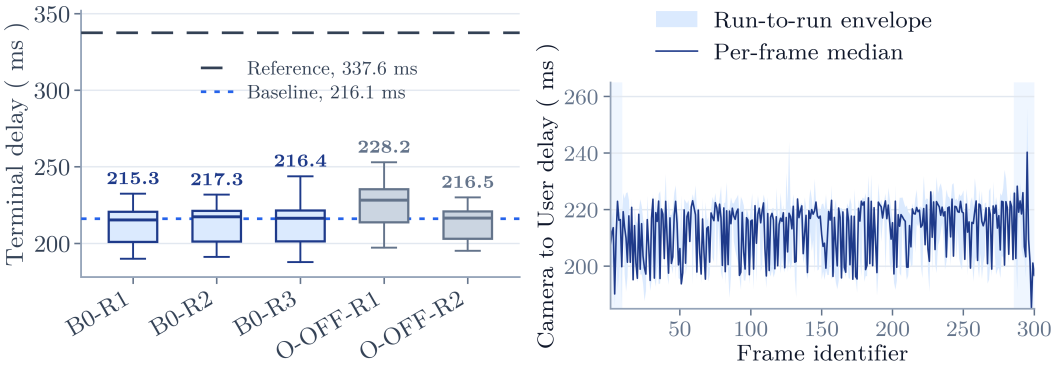
Three executions of the nominal build were collected in one boot session. Table 6.1 reports each repetition at the terminal frontier, its source cadence and frames, and describes one operating point, not three experiments.

Execution	All-frame median	Steady median	Steady 95th	Cadence
B0-R1	215.257 ms	215.257 ms	225.741 ms	30.001 fps
B0-R2	217.000 ms	217.283 ms	225.820 ms	30.000 fps
B0-R3	216.104 ms	216.371 ms	227.347 ms	30.000 fps
O-OFF-R1	228.866 ms	228.177 ms	244.868 ms	30.003 fps
O-OFF-R2	216.592 ms	216.523 ms	225.900 ms	29.999 fps

Table 6.1. Terminal frontier of the repeated conditions

6.2.1 Spread of the Unchanged Configuration

The three repetitions place the all-frame median between 215.257 and 217.000 ms, a band of 1.743 ms whose relative width is close to four parts in a thousand, and the median of those three medians is 216.104 ms. Computed over the steady window the same quantity becomes 216.371 ms, the band widens marginally to 2.026 ms, and neither figure moves by more than one part in a thousand from its all-frame counterpart. That agreement is worth a remark of its own, because it shows that the warm-up and the drain of this platform occupy a small enough share of the sequence to leave the central value almost untouched, which is not true of every stage taken separately. Figure 6.1 displays the same evidence in two ways, as the per-run distribution of the terminal delay against the reference level and as the band that the three repetitions trace frame by frame.



(a) Distribution of each execution **(b)** Envelope across the sequence

Figure 6.1. Terminal delay of the repeated and ablated conditions

Two features of Figure 6.1a deserve attention. The boxes of the three baseline repetitions overlap almost completely, so no repetition is exceptional, and the whiskers reach roughly 190 and 245 ms without producing a tail that would make the median unrepresentative. Figure 6.1b shows why the **envelope** matters more than any single run. The band between the fastest and the slowest of the three

repetitions is narrow through the whole steady window and widens only at the very end, where the terminal marker drains what remains in the chain. A difference smaller than that band is therefore not a finding, and the ablation examined later is read against exactly this threshold rather than against a single trace chosen for its favourable value. The three runs are never averaged into one.

6.2.2 Cadence Sustained at the Source

The absolute schedule of the source is the second property the baseline has to establish, since a comparison of delays means little if the cadence underneath differs. Figure 6.2 reports the selected-frame rate measured over the steady window for all fifteen executions of the campaign, the two capture-enabled controls included, since the schedule of the source is the one quantity on which the two of them can be read beside the runs that carry no instrumentation at all.

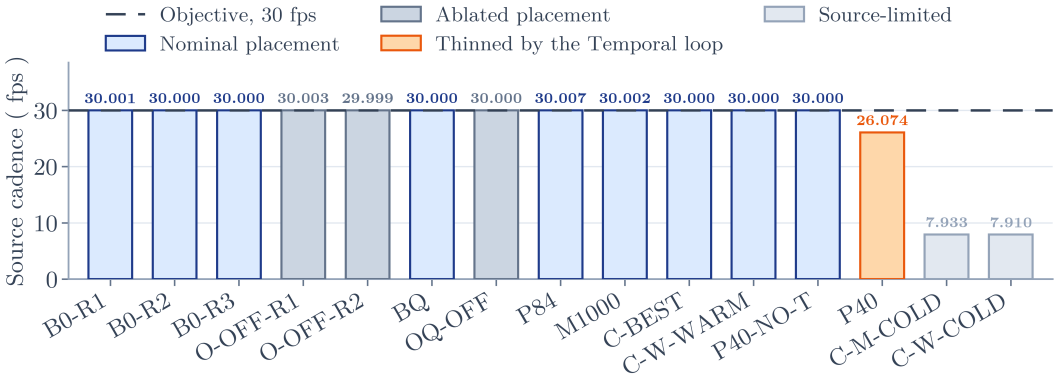


Figure 6.2. Source cadence held across the campaign

Twelve of the fifteen conditions sit within three hundredths of a frame per second of the nominal rate, which is the accuracy an absolute timeline can hold on a machine whose processors are all occupied. Three depart from it, each for a cause the campaign anticipated. The two cold conditions fall to 7.933 and 7.910 fps because the source waits on storage before every deadline, so the chain is starved rather than congested. The elevated datagram rate settles at 26.074 fps because the regulator thins the source over the three intervals in which it raises the factor. No other execution shows a cadence effect, so the differences discussed in the rest of this chapter reflect changes in how a frame is processed and not in how often one is offered. The schedule underneath the campaign never drifts. The three repetitions of the nominal build differ from one another by less than a thousandth of a frame per second, and the interval between two departures has a median of 33.491, 33.385, and 33.417 ms against a budget of 33.333 ms, so the source holds its own clock to

within a fifth of a millisecond over the whole sequence, which is the property that every later comparison of delays in this chapter depends on.

6.3 The Chain Against the Reference Implementation

The application-level pipeline of Chapter 3 was measured on the same host with its ordinary probes active, and its reception frontier, joined frame by frame from the source and terminal logs it produces, reaches a median of 337.599 ms over the complete sequence and 336.909 ms over the steady window. Against the repeated baseline of 216.104 ms the reduction is 35.99 per cent, and the three repetitions taken individually give 36.24, 35.72, and 35.99 per cent, so the figure is stable to a quarter of a point across the envelope above. The comparison is drawn at the moment a complete reconstructed frame becomes available natively, which both systems express and neither reports as a displayed pixel, and it avoids the terminal of the reference that includes its own presentation work. Two limits attach to the number and both were stated before it was computed. The two systems differ in transport, in process decomposition, in control arrangement, and in deployment as well as in the site of the geometric frontier, so the reduction belongs to the redesign taken whole and cannot be apportioned among them. It is also tied to this host, the reported conditions, and this sequence, which is why the ablation reported below holds all three of them fixed and changes one thing only.

6.4 The Placement Ablation

The pair that carries the claim of this work keeps every constant except the two switches that decide where the geometric frontier is resolved. Both repetitions of the ablated build were audited as the baseline was, both sustain the nominal cadence, and both deliver the complete sequence, so the comparison rests on runs of equal standing. Neither trace was repaired after the fact. Where a distribution along the route needs one baseline trace, the figures draw the first repetition rather than the most favourable one, and they draw it descriptively, since every quantitative comparison here rests on the median of the three run medians that Chapter 5 defines.

6.4.1 Where the Computation Is Paid

The first question is whether the work actually moves, and the answer is recorded independently at every unit. The quantity that records it is the active interval each unit charges to a frame, and its definition is deliberately local rather than uniform. At the source it is the aggregate of acquisition, serialisation, and the wall-clock

duration of the transmission, at a forwarding **placement** it is the per-packet work performed on the traffic that crosses it, and at the two accelerated units it is the host-side work that precedes and follows the device, with the asynchronous coder excluded because it runs beside the loop rather than inside it. Those definitions are not interchangeable and the chapter never treats them as such, yet each of them is stable across the two builds, which is the only property an attribution needs. Figure 6.3 shows the median of that interval for each of the nine placements over the steady window, on a logarithmic scale, where the contrast between the units that change and those that do not is easiest to see at a single glance. Each bar is a median over the two hundred and seventy-six frames the steady window retains, and the axis spans four decades, so a unit that charges microseconds and one that charges tens of milliseconds can be read beside each other without either being flattened. The nine placements are listed as the route visits them.

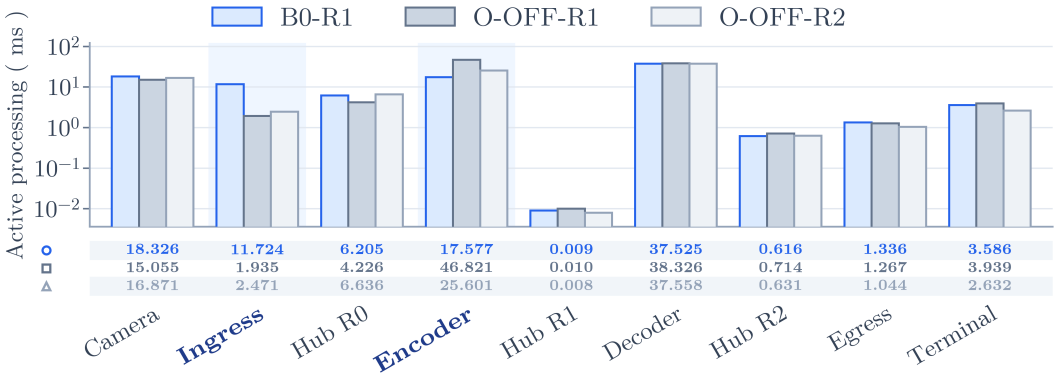


Figure 6.3. Active processing along the route

The two units that the switches address behave as the design predicted. The ingress node falls from 11.724 ms to 1.935 and 2.471 ms, retaining only the classification, the envelope handling, and the forwarding that it performs in either build, while the compression unit climbs from 17.577 ms to 46.821 and 25.601 ms. Nothing else moves in a direction that the placement explains. The hub pays the same negligible amount on the media route in both builds, the reconstruction unit differs by less than a millisecond, and the exit boundary is lighter in the ablated runs than in the baseline, which no account of the **offload** would predict and which the source variation of the same runs already explains. In the first ablated repetition only two of the nine bars change by more than a factor of two, and they are the two the ablation addresses. Taken as proportions of the baseline, the compression unit pays one hundred and sixty-six per cent more in the first ablated repetition and forty-six per cent more in the second, while the ingress node gives up between eighty-three and seventy-nine per cent of what it charged before. Table 6.2 gives the same values

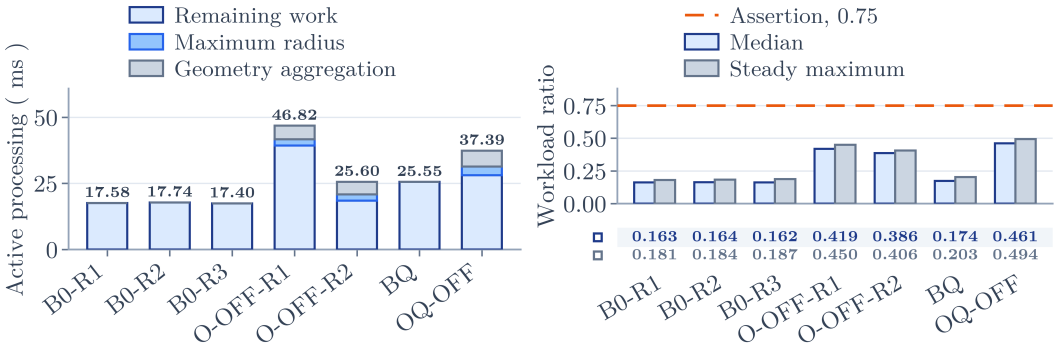
in figures, together with the reading that each placement supports. The nine rows follow the order of the figure above. One caution belongs here, and it repeats a rule rather than introducing one. These nine values are local attributions and several of them overlap in time, so they may be compared column by column within a row and must never be added along a column to yield a delay. The quantity that does accumulate is examined later in this section. That distinction governs every column of the table that follows and every bar of the figure above.

Route unit	Baseline	First ablated	Second ablated	Placement reading
Camera source	18.326 ms	15.055 ms	16.871 ms	Source-side variation
Ingress node	11.724 ms	1.935 ms	2.471 ms	In-path geometry off
Hub, route 0	6.205 ms	4.226 ms	6.636 ms	Proxy cost stays local
Encoder unit	17.577 ms	46.821 ms	25.601 ms	Frontier lands here
Hub, route 1	0.009 ms	0.010 ms	0.008 ms	Media relay is free
Decoder unit	37.525 ms	38.326 ms	37.558 ms	Rebuild is untouched
Hub, route 2	0.616 ms	0.714 ms	0.631 ms	Point relay is light
Egress node	1.336 ms	1.267 ms	1.044 ms	Exit is indifferent
User terminal	3.586 ms	3.939 ms	2.632 ms	Terminal follows use

Table 6.2. Active processing charged to each placement

6.4.2 The Margin Recovered at the Compression Unit

What the ablation costs the consumer is visible in the composition of its work and not only in its total. Figure 6.4 decomposes the active interval of that unit into the two explicitly attributed geometric fields and the remainder, and sets the smoothed occupancy of its budget beside them for the same executions.



(a) Composition of the interval **(b)** Occupancy of the budget

Figure 6.4. Composition of the work at the compression unit

Under the nominal placement the two geometric fields are empty, because the values arrive with the traffic and the unit reads them from the descriptor. Under the ablated placement they carry 5.110 and 2.391 ms in the first repetition and 4.761 and 2.411 ms in the second, which is the aggregation over the whole cloud followed by the **radius** that depends on the **centre** it produces. Those two terms account for almost the whole of the increase in the second repetition and for a quarter of it in the first, and the balance falls on the remainder, where a longer traversal, a colder working set, and the scheduling consequences of both are paid without being separable. The occupancy of the budget rises from 0.163 to 0.419 and 0.386, an increase of one hundred and fifty-seven and one hundred and thirty-seven per cent, and its steady maximum rises from 0.181 to 0.450 and 0.406. None of those maxima reaches the assertion level of 0.75, so the regulator never leaves its idle state in either build, and the whole of the effect is absorbed as lost **headroom** rather than as a change of admitted rate. The two capture-enabled bars in the same figure repeat the experiment under instrumentation and may be read only against each other, since the retained material inflates both. Within that pair the expected trend appears, from 25.552 to 37.393 ms and from 0.174 to 0.461, which reproduces the placement cost once more. These runs never stand beside the others.

6.4.3 Residency and the Timing Behind It

Active processing counts only the work a unit performs. **Residency** counts everything that happens to a frame between its arrival and its departure, waiting included, and the two answer different questions about the same instant. Figure 6.5 gives the residency distribution of every placement under both builds, on a logarithmic axis that lets units three orders of magnitude apart share one panel.

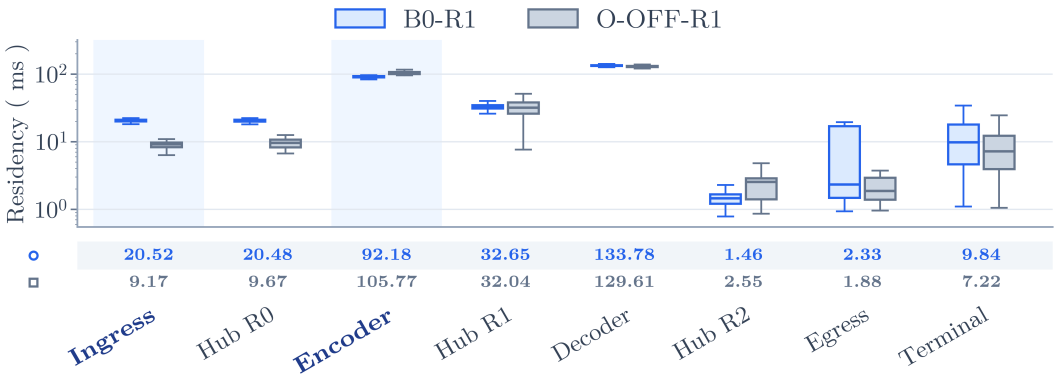


Figure 6.5. Residency of a frame at each placement

The ingress node halves its residency when the frontier leaves it, from a median of 20.52 to 9.17 ms, and the hub that follows it does the same, since a frame can only leave the hub once the node before it has finished. The compression unit moves the other way, from 92.18 to 105.77 ms, and its distribution widens as well, which no measurement of active work alone would have revealed. The reconstruction stage is the heaviest placement in either build at about 134 and 130 ms and does not react to the change, while the three light placements at the end of the route keep distributions that overlap almost completely. The effect of the placement is therefore localised rather than uniform, redistributing residency around the frontier without shifting the downstream stages as a group. Figure 6.6 separates the phases that the compression unit reports for every frame it handles.

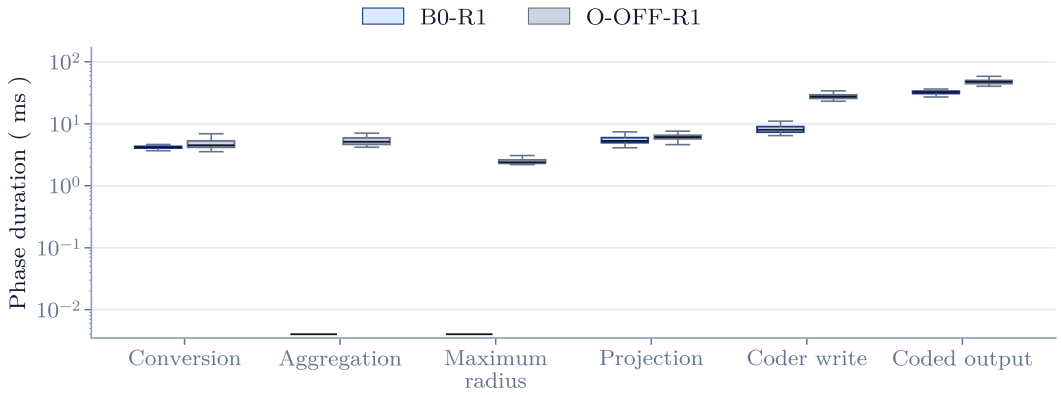


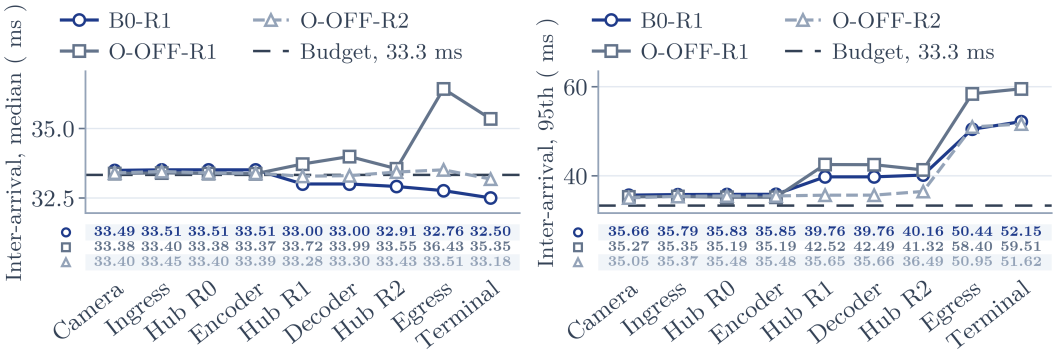
Figure 6.6. Phases of the compression unit

Two of the six phases are empty in the nominal build and populated in the ablated one, which is the offload seen from the consumer side, and they contribute medians of 5.110 and 2.391 ms. Payload conversion and projection grow only slightly, from 4.182 to 4.508 ms and from 5.285 to 6.096 ms, so the accelerated stage is almost indifferent to where its parameters originate. The last two phases change more markedly. The write that feeds the coder rises from 8.020 to 27.344 ms and the interval ending with the first compressed bytes from 32.158 to 47.800 ms, although neither is geometric work. They instead reflect a pipe receiving material later and a coder falling behind its input cadence. The cost of withdrawing the offload therefore propagates beyond the arithmetic it restores, into a stage the ablation does not otherwise modify, and the terminal behaviour follows from it.

6.4.4 The Pace Observed Along the Route

A frame that arrives late at one unit is not necessarily late at the next, so the cadence has to be examined boundary by boundary rather than at the terminal

alone. Figure 6.7 reports the interval between consecutive arrivals at each placement, as a median and as a ninety-fifth percentile, for the baseline and for the two ablated repetitions. The budget of the nominal rate is drawn on both.



(a) Median spacing **(b)** Ninety-fifth percentile

Figure 6.7. Arrival spacing at each boundary

Through the first half of the chain the three traces are indistinguishable and sit on the budget of 33.333 ms that the nominal rate allows. They separate after the compression unit. The baseline drifts slightly below the budget from the media route onwards, ending at 32.50 ms at the terminal, which is the signature of a chain that has caught up with a schedule it never fell behind. The first ablated repetition drifts the other way, reaching 36.43 ms at the exit boundary and 35.35 ms at the terminal, so it delivers frames more slowly than the source offers them and accumulates the difference. The second ablated repetition stays within a third of a millisecond of the budget throughout. Figure 6.7b tells the same story with a sharper edge, since the three traces agree to within half a millisecond as far as the hub and then diverge at the exit boundary, where the first ablated repetition reaches 58.40 ms against 50.44 ms for the baseline. That is where the asynchronous behaviour of the coder, the depth of the pipe behind it, and the publication frontier of the terminal combine, and it is the only region of the route in which the placement is visible in the spacing of arrivals rather than in the work charged to a unit.

6.4.5 Progressive Frontiers of a Frame

The quantity that does accumulate along the route is the time elapsed since the source dated the frame, measured at the exit of every unit that handles it. It is obtained from a single clock rather than by composition, since every datagram carries the instant of capture written by the source and each unit subtracts that instant from its own exit, so the curve is cumulative by construction and owes nothing to the attributions of the preceding subsections. Two consequences follow. A stage

that overlaps its predecessor contributes only the part of its work the predecessor did not already cover, and a stage that waits contributes the wait as fully as the work, which is what makes the shape of the curve more informative than any of its individual steps. The exits of the hub are the clearest instance, since they sit almost exactly on the exits of the units they surround and confirm that relaying costs the route nothing worth measuring. Figure 6.8 follows that quantity through the whole chain under the two placements of the geometric frontier.

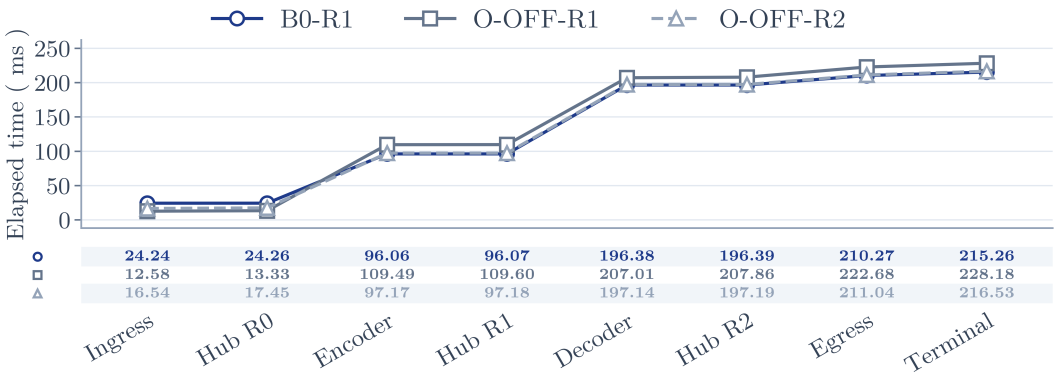


Figure 6.8. Elapsed time at successive exits

Two plateaux dominate the profile, the compression unit and the reconstruction unit, which together account for more than four fifths of the terminal delay in every build. Against that background the ablation appears early and persists. The ingress node completes at 24.24 ms in the baseline and at 12.58 and 16.54 ms when it no longer resolves the frontier, which is the arithmetic it has stopped performing. By the exit of the compression unit the ordering has reversed, from 96.06 ms in the baseline to 109.49 ms in the first ablated repetition, and that separation of about thirteen milliseconds is carried unchanged through the reconstruction unit, the hub, and the exit boundary. The second ablated repetition recovers most of its early advantage at the compression unit and ends only 1.27 ms above the baseline. The two agree on where the difference is created and disagree on how much survives to the terminal, a distinction the next subsection draws.

6.4.6 What the Pair Establishes

Three statements survive the pair and a fourth does not. The first is that the work moves, since the ingress node loses between nine and ten milliseconds of active processing and the compression unit gains between eight and twenty-nine, with the two geometric quantities that the telemetry names apart appearing at the consumer exactly when they leave the path. The second is that the effect is a matter of

headroom, since the smoothed share of the budget grows to some two and a half times its nominal value in both repetitions while the admitted rate, the population of every frame, and the cadence remain untouched. The third is that the direction at the terminal is reproducible, since both ablated repetitions end above the baseline median and one of them ends above the whole baseline envelope. What does not survive is a fixed magnitude at the terminal. The two repetitions differ from the baseline by 11.81 and 0.15 ms in the steady window, and the second of those is well inside the interval of 2.026 ms that the repeated condition itself covers, so the honest reading is that removing the offload always costs the consumer its margin and only sometimes costs the viewer a measurable delay. That asymmetry is not a defect of the experiment. It follows from a pipelined chain in which the slowest stage sets the pace, and it is the reason this work reports the local attribution and the terminal frontier as two findings rather than one, never combined.

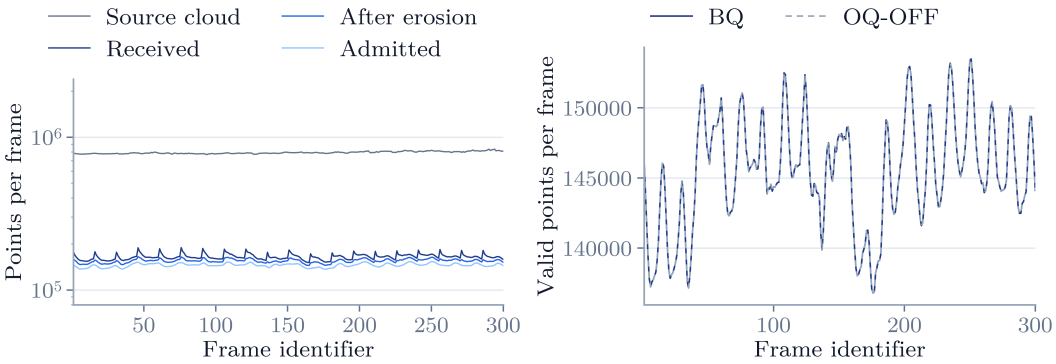
6.5 Fidelity Across the Two Placements

The runtime evidence says where the arithmetic is paid. It says nothing about whether the arithmetic is the same, and that question is answered by the capture-enabled pair, which repeats the ablation with the fidelity path armed and the presentation bridge absent. Because the two executions differ in the site of the frontier and in nothing else, an equality between them is the outcome the design predicts and a divergence anywhere would refute it.

6.5.1 Population and Coded Signal

The strictest of the three levels is also the simplest. The terminal admits 43 620 972 points over the sequence under the nominal placement and the same total under the ablated one, and the two agree frame by frame in all 300 instances rather than only in aggregate. That agreement fixes how many points survive rather than where they lie, so it is the first of the three levels and not the whole of them, and the distances reported later in this section are what place the surviving points in space. The two intermediate populations behave differently, and the difference is worth stating because it bounds what the identity means. The points that reach the reconstruction unit total 49 531 602 under the nominal placement against 49 516 558 under the ablated one, and the points that survive the erosion filter total 46 561 369 against 46 555 722, so neither of those two counts is identical across the pair. Only the population the terminal admits is, which is also the only one of the three that the published descriptor is required to determine. Figure 6.9 places that population in the context of the whole route, from the cloud the source holds to the points the

terminal finally keeps and publishes to the page, with the two intermediate stages of the reconstruction drawn between them on the same axis.

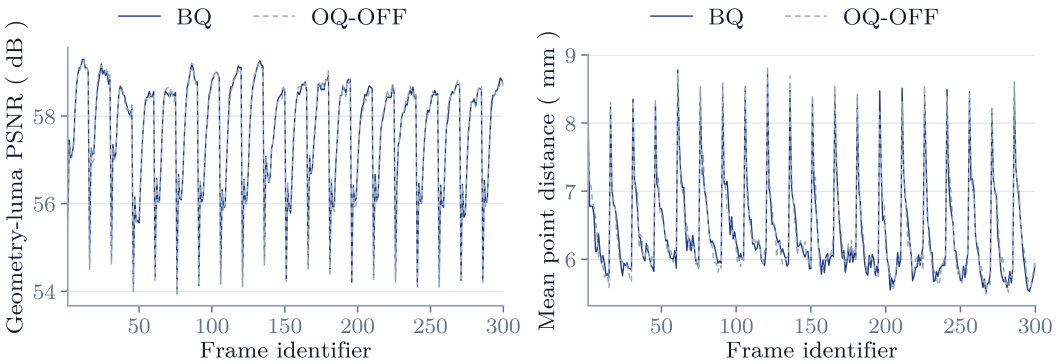


(a) Population along the route **(b)** Valid points across the pair

Figure 6.9. Point population

Figure 6.9a explains why the identity is a strong test rather than a trivial one. A source frame carries about 790 000 samples, the six-view projection retains the roughly 164 000 of them that survive the depth arbitration and reach the terminal, the erosion filter removes some five per cent of those, a validity test a further six, and about 146 000 are finally admitted. Every one of the three reductions depends on the frontier the descriptor announces, so an error of a single unit in the scale or the centre would move the last curve visibly. Figure 6.9b sets the two capture-enabled executions one over the other, and the dashed trace never leaves the solid one at any point of the sequence. Figure 6.10 carries the second level, the indicators computed on the geometry stripe of the composed picture and the mean distance measured on the rebuilt clouds after alignment. Both are obtained after the stream has closed, from material the two units retained while it ran, so neither of them competes with the schedule it describes. The stripe indicators are evaluated on the depth band alone, taken from the top of the picture the coder received, which leaves appearance and the binary mask outside a figure meant to describe the surface a viewer actually sees. The distances are evaluated on the complete clouds, not on a sample of them, and they are therefore the strictest statement the campaign is able to make about what any reconstructed frame of the sequence contains. The retention itself is bounded and was measured while it ran. The terminal copies each complete frame into a reserved region in a median of 2.256 ms under the nominal placement and 2.320 ms under the ablated one, with maxima of 18.233 and 17.394 ms, every one of them below the 33.333 ms period of the source, while the reconstruction unit holds its own copy in memory until the stream has closed. That cost is charged after the terminal has published the frame it describes, so it is reported beside the frame

rather than inside it and is excluded from every runtime quantity of this chapter, which is what allows the capture-enabled pair to be read as a fidelity control rather than as a second runtime condition of the campaign proper.



(a) Coded signal **(b)** Rebuilt geometry

Figure 6.10. Coded and reconstructed fidelity

The two curves are superimposed for the whole sequence in both panels, and the sawtooth they share is the picture group of the coder rather than an effect of placement. Averaged over the sequence the **MSE** of the stripe moves from 0.116533 to 0.116633, the **PSNR** from 57.6932 to 57.6876 dB, and the **SSIM** from 0.997945 to 0.997948, so the fourth significant figure is the first that changes in any of them. The mean point distance after the robust alignment moves from 6.3873 to 6.3965 mm, less than a hundredth of a voxel on a grid whose spacing is 1.820 mm. Figure 6.11 reports the three indicators as distributions rather than as averages. The boxes coincide on all three panels, the whiskers end within a few hundredths of one another, and the scattered frames below them appear in both executions.

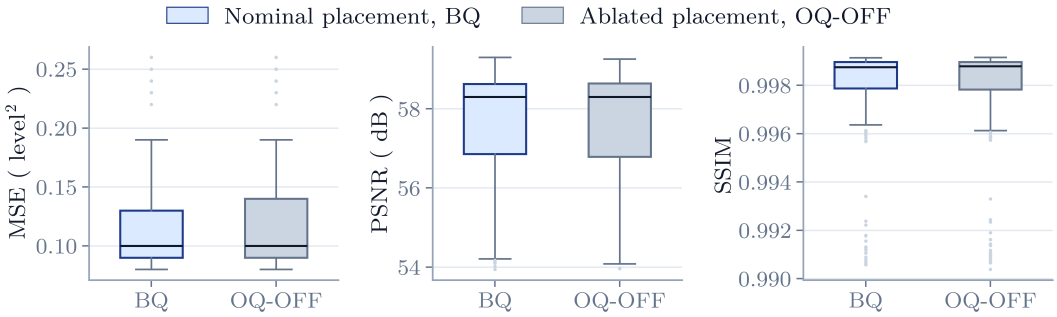


Figure 6.11. Distribution of the geometry-luma indicators

6.5.2 Deltas Taken Frame by Frame

An average can conceal a compensation, so the pair is also examined as a difference taken on matching frames. Table 6.3 reports that difference for seven indicators, as

the median of the signed values and as the ninety-fifth percentile and maximum of their magnitudes. The pairing is made by frame identifier.

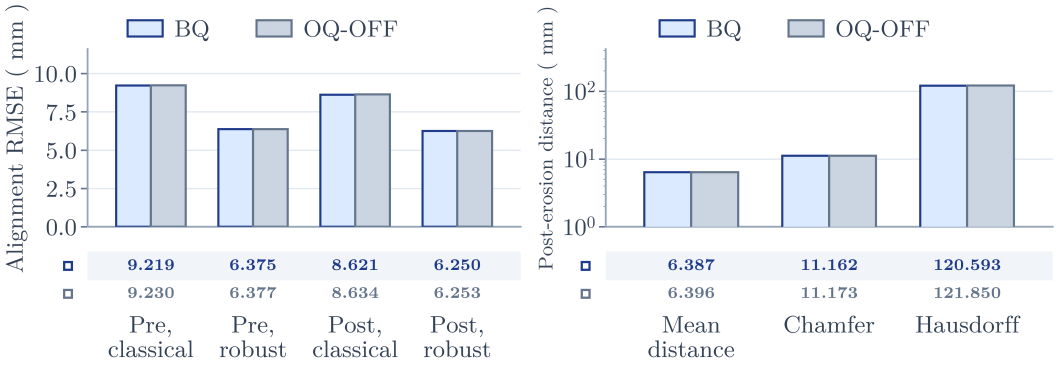
Indicator	Median change	95th of the size	Largest size
Stripe MSE	0.000 level^2	0.010 level^2	0.010 level^2
Stripe PSNR	−0.010 dB	0.140 dB	0.260 dB
Stripe SSIM	$+8 \cdot 10^{-6}$	$2.8 \cdot 10^{-4}$	$4.7 \cdot 10^{-4}$
Mean distance	+0.016 mm	0.281 mm	0.485 mm
Distance **RMSE**	−0.005 mm	0.208 mm	0.374 mm
Chamfer distance	+0.012 mm	0.382 mm	0.694 mm
Hausdorff distance	+0.999 mm	38.7 mm	257.0 mm

Table 6.3. Change introduced by the ablated placement

Six of the seven rows are centred on zero and bounded by a fraction of a voxel or of a decibel. The seventh behaves differently for a reason that is structural rather than accidental. A Hausdorff distance reports the single worst correspondence in a cloud of some hundred and forty thousand points, so one sample that the erosion filter admits on one side and rejects on the other moves it by tens of millimetres while leaving every averaged quantity untouched. The paired evidence therefore supports the statement the campaign was built to test, namely that relocating the frontier consumes processing margin at the consumer without introducing a systematic alteration of what the viewer receives.

6.5.3 The Two Alignment Modes

The distances reported above are not read from the clouds directly. Each rebuilt frame is first brought onto its source under a rigid registration, and the assessment computes that registration twice, once in coupled form and once in decoupled form. The two definitions answer different concerns, the first measuring how the whole population sits against its reference and the second how the surface does once isolated reconstruction failures are set aside. The decoupled form rejects the correspondences that a statistical filter declares unreliable, at twenty neighbours and two standard deviations, and then applies the transform estimated from the surviving subset to the complete unfiltered cloud, whereas the coupled form reports the radius-gated dispersion of the fit itself. Both are evaluated at the two stages of reconstruction, before and after the erosion filter, which gives the four combinations the picture presents and the thirty-two fields the archive records for every frame. Figure 6.12 compares both, before and after erosion, for the two placements.



(a) Dispersion of the fit **(b)** Distances after erosion

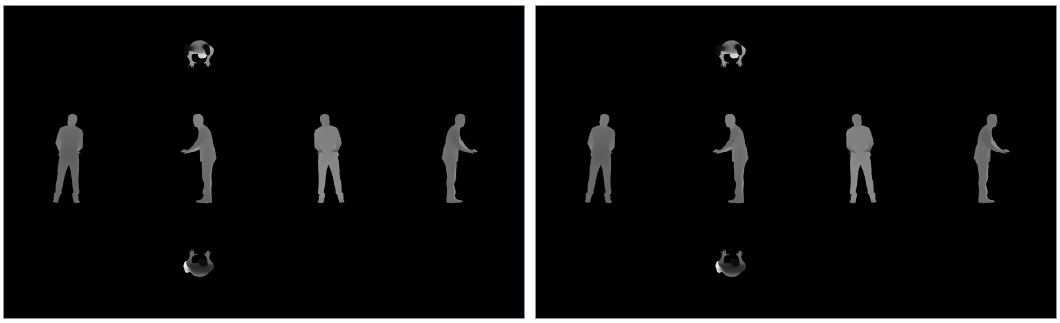
Figure 6.12. Alignment forms

The alignment definition matters far more than the choice of placement, which is the point of showing them together. The decoupled fit lowers the dispersion from 9.219 to 6.375 mm before erosion and from 8.621 to 6.250 mm after it, a reduction of about a quarter that comes entirely from the handful of correspondences the coupled fit allows to dominate a sum of squares. The averaged quantities barely notice the difference, since the mean distance and the Chamfer term move in the third decimal between the two modes, which confirms that the surface itself is where both definitions agree. Against that background the placement is invisible. Taken as relative changes over the sequence, the ablated placement moves the mean distance by 0.144 per cent, the dispersion of the decoupled fit by 0.037 per cent, and the Chamfer term by 0.096 per cent, while the extreme-value distance moves by 1.042 per cent. Each pair of bars in the first part differs by at most thirteen thousandths of a millimetre, whichever definition applies and at either stage, and Figure 6.12b shows the same agreement for the mean and the Chamfer terms across three orders of magnitude, with the extreme-value distance as the single exception. The conclusion the chapter draws from the whole of this section is therefore a conjunction rather than a single claim, namely that the in-network placement returns processing margin to the consumer and returns the viewer a reconstruction that no averaged measurement in the campaign can distinguish from the original.

6.5.4 The Frame Selected for Inspection

Numbers of this kind are hard to relate to the pictures themselves, which is why one frame is preserved on both sides of the coder. The diagnostic execution writes eight artefacts, the complete planar picture consumed by the coder together with the three layers separately, once at the compression unit and once at the reconstruction unit. The picture measures 2560 by 4608 samples and occupies 17,694,720 bytes,

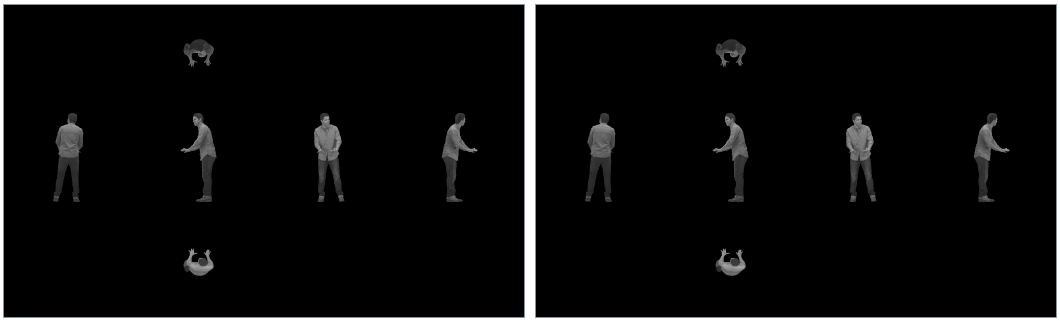
so retaining it costs one memory copy at each unit during streaming and no work on the schedule it documents, and the execution is kept outside the campaign for that reason. Figure 6.13 places the composed **atlas** beside the one recovered after decoding, so the six views can be compared cell by cell.



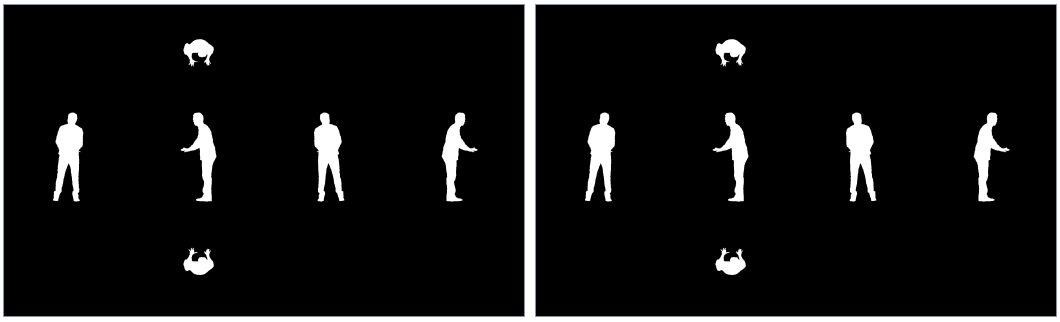
(a) Composed at the compression unit **(b)** Received at the reconstruction unit

Figure 6.13. Depth atlas on either side of the coder

The six views occupy the cells the assembly assigns to them, the subject is framed identically, and no view is missing or displaced. Figure 6.14 repeats the comparison for the two remaining layers, where appearance and the binary mask are shown for the same frame. No scaling was applied to either picture.



(a) Appearance, composed **(b)** Appearance, received



(c) Mask, composed **(d)** Mask, received

Figure 6.14. Appearance and mask across the same frontier

What the eye cannot resolve, the arithmetic can. Table 6.4 compares the two sides sample by sample, over the complete picture and over each of the three layers separately. Deviations are counted in levels of the scale. The assessment uses every sample of the picture rather than a region of interest, so a difference confined to a contour is diluted by the area around it and a difference spread over a surface is not. Both pictures are written once the sequence has finished, from regions the two units reserved before streaming began, which is what allows a byte-level comparison without the retention disturbing the frame it preserves.

Layer compared	Signal ratio	Exact samples	Mean error	Largest
Complete picture	56.13 dB	98.299 per cent	0.040 levels	27 levels
Geometry stripe	58.81 dB	97.875 per cent	0.033 levels	27 levels
Appearance stripe	50.93 dB	96.508 per cent	0.108 levels	24 levels
Occupancy stripe	63.28 dB	99.509 per cent	0.010 levels	13 levels

Table 6.4. Sample agreement for one frame

The ordering of the three layers follows what Chapter 3 predicted of them. The mask survives best, because a binary field quantises to two levels and the coder spends almost nothing describing it. Depth follows, because a physical surface varies smoothly and the transform represents it with few coefficients. Appearance is the weakest of the three, since colour changes abruptly between neighbouring samples and the same rate budget has to cover that variation. Not one of the deviations exceeds 27 levels out of 255, they are confined to the contours of the subject, and the region the mask declares empty is recovered unchanged, which is the property on which the erosion filter of the reconstruction unit depends.

6.6 Sensitivity of the Operating Point

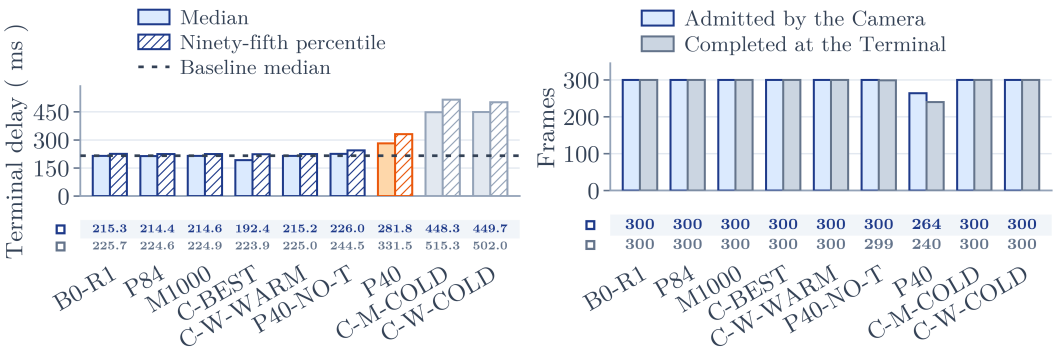
The conditions examined so far hold the operating point fixed. The remaining eight move one constant at a time and ask how far it can be displaced before the chain stops behaving as it did. Table 6.5 records each condition with the cadence it sustained, the delay observed at the terminal, the frames that survived the route, and the source quantity displaced. The baseline heads neither list, since every row is read against it. Taken together, the eight rows form three regimes. Two departures leave the chain essentially unchanged, four move the source without threatening completeness, and two reduce the terminal population, the only outcome in the campaign that a service would notice. The first regime contains the two packetisation departures, whose terminal medians remain within two milliseconds of the baseline

and therefore within the spread of the repeated condition. The second contains the four source variants, where the two warm conditions remain within three milliseconds of the baseline and the two cold ones more than double it without losing a frame anywhere. The third contains the two datagram-rate conditions, the only executions whose terminal population falls short of what the source released and the only ones in which a control loop intervenes or is deliberately disabled.

Condition and departure	Cadence	Steady median	Frames kept
`P84`, larger datagrams	30.007 fps	214.373 ms	300 of 300
`M1000`, media grouping	30.002 fps	214.600 ms	300 of 300
`C-BEST`, no read in the loop	30.000 fps	192.394 ms	300 of 300
`C-W-WARM`, a buffer per frame	30.000 fps	215.157 ms	300 of 300
`C-M-COLD`, cold pages	7.933 fps	448.339 ms	300 of 300
`C-W-COLD`, both penalties	7.910 fps	449.744 ms	300 of 300
`P40`, twice the datagrams	26.074 fps	281.801 ms	240 of 264
`P40-NO-T`, that rate uncontrolled	30.000 fps	225.984 ms	299 of 300

Table 6.5. Conditions that displace the operating point

Figure 6.15 presents the same conditions in two views, showing terminal delay with its dispersion and the frames that reach the end of the route, in the same order. Keeping the ordering fixed makes the latency and completeness effects directly comparable across the sweep, while separating a slower run from one that actually changes the population observed at the terminal.



(a) Terminal delay **(b)** Frames through the route

Figure 6.15. Terminal delay and completeness

6.6.1 Packet Capacity and Media Grouping

Neither of the two packetisation departures disturbs the chain. Raising the point capacity to the largest value the envelope admits reduces the source datagrams from

2 976 979 to 2 835 230, a saving of 4.76 per cent, and lowers the terminal median by about two milliseconds while the cadence holds to seven thousandths of a frame per second. Breaking the alignment of the media grouping costs even less, because the container units are reassembled from the byte stream rather than from the boundaries of a datagram, so the reconstruction unit accepts the residue without a complaint and the terminal median lands within two milliseconds of the baseline. The instructive part of the pair is what it excludes. Two parameters that a reader might expect to dominate a packet-oriented chain move the result by less than the spread of the unchanged configuration, which places them firmly below the placement of the geometric frontier in any ranking of what matters on this host.

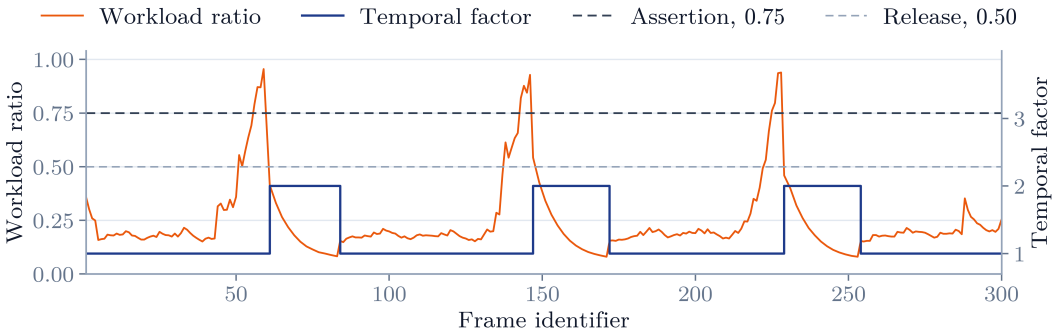
6.6.2 Residency of the Source Material

The acquisition policy behaves in the opposite way and produces the largest effects in the archive. Lifting acquisition out of the timed loop removes the read and the conversion from every iteration, drops the work charged to the source from 18.326 to 10.275 ms, and brings the terminal median to 192.394 ms, some twenty-four milliseconds below the baseline. That condition is a bound and not a recommendation, since it describes a source whose material is already in memory and therefore answers a question about the platform rather than about a streaming service. Allocating and releasing a buffer at every iteration costs almost nothing once the pages are resident, which the third condition establishes at 215.157 ms. The two cold conditions are decisive. A read from storage that has not been mapped in advance takes 115.3 and 114.8 ms at the median, the active work of the source exceeds 123 ms, the cadence collapses to about one frame in 126 ms, and the terminal delay rises to some 449 ms. Every frame still completes, so the chain degrades gracefully rather than failing, and the whole of the penalty is charged upstream of the first packet. Page residency is therefore not an optimisation of this platform but a precondition of the schedule it is asked to hold, and any archive that omits the field cannot be compared with one that reports it, whatever else the two may share.

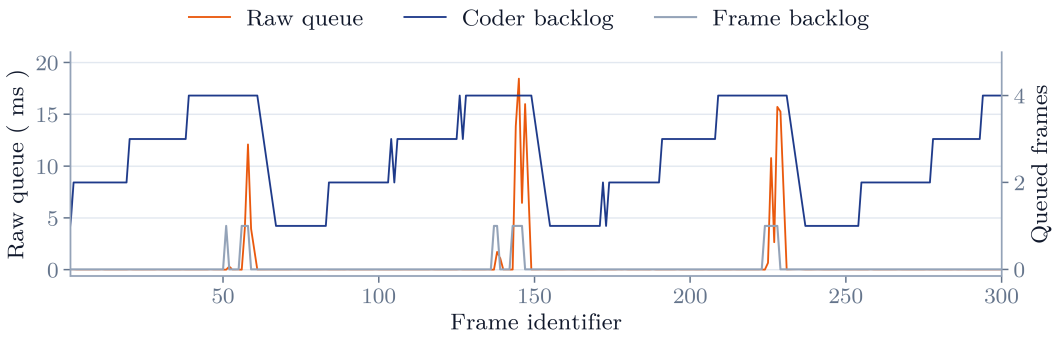
6.6.3 Pressure Imposed Through the Datagram Rate

Halving the capacity of a point datagram leaves the cloud untouched and doubles the population that every unit must handle, which raises the source datagrams to 5 237 270 and is the only condition in the campaign that drives the regulator out of its idle state. The pressure it creates is worth locating before the trace is read, because it arrives from three directions at once. Each frame becomes twice as many packets, so the source spends longer presenting them to its ring, the ingress node performs twice as many acceptance tests for the same cloud, and the compression

unit waits on a frame that now completes later even though its own arithmetic has not changed. None of those three is an overload on its own, and the controller is deliberately insensitive to all of them individually, since it observes the smoothed service interval of the unit it protects rather than any count of packets upstream. What it does observe is the consequence, which is a frame arriving late enough for the budget to be exceeded, and the condition below was chosen precisely because it produces that consequence without altering a single geometric quantity in the stream. Figure 6.16 follows that loop through the sequence.



(a) Control decisions



(b) Queues behind them

Figure 6.16. Control behaviour based on the datagram rate

The trace contains three complete cycles and they are remarkably regular. The smoothed occupancy climbs past the assertion level of 0.75 and peaks at 0.955, the **Temporal** loop raises the factor to two at frames 57, 144, and 226, and the occupancy collapses within a dozen frames of each transition because the source is then offering half as many frames. The stricter recovery predicate holds the factor at two for about twenty-five frames before releasing it at frames 83, 171, and 253, after which the pressure rebuilds and the cycle repeats. Beside it stand the two quantities that corroborate the decision. The queue of raw frames waiting at the compression unit climbs to between 4 and 18 ms in the three or four frames around each transition

and stays below a millisecond everywhere else, while the coder **backlog** oscillates between one and four pictures with its maxima aligned to the same instants. The loop therefore acts on a condition that is visible in three independent places and not on a single noisy indicator, which is exactly the property the regulator was designed around, and the three indicators agree on every cycle.

6.6.4 Admission Against the Final Boundary

The same datagram rate was repeated with the regulator disabled, and the pair separates two effects a single trace would confuse. With adaptation active the source admits 264 of the 300 frames, all of which reach the hub on the reconstruction route, while the exit boundary hands 240 to the terminal. With adaptation disabled the source admits all 300, which traverse the chain intact, and the exit boundary forwards 299. In both cases the loss is confined to the last hop, so no frame disappears silently at an intermediate unit. The pressure behind it is visible at the source, where packets presented again to the local ring rise from 15.4 million in the baseline to 47.1 million with adaptation and 68.0 million without it. The unregulated condition therefore delivers more frames at the cost of forty-four per cent more transmit pressure and a terminal median of 225.984 ms against 281.801 ms, the latter inflated because a quarter of its steady window runs at half the nominal cadence. The pair also keeps admission and delivery distinct, with the former recording what enters the route and the latter what survives its final boundary under stress. Figure 6.17 follows complete frames across the nine boundaries under both stressed conditions and the baseline, all three staying flat until one drops at the exit.

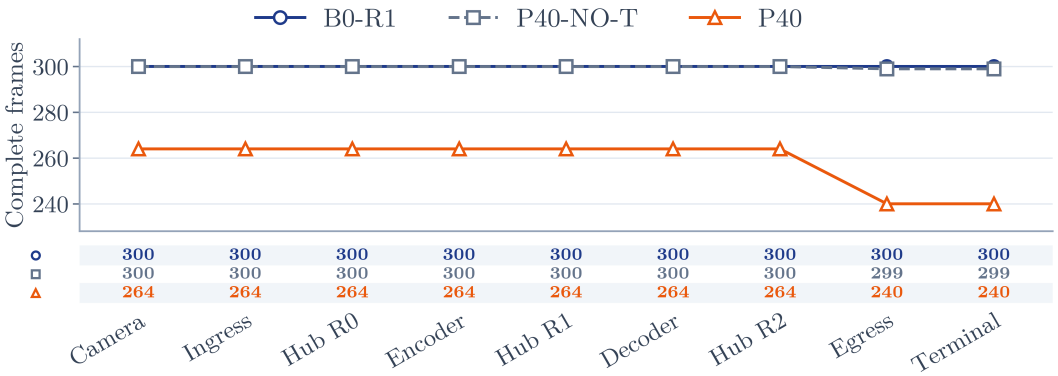


Figure 6.17. Frames complete at every boundary

The reading that survives the pair is narrow. Thinning the source reduces the stress the chain absorbs and does not remove the boundary at which it becomes visible, and that boundary lies between the exit unit and the terminal rather than in the

geometry, the coder, or the reconstruction stage. Any remedy aimed at completeness must therefore address the final handoff as well as source admission.

6.7 The Cost of an Interaction

Every quantity reported so far travels with the media. The last one travels against it, and it is the only measurement in the campaign that closes a loop rather than crossing a chain. The first baseline repetition carries 34 viewing commands issued from the browser while the stream was running, all 34 reappear in the returning state that the source and the reconstruction unit report, and 32 of them acquire a drawing acknowledgement before the session closes. The two remaining repetitions carry nine commands and none at all, and that asymmetry was preserved rather than normalised, for the reason the campaign gave when it declined to average the three runs. It also bounds what this section can claim, since thirty-four samples describe the shape of the loop and not the distribution of a population. A command is counted as matched only when the stream confirms it, which the terminal establishes by re-presenting an unresolved request every three completions until the returning state carries the values it asked for. That rule removes the ambiguity a lost request would otherwise introduce, since a command that never arrives is never counted rather than being recorded with an arbitrary delay, and it also explains why the four frontiers below do not all carry the same number of samples. Table 6.6 reports the four frontiers that the campaign keeps distinct from one another.

Frontier measured	Median	95th value	Where it terminates
Command reference	13.45 ms	37.14 ms	The frame then held
Reverse control path	23.43 ms	33.12 ms	The unit applying it
Applied at the terminal	38.90 ms	58.69 ms	The first frame to obey
Acknowledged by the page	159.0 ms	230.6 ms	A drawing call returning

Table 6.6. Frontiers of a viewing command

The native part of the loop is fast and its budget is easy to account for. A command reaches the reconstruction unit and is applied there within 23.43 ms at the median, which is less than one frame interval, and the first frame that carries the requested stance reaches the terminal 38.90 ms after the command was issued, or slightly more than one interval. Since the **Pose** path does not traverse the compression unit and forces no reprojection, an orientation change costs the chain one frame of latency and no additional work, which is the property the reassignment of the two reverse loops was meant to obtain. The browser frontier is four times larger and belongs to a different category. It includes the return of the shared memory

publication, the transfer over the page connection, and the completion of a drawing call inside a renderer this work neither controls nor instruments, and it is reported at the granularity of a millisecond because that is the resolution the page can offer. It is quoted here as the order of magnitude a viewer experiences and never as a measurement of the platform, and the two must not be added, since the second already contains the first of them by construction.

6.8 What the Measurements Establish

The archive answers the four questions of Chapter 1 unevenly, which is the most useful outcome such a campaign can have. The first question, whether a geometric frontier admits a progressive evaluation in transit, is answered affirmatively and with the chapter's strongest evidence, since the population, the coded stripe, and the rebuilt cloud are indistinguishable under the two placements while the two geometric fields appear at whichever unit computes them. The second, whether moving that frontier relieves the consumer, is answered affirmatively for that unit and conditionally for the viewer. The share of the budget consumed at the compression unit grows by a factor near two and a half when the offload is withdrawn, in two repetitions and again under capture, whereas the terminal delay rises decisively in one repetition and stays inside the baseline envelope in the other. The third, whether the graph sustains the nominal rate and surfaces overload as adaptation rather than as silent loss, is answered affirmatively within the bounds explored, since twelve of the fifteen executions hold the source schedule, every loss observed is localised at a single hop and counted there, and the one condition that saturates the chain drives a control loop that behaves as specified on three indicators. The fourth, whether the decoded stream retains its fidelity, is answered affirmatively at every level the campaign measures, with the sole reservation that an extreme-value distance is governed by single samples and is reported beside the averaged quantities rather than in place of them. Two results belong to no single question. The comparison with the application-level pipeline gives a reduction of 35.99 per cent at the shared frontier and belongs to the redesign entire. The sensitivity sweep shows that the residency of the source material dominates every packetisation parameter by an order of magnitude on this host, which is a property of the host rather than of the placement. Three thresholds governed every verdict above, and a reader who disagrees with one of them can recompute the chapter from the archive. A runtime difference was called real only where it exceeded the band of 2.026 ms the unchanged configuration spans over its steady window, which is why one ablated repetition supports a terminal claim and the other does not. A change of load was

called material only where the smoothed occupancy moved by more than the six thousandths separating the idle maxima of the three baseline repetitions, against which an increase from 0.163 to 0.419 is beyond argument. A fidelity difference was called absent only where the paired change stayed inside the precision at which the indicator is recorded, which held for six of the seven quantities and failed for the seventh as an extreme-value statistic fails. None was chosen after the measurements were seen. Four statements carry the weight of the chapter and each can be falsified by a measurement the archive already supports. The first is that the redistributed chain reaches a complete frame at the terminal in 216.104 ms where the pipeline it departs from needs 337.599 ms, and it would fall if a repetition of either system on this host returned a median outside the bands both were measured with. The second is that resolving the frontier in transit leaves the compression unit at 17.577 ms of active work and an occupancy of 0.163, against 46.821 and 25.601 ms at occupancies of 0.419 and 0.386 when the same arithmetic returns to it, and it would not survive a third ablated repetition landing at the baseline rather than above it. The third is that the final population is identical while the mean point distance is effectively unchanged under both placements, exactly in the first case and within a hundredth of a millimetre in the second, and it would be refuted by a single frame of a capture-enabled pair disagreeing on its admitted count. The fourth is that the only loss the campaign observes anywhere sits at the boundary between the exit unit and the terminal, appears solely under a doubled datagram rate, and is counted where it happens, and it would fall if an intermediate placement were ever found to discard frames it had accepted. Stated in that form the results are neither a claim that in-network computation is generally advantageous nor a measurement of one optimisation, but a description of what one dependency-driven relocation does to one real-time chain on one machine. Chapter 7 weighs these outcomes against the contributions claimed at the outset and sets out, in the same order, what the present evidence does not reach and what a further campaign would have to measure.

Chapter 7

Conclusions

7.1 What the Redistribution Achieved

This thesis asked whether a volumetric chain that treats the forwarding path as a passive carrier can be rearranged so that part of its geometry is resolved while the packets travel, and whether the restructuring pays for itself. The campaign answers affirmatively in three respects and neutrally in a fourth. The first is structural. The four coarse blocks of the arrangement this work departs from become seven units that the graph addresses individually, each exporting its own per-frame record, and the two reverse loops that the original design bound to one application boundary are reassigned to the units that need them, the **Temporal** factor to the compression unit whose load justifies it and the **Pose** command to the reconstruction unit that applies it without a reprojection. What makes the arrangement more than a relabelling is the criterion behind it, since a task is moved into the path only when it can be advanced packet by packet and completed when the last sample of a frame arrives, which is what the ingress node does with the **centroid**, the extrema, and the **radius** defined about it. The second is the delay. At the one frontier both systems express, a complete frame stands ready at the terminal after 216.104 ms where the former architecture needed 337.599 ms, which is a reduction of 35.99 per cent, with the three repetitions placing it individually at 36.24, 35.72, and 35.99 per cent. That figure covers the redesign in its entirety and this host alone, and it is quoted once with the caution that accompanies it. The third is the margin. With nothing free to vary but the pair of switches that settle the site of that frontier, the compression unit spends 17.577 ms on a frame and fills 0.163 of its budget while the path prepares the descriptor it reads, and it spends 46.821 and 25.601 ms while filling 0.419 and 0.386 of the same budget once that arithmetic is handed back. What the **offload** buys is therefore recovered **headroom** at the unit that has the least of it, confirmed by both repetitions and once more under instrumentation. The fourth had to remain

unchanged and did. The reconstructed output shows no material variation when the frontier moves, since the two capture-enabled executions admit the same population in all three hundred frames, the averaged indicators of the coded stripe first change at the fourth significant figure, and the mean distance between a rebuilt cloud and its source moves from 6.3873 to 6.3965 mm, which is half of one per cent of the 1.820 mm pitch at which the source sequence is voxelised.

7.2 Where the Evidence Stops

Every result above comes from one host under the reported conditions, and the machine is the first boundary. Four cores and their eight hardware threads are spoken for entirely, each coder child sits on a sibling of a polling loop, and a single graphics generation answers to both accelerated units, so the assignment of Chapter 5 fits that topology rather than prescribing one. The generated device code names one compute capability as well, so an archive produced elsewhere stays internally valid and externally incomparable. The second is the shape of the experiment. One sequence of three hundred frames, one coder configuration, and one stream were measured, so nothing here speaks to concurrent sessions, another subject, or a non-local link. The service representation is modelled on the standard header rather than compliant with it, and the transport assumptions hold inside a controlled topology. The third is the asymmetry the previous chapter reported and left open. The direction of the effect at the terminal is reproducible and its magnitude is not, since the ablated pair sits 11.81 and 0.15 ms above the baseline and the smaller of the two falls inside the interval the unchanged condition itself spans. A pipelined chain whose slowest stage sets the pace admits that outcome, which is why what a unit is charged for and what a viewer ends up waiting for are reported as two separate findings. The fourth concerns what the fidelity statement carries. It is derived once the stream has closed, from material held back while it ran, so it owes nothing to the load of the machine, yet it rests on the procedure the assessment declares and on the device code beneath it. The extreme-value distance is the one indicator that moves, by about one per cent, for a structural reason. The interaction figures draw on thirty-four commands issued during one repetition, which outline the behaviour of the loop rather than the spread of a population.

7.3 What Should Be Attempted Next

Four directions follow from those boundaries, and the first two concern the loop this work introduced. The constants of the regulator were fixed by design and never swept. A study that varies the two occupancy thresholds, the two streak lengths, and

the retry interval of Chapter 4 would map the frontier between a loop that reacts too readily and one that admits an overload before acting. The natural continuation is to abandon thresholds altogether, since the present controller compares a smoothed service interval against two fixed fractions of the budget and moves the factor by one step, which is robust and coarse, whereas a controller that estimates that quantity and derives the factor from it would place the source rate where the measurement sets it rather than at the nearer of two positions. The second direction is the frame-global boundary. Projection and visibility are resolved over a complete cloud today and the reconstruction waits for a complete picture, so the chain retains one synchronous point that packet-progressive work cannot cross. **Independent HEVC slice segments** offer one candidate mechanism for moving it, provided the order of the investigation is respected [61]. A coder environment that genuinely exposes independent slice boundaries must be built first, the partitions and the rate they cost measured second, and only if the latency of the first slice is an explicit objective does it become reasonable to redesign the attribution of the container units and the consumption of the reconstruction unit around them. Presented in any other order, a coder flag is easily mistaken for packet-wise projection. The third direction is the terminal. It is the boundary at which the only loss of the campaign appears and the browser frontier it publishes to is four times the native one, so a temporal control belongs there as well as at the source. Two forms deserve comparison, a manual selection of the delivered quality of the kind modern players expose, and a dynamic one driven by the publication frontier the terminal already measures, either of which would let a viewer trade population for pace without involving the compression unit. The fourth is deployment. Repeating the campaign on hosts with other core counts and topologies, across physically separated machines, and with an explicit stream identity admitting concurrent sessions would establish how much of the operating point belongs to the architecture and how much to this laboratory. Utilisation, memory bandwidth, and energy would complete the picture, and a second dataset of another density and motion would test the packetisation and the coder parameters that this work fixed once. Taken together the four describe a programme that begins from a chain which is complete and measured rather than from one still to be built, which is the position this thesis set out to reach.

Bibliography

[1] Harald T. Alvestrand. *Overview: Real-Time Protocols for Browser-Based Applications*. RFC 8825. Internet Engineering Task Force, 2021.

[2] Tom Barbette, Cyril Soldani, and Laurent Mathy. "Fast Userspace Packet Processing". In: *2015 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. IEEE, 2015, pp. 5–16.

[3] Paul J. Besl and Neil D. McKay. "A Method for Registration of 3-D Shapes". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14.2 (1992), pp. 239–256.

[4] Pat Bosshart et al. "Forwarding Metamorphosis: Fast Programmable Match-Action Processing in Hardware for SDN". In: *Proceedings of the ACM SIGCOMM 2013 Conference*. ACM, 2013, pp. 99–110.

[5] Pat Bosshart et al. "P4: Programming Protocol-Independent Packet Processors". In: *ACM SIGCOMM Computer Communication Review* 44.3 (2014), pp. 87–95.

[6] Yang Cai et al. "Compute- and Data-Intensive Networks: The Key to the Metaverse". In: *2022 1st International Conference on 6G Networking (6GNet)*. IEEE, 2022, pp. 1–8.

[7] Yang Cai et al. "Joint Compute-Caching-Communication Control for Online Data-Intensive Service Delivery". In: *IEEE Transactions on Mobile Computing* 23.5 (2024), pp. 4617–4633.

[8] Edwin E. Catmull. "A Subdivision Algorithm for Computer Display of Curved Surfaces". PhD thesis. University of Utah, 1974.

[9] Leonardo Chiarparin. *Enhancing Real-Time Point-Cloud Streaming through In-Network Computing — Implementation Sources and Platform Specification*. Software repository. 2026. URL: `https://github.com/Leonardo-Chiarparin/Thesis`.

[10] Brian Curless and Marc Levoy. "A Volumetric Method for Building Complex Models from Range Images". In: *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*. ACM, 1996, pp. 303–312.

[11] Eugene d'Eon et al. *8i Voxelized Full Bodies — A Voxelized Point Cloud Dataset*. Tech. rep. WG11M40059/WG1M74006. ISO/IEC JTC1/SC29 Joint WG11/WG1 (MPEG/JPEG), 2017.

[12] Docker Inc. *Docker Documentation: Resource Constraints*. 2026. URL: `https://docs.docker.com/engine/containers/resource_constraints/`.

[13] DPDK Project. *Data Plane Development Kit — Programmer's Guide and API Documentation, Release 22.11.4 LTS*. URL: `https://doc.dpdk.org/guides-22.11/`.

[14] Wesley M. Eddy. *Transmission Control Protocol (TCP)*. RFC 9293. Internet Engineering Task Force, 2022.

[15] Lars Eggert, Godred Fairhurst, and Greg Shepherd. *UDP Usage Guidelines*. RFC 8085. Internet Engineering Task Force, 2017.

[16] Mohammed S. Elbamby et al. "Toward Low-Latency and Ultra-Reliable Virtual Reality". In: *IEEE Network* 32.2 (2018), pp. 78–84.

[17] Ian Fette and Alexey Melnikov. *The WebSocket Protocol*. RFC 6455. Internet Engineering Task Force, 2011.

[18] FFmpeg Project. *FFmpeg Documentation, Release 4.4*. URL: `https://ffmpeg.org/documentation.html`.

[19] Sebastian Gallenmüller et al. "Comparison of Frameworks for High-Performance Packet IO". In: *Proceedings of the ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2015, pp. 29–38. DOI: `10.1109/ANCS.2015.7110118`.

[20] Rafael C. Gonzalez and Richard E. Woods. *Digital Image Processing*. 4th ed. Pearson, 2018.

[21] Danillo Graziosi et al. "An Overview of Ongoing Point Cloud Compression Standardization Activities: Video-Based (V-PCC) and Geometry-Based (G-PCC)". In: *APSIPA Transactions on Signal and Information Processing* 9 (2020), e13.

[22] Joel Halpern and Carlos Pignataro. *Service Function Chaining (SFC) Architecture*. RFC 7665. Internet Engineering Task Force, 2015.

[23] Bo Han, Yu Liu, and Feng Qian. "ViVo: Visibility-Aware Mobile Volumetric Video Streaming". In: *Proceedings of the 26th Annual International Conference on Mobile Computing and Networking (MobiCom)*. ACM, 2020, pp. 1–13.

[24] Toke Høiland-Jørgensen et al. "The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel". In: *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT)*. ACM, 2018, pp. 54–66.

[25] Quan Huynh-Thu and Mohammed Ghanbari. "Scope of Validity of PSNR in Image/Video Quality Assessment". In: *Electronics Letters* 44.13 (2008), pp. 800–801. DOI: 10.1049/el:20080522.

[26] ISO/IEC. *ISO/IEC 13818-1: Information Technology — Generic Coding of Moving Pictures and Associated Audio Information — Part 1: Systems*. International Standard. 2023.

[27] ISO/IEC. *ISO/IEC 23090-9: Information Technology — Coded Representation of Immersive Media — Part 9: Geometry-Based Point Cloud Compression*. International Standard. 2023.

[28] ISO/IEC. *ISO/IEC 23009-1: Information Technology — Dynamic Adaptive Streaming over HTTP (DASH) — Part 1: Media Presentation Description and Segment Formats*. International Standard. 2026.

[29] ISO/IEC. *ISO/IEC 23090-5: Information Technology — Coded Representation of Immersive Media — Part 5: Visual Volumetric Video-Based Coding (V3C) and Video-Based Point Cloud Compression (V-PCC)*. International Standard. 2026.

[30] ITU-R. *Recommendation ITU-R BT.601-7: Studio Encoding Parameters of Digital Television for Standard 4:3 and Wide-Screen 16:9 Aspect Ratios*. International Telecommunication Union, Radiocommunication Sector. 2011.

[31] Jack Jansen et al. "A Pipeline for Multiparty Volumetric Video Conferencing: Transmission of Point Clouds over Low Latency DASH". In: *Proceedings of the 11th ACM Multimedia Systems Conference (MMSys)*. ACM, 2020, pp. 341–344.

[32] Xin Jin et al. "NetCache: Balancing Key-Value Stores with Fast In-Network Caching". In: *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*. ACM, 2017, pp. 121–136.

[33] Xin Jin et al. "NetChain: Scale-Free Sub-RTT Coordination". In: *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2018, pp. 35–49.

[34] Yili Jin et al. “From Capture to Display: A Survey on Volumetric Video”. In: *arXiv preprint arXiv:2309.05658* (2023).

[35] Changhoon Kim et al. “In-band Network Telemetry via Programmable Dataplanes”. In: *ACM SIGCOMM 2015 Conference, Demo Session*. ACM, 2015.

[36] Diego Kreutz et al. “Software-Defined Networking: A Comprehensive Survey”. In: *Proceedings of the IEEE* 103.1 (2015), pp. 14–76.

[37] Ike Kunze et al. *Use Cases for In-Network Computing*. RFC 9817. Computing in the Network Research Group. Internet Research Task Force, 2025.

[38] Maria Giovanna Lacaria. “Point Cloud Coding for Extended Reality Services”. Master’s Degree in Communications Engineering. MA thesis. Sapienza University of Rome, Faculty of Information Engineering, Informatics and Statistics, 2026.

[39] Zhi Liu et al. “Point Cloud Video Streaming: Challenges and Solutions”. In: *IEEE Network* 35.5 (2021), pp. 202–209.

[40] Alessandro Mauro, Antonia M. Tulino, and Jaime Llorca. “End-to-End Orchestration of NextG Media Services over the Distributed Compute Continuum”. In: *IEEE Transactions on Mobile Computing* 25.3 (2026), pp. 3068–3086. DOI: 10.1109/TMC.2025.3613949.

[41] Steven McCanne and Van Jacobson. “The BSD Packet Filter: A New Architecture for User-Level Packet Capture”. In: *Proceedings of the USENIX Winter 1993 Conference*. USENIX Association, 1993, pp. 259–269.

[42] Nick McKeown et al. “OpenFlow: Enabling Innovation in Campus Networks”. In: *ACM SIGCOMM Computer Communication Review* 38.2 (2008), pp. 69–74.

[43] Rufael Mekuria, Kees Blom, and Pablo Cesar. “Design, Implementation, and Evaluation of a Point Cloud Codec for Tele-Immersive Video”. In: *IEEE Transactions on Circuits and Systems for Video Technology* 27.4 (2017), pp. 828–842.

[44] Rui Miao et al. “SilkRoad: Making Stateful Layer-4 Load Balancing Fast and Cheap Using Switching ASICs”. In: *Proceedings of the Conference of the ACM Special Interest Group on Data Communication (SIGCOMM)*. ACM, 2017, pp. 15–28. DOI: 10.1145/3098822.3098824.

[45] Ben Mildenhall et al. “NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis”. In: *Computer Vision – ECCV 2020*. Vol. 12346. Lecture Notes in Computer Science. Springer, 2020, pp. 405–421.

[46] NVIDIA Corporation. *NVIDIA Video Codec SDK*. 2026. URL: https://developer.nvidia.com/video-codec-sdk.

[47] NVIDIA Corporation. *CUDA Toolkit Documentation, Version 12.2*. URL: https://docs.nvidia.com/cuda/archive/12.2.0/.

[48] Quirino Pagliuca et al. "Dual Timescale Orchestration System for Elastic Control of NextG Cloud-Integrated Networks". In: *27th Conference on Innovation in Clouds, Internet and Networks (ICIN)*. IEEE, 2024, pp. 234–241.

[49] Ben Pfaff et al. "The Design and Implementation of Open vSwitch". In: *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2015, pp. 117–130.

[50] Dan R. K. Ports and Jacob Nelson. "When Should the Network Be the Computer?" In: *Proceedings of the Workshop on Hot Topics in Operating Systems (HotOS)*. ACM, 2019, pp. 209–215.

[51] Jon Postel. *User Datagram Protocol*. RFC 768. Internet Engineering Task Force, 1980.

[52] Ricardo L. de Queiroz and Philip A. Chou. "Compression of 3D Point Clouds Using a Region-Adaptive Hierarchical Transform". In: *IEEE Transactions on Image Processing* 25.8 (2016), pp. 3947–3956.

[53] Paul Quinn, Uri Elzur, and Carlos Pignataro. *Network Service Header (NSH)*. RFC 8300. Internet Engineering Task Force, 2018.

[54] Luigi Rizzo. "netmap: A Novel Framework for Fast Packet I/O". In: *2012 USENIX Annual Technical Conference (USENIX ATC)*. USENIX Association, 2012, pp. 101–112.

[55] Rusty Russell. "virtio: Towards a De-Facto Standard for Virtual I/O Devices". In: *ACM SIGOPS Operating Systems Review* 42.5 (2008), pp. 95–103. DOI: 10.1145/1400097.1400108.

[56] Radu Bogdan Rusu et al. "Towards 3D Point Cloud Based Object Maps for Household Environments". In: *Robotics and Autonomous Systems* 56.11 (2008), pp. 927–941.

[57] Amedeo Sapio et al. "Scaling Distributed Machine Learning with In-Network Aggregation". In: *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2021, pp. 785–808.

[58] Henning Schulzrinne et al. *RTP: A Transport Protocol for Real-Time Applications*. RFC 3550. Internet Engineering Task Force, 2003.

[59] Sebastian Schwarz et al. "Emerging MPEG Standards for Point Cloud Compression". In: *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 9.1 (2019), pp. 133–148.

[60] Evangelos Stafidas and Fotis Foukalas. “A Survey on Enabling XR Services in Beyond 5G Mobile Networks”. In: *IEEE Access* 12 (2024), pp. 59170–59197. DOI: 10.1109/ACCESS.2024.3392509.

[61] Gary J. Sullivan et al. “Overview of the High Efficiency Video Coding (HEVC) Standard”. In: *IEEE Transactions on Circuits and Systems for Video Technology* 22.12 (2012), pp. 1649–1668.

[62] Yaping Sun et al. “Communications, Caching, and Computing for Mobile Virtual Reality: Modeling and Tradeoff”. In: *IEEE Transactions on Communications* 67.11 (2019), pp. 7573–7586.

[63] Jianfeng Tan et al. “VIRTIO-USER: A New Versatile Channel for Kernel-Bypass Networks”. In: *Proceedings of the Workshop on Kernel-Bypass Networks (KBNets)*. ACM, 2017. DOI: 10.1145/3098583.3098586.

[64] David L. Tennenhouse and David J. Wetherall. “Towards an Active Network Architecture”. In: *ACM SIGCOMM Computer Communication Review* 26.2 (1996), pp. 5–18.

[65] The Linux Kernel Documentation. *HugeTLB Pages*. 2026. URL: https://docs.kernel.org/admin-guide/mm/hugetlbpage.html.

[66] The Linux Kernel Documentation. *The Kernel’s Command-Line Parameters (isolcpus)*. 2026. URL: https://docs.kernel.org/admin-guide/kernel-parameters.html.

[67] Dong Tian et al. “Geometric Distortion Metrics for Point Cloud Compression”. In: *2017 IEEE International Conference on Image Processing (ICIP)*. IEEE, 2017, pp. 3460–3464.

[68] Yuta Tokusashi et al. “The Case for In-Network Computing on Demand”. In: *Proceedings of the Fourteenth EuroSys Conference*. ACM, 2019, pp. 1–16.

[69] Irene Viola and Pablo Cesar. “Volumetric Video Streaming: Current Approaches and Implementations”. In: *Immersive Video Technologies*. Ed. by Cagri Ozcinar. Academic Press, 2023. Chap. 15, pp. 425–443. DOI: 10.1016/B978-0-32-391755-1.00021-3.

[70] Zhou Wang et al. “Image Quality Assessment: From Error Visibility to Structural Similarity”. In: *IEEE Transactions on Image Processing* 13.4 (2004), pp. 600–612.

[71] Thomas Wiegand et al. “Overview of the H.264/AVC Video Coding Standard”. In: *IEEE Transactions on Circuits and Systems for Video Technology* 13.7 (2003), pp. 560–576.

[72] Yusheng Xu, Xiaohua Tong, and Uwe Stilla. "Voxel-Based Representation of 3D Point Clouds: Methods, Applications, and Its Potential Use in the Construction Industry". In: *Automation in Construction* 126 (2021), p. 103675.

[73] Anlan Zhang et al. "YuZu: Neural-Enhanced Volumetric Video Streaming". In: *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2022, pp. 137–154.

Acknowledgements

First and foremost, I wish to express my deepest gratitude to my advisor, Prof. Marco Polverini, who placed his trust in me from the very first moment and allowed me to explore subjects of real interest with complete freedom and independence. His guidance and his advice were essential to the making of this thesis. He supported me throughout, letting me put forward proposals of my own and work in an environment I found genuinely rewarding. A heartfelt thank you also goes to all the professors and to the university staff, whose commitment and dedication played a central part in my education, who stood out for their professionalism and their warmth, and who passed on to me knowledge that will stay with me in the years ahead.

My thoughts go, in a special way, to my parents, whose affection and unconditional support, sometimes shown in entirely original ways, have carried me through countless moments we lived together, always encouraging me to chase my dreams and to become the person I want to be, and never asking for anything in return. I love you both, and although to others you might look like two ordinary people, to me you are everything that truly matters, my whole world. I hope I have given you happiness, and that I may go on doing so in the years to come.

I thank Matteo and Simone, my cousins, whom I consider brothers. You were the first with whom I built real bonds, and time has never worn them down. As we grew up, even though life has brought us together less often than in childhood, I keep the memory of the time we spent together with great affection, and those days remain a precious point of reference for me. In every goal I have tried to reach, I have always drawn on your example and your determination, and for that I shall be forever grateful.

I thank my relatives, whose affection has been a source of comfort and a constant guide along my way, like stars that light up the sky on the darkest nights. Each of you has helped make this milestone more meaningful and more special, leaving with your presence an indelible mark on my heart and allowing me to become who I am today. To you goes my deepest gratitude.

I now wish to turn an equally important thought to my group of fellow students, extraordinary people, gifted with great sensitivity and always ready to support

me in the hardest moments. Your company made this master's course not only more manageable but also unique, enriching it with unforgettable experiences. I am convinced that without you this milestone would have been impossible to reach. For precisely that reason, it is with sincere appreciation that I feel bound to express, in the words that follow, everything I feel towards you.

I am grateful to Matteo V. and Stefano, whose presence has been a cornerstone along this path. Your honesty and generosity, together with a constant willingness to help, gave me support I could always count on. The countless adventures we lived through together will stay impressed in my memory for ever, cementing a friendship I regard as a priceless treasure. There are no words enough to thank you for the guidance, the understanding and the inspiration you have given me, but know that I am immensely grateful.

I am grateful to Cosimo and Ettore, two exceptional people with whom I had the privilege of sharing not only lighthearted moments but also a more hidden part of myself. The days we spent together, as far as the courses we each chose allowed, were full of calm, of discussion and of mutual growth, and they marked this path for the better. What truly made our bond special, however, was the ability to enjoy ourselves together, to laugh without reserve and to create memories I shall always carry with me. Your presence has been a refuge of lightness and honesty, and to you goes my most sincere thanks for the support you have known how to give me.

I am grateful to Pierluca and Valerio, travelling companions with whom I shared unforgettable experiences both inside the university and on the train there and back. It was a pleasant surprise to find myself beside you, some from earlier on and some later, at this stage of my life, and to discover in you a constant support and a reassuring presence, on the academic side as much as in the lighter moments, with all our talk about the interests we share. You cannot imagine how much our conversations helped me, because you treated me as a friend from the very start, without ever putting up a barrier. You gave me the kind of confidence usually reserved for someone known for years. I promise you that I shall keep the precious memory of our friendship for ever, and I want to take this occasion to express my deepest appreciation for what you have meant to me.

I am grateful to Federico and Serena, who with their sensitivity and their attention managed to get past my natural reserve and my tendency to keep a certain distance in some situations. Your ability to draw me in and to make me feel part of an open conversation had a profound effect on me. I thank you sincerely for helping me build, not only with you but with others too, genuine friendships and relationships founded on understanding and closeness. For my part, I hope I have managed, if only in small measure, to repay your efforts properly, offering my own contribution

and making myself useful in creating something meaningful for you as well.
Still in the academic setting, I wish to offer my thanks to just as many special people, who accompanied me through the course that came before and with whom I shared moments of calm and of fun, above all outside the walls of the department. I am grateful to Antonio, Cristian, Denis, Diego, Edoardo, Elisa, Elvira, Federica, Francesca, Gianluca, Giorgia, Giulia, Marco, Matteo M., Riccardo, Samuele Di M., Samuele S., Simone B., Simone S. and Valerio B. Without you I could not have made it this far. Some of you enriched my university days by putting your own experience and skill at my disposal to help me over more than one difficulty, others offered me constant support simply by being there, others again made this path unique with stimulating and enjoyable conversation, and many of you made it lighter merely by making me smile. Your closeness, in all its forms, added something to this journey, begun fully three years ago, not only academically but above all personally. Although in some settings I should have liked to share more time with one or another of you, I never stopped feeling supported and appreciated. Your attention, your willingness and your generosity never failed, not even in moments of uncertainty or of difficulty, and the strength you passed on to me has been a constant motivation in facing each day's challenges. I thank you deeply for the readiness and the friendship you have shown me.
I should also like to thank Dario, Federico F., Flavia, Gabriel, Giovanni, Luca, Lorenzo, Manuel, Mattia, Mauro, Raffaele and Samuele P., who, even before this path began, had a significant influence on my growth and on the choices that brought me here. With each of you I shared experiences that left a deep mark and that helped make me the person I am today. Your affection, your friendship and, more simply, having someone with whom to share thoughts and moments, were essential to me. We lived through many experiences together, some carefree and some more intense, and I shall always carry them with me. I am sincerely sorry that I have not managed to keep in touch more often, but every time I hear from you, even after long stretches, I feel an immense joy in rediscovering the value of our bond. I am deeply grateful for everything you have meant in my life, and I shall do everything in my power to make up for the time spent apart. My greatest wish is that you may go on being a meaningful part of my future, just as you have been of my past. With gratitude, I thank you from the bottom of my heart.
Finally, I wish to devote a special thought to those who have not been named explicitly but who have played an important part in my life. Thanks also go to the students who have chosen to take this path, whether at bachelor's or at master's level, in whom I see a promising future full of possibility, and to the friends of the family, some of whom have been with me since I was in the cradle. To Lino, Jenny,

Alessandro, Angelo, Chiara, Claudia B., Claudia S., Emanuele, Francesca, Francesco, Giulia S., Giuseppe R., Giuseppe S., Maria, Rita, Sara, Sofia and Bruno, I want to express all my gratitude. Thank you for everything you have done for me, for your support, your presence and the warmth you have always known how to offer. You have been a fundamental part of my personal growth. I carry you always in my heart, and know that I wish you all the very best.
To each of you I offer my most sincere wishes, that you may achieve every goal and meet with success the challenges of whatever path you choose to take. I shall always remain at your disposal, ready to offer you the support you have always given me as a gift. I do hope I have not gone on too long, but, given the occasion, I could never have given up the chance to express what I feel towards you all. Thank you, endlessly, for everything.

Desidero innanzitutto esprimere la mia più profonda gratitudine al mio relatore, il prof. Marco Polverini, che fin dal primo momento ha riposto fiducia in me, permettendomi di approfondire tematiche di grande interesse con piena libertà e autonomia. La sua guida e i suoi consigli sono stati essenziali per la realizzazione di questo lavoro di tesi. Mi ha sempre sostenuto, lasciandomi avanzare proposte e collaborare in un ambiente decisamente appagante. Un sentito ringraziamento va inoltre a tutti i professori e al personale universitario, il cui impegno e la cui dedizione hanno avuto un ruolo centrale nella mia formazione, distinguendosi per professionalità e cordialità, e trasmettendomi conoscenze che accompagneranno il mio percorso futuro.
Rivolgo un pensiero speciale ai miei genitori, che con il loro affetto e il loro sostegno incondizionato, talvolta manifestati in modi del tutto originali, hanno condiviso con me infinite emozioni, incoraggiandomi sempre a inseguire i miei sogni e a diventare la persona che desidero essere, senza mai chiedere nulla in cambio. Vi voglio bene e, anche se per gli altri potreste sembrare solo due persone qualunque, per me siete tutto ciò che conta davvero, il mio mondo. Spero di avervi regalato felicità e di poter continuare a farlo negli anni a venire.
Ringrazio Matteo e Simone, i miei cugini, che considero come fratelli. Siete stati i primi con cui ho costruito legami autentici, che il tempo non ha mai scalfito. Crescendo, anche se la vita ci ha portato a vederci meno frequentemente rispetto all'infanzia, custodisco con affetto i ricordi delle giornate trascorse insieme, che rappresentano per me un riferimento prezioso. In ogni traguardo che ho cercato di raggiungere, mi sono sempre ispirato al vostro esempio e alla vostra determinazione, e per questo vi sarò eternamente riconoscente.
Ringrazio i miei parenti, il cui affetto è stato una fonte di conforto e una guida costante durante il mio cammino, come stelle che illuminano il cielo nelle notti più buie. Ognuno di voi ha contribuito a rendere questo traguardo ancora più

significativo e speciale, lasciando con la vostra presenza un'impronta indelebile nel mio cuore e permettendomi di essere quello che sono oggi. A voi va la mia più profonda gratitudine.
Desidero ora rivolgere un pensiero altrettanto importante al mio gruppo di colleghi universitari, persone straordinarie, dotate di grande sensibilità e sempre pronte ad offrirmi sostegno nei momenti più difficili. La vostra compagnia ha reso questo percorso magistrale non solo più gestibile, ma anche unico, impreziosendolo con esperienze indimenticabili. Sono convinto che in vostra assenza questo traguardo sarebbe stato impossibile da raggiungere. Proprio per questo, è con sincera riconoscenza che sento il dovere di esprimere, attraverso le seguenti parole, tutto il mio apprezzamento nei vostri confronti.
Sono grato a Matteo V. e Stefano, la cui presenza è stata un pilastro fondamentale lungo questo percorso. La vostra autenticità e generosità, unite a una costante disponibilità, mi hanno offerto un sostegno su cui ho sempre potuto contare. Le innumerevoli avventure vissute insieme resteranno per sempre impresse nella mia memoria, consolidando un'amicizia che considero come un tesoro inestimabile. Non esistono parole sufficienti per ringraziarvi del supporto, della comprensione e dell'ispirazione che mi avete dato, ma sappiate che vi sono immensamente riconoscente.
Sono grato a Cosimo ed Ettore, due ragazzi eccezionali con cui ho avuto il privilegio di condividere non solo momenti di spensieratezza, ma anche una parte più nascosta di me stesso. Le giornate trascorse insieme, per quanto le facoltà che abbiamo scelto ce lo abbiano concesso, sono state colme di serenità, confronti e crescita reciproca, e hanno segnato positivamente questo percorso. Tuttavia, ciò che ha reso davvero speciale il nostro legame è stata la capacità di divertirci insieme, di ridere senza riserve e di creare ricordi che porterò sempre con me. La vostra presenza è stata un rifugio di leggerezza e genuinità, e a voi va il mio più sincero ringraziamento per il sostegno che avete saputo donarmi.
Sono grato a Pierluca e Valerio, compagni di viaggio con cui ho condiviso esperienze indimenticabili sia dentro l'università sia durante l'andata e il ritorno sul treno. È stata per me una piacevole sorpresa trovarmi al vostro fianco, chi già da prima e chi dopo, in questa fase della mia vita, scoprendo in voi un supporto costante e una presenza rassicurante, tanto sul piano accademico quanto in quello dello svago, con tutti i discorsi sulle passioni che condividiamo. Non potete neanche immaginare quanto le nostre chiacchierate mi abbiano aiutato, poiché mi avete trattato fin da subito come un amico, senza mostrare alcuna barriera. Mi avete dato quella confidenza che solitamente si riserva a una persona che si conosce da tempo. Vi prometto che conserverò per sempre il ricordo prezioso della nostra amicizia e desidero cogliere questa occasione per manifestare la mia più profonda riconoscenza per quello che

avete rappresentato per me.
Sono grato a Federico e Serena, che con la loro sensibilità e attenzione sono riusciti a superare la mia naturale riservatezza e la mia propensione a mantenere una certa distanza in alcune situazioni. La vostra capacità di coinvolgermi e di farmi sentire parte di un dialogo aperto ha avuto per me un impatto profondo. Vi ringrazio di cuore per avermi aiutato a costruire, non solo con voi ma anche con gli altri, amicizie autentiche e rapporti basati su comprensione e vicinanza. Dal canto mio, mi auguro di essere riuscito, anche se in minima parte, a contraccambiare adeguatamente il vostro impegno, offrendo il mio contributo e rendendomi utile per creare qualcosa di significativo anche per voi.
Sempre in ambito accademico, desidero porgere i miei ringraziamenti ad altrettante persone speciali, che mi hanno accompagnato durante il percorso precedente e con cui ho condiviso momenti di serenità e di divertimento, soprattutto al di fuori delle mura della sede. Sono grato ad Antonio, Cristian, Denis, Diego, Edoardo, Elisa, Elvira, Federica, Francesca, Gianluca, Giorgia, Giulia, Marco, Matteo M., Riccardo, Samuele Di M., Samuele S., Simone B., Simone S. e Valerio B. Senza di voi non sarei potuto arrivare fino a qui. Alcuni hanno arricchito le mie giornate universitarie mettendo a disposizione la propria esperienza e competenza per aiutarmi a superare diverse difficoltà, altri mi hanno offerto un sostegno costante con la loro sola presenza, altri ancora hanno reso unico questo percorso con conversazioni stimolanti e piacevoli, e molti di voi lo hanno reso più leggero semplicemente facendomi sorridere. La vostra vicinanza, in tutte le sue forme, ha dato un valore aggiunto a questo viaggio, iniziato ben tre anni fa, non solo dal punto di vista accademico ma soprattutto personale. Sebbene in alcuni contesti mi sarebbe piaciuto condividere più tempo con qualcuno di voi, non ho mai smesso di sentirmi sostenuto e apprezzato. La vostra attenzione, disponibilità e generosità non sono mai venute meno, nemmeno nei momenti di incertezza o di difficoltà, e la forza che mi avete trasmesso è stata una motivazione costante per affrontare le sfide quotidiane. Vi ringrazio profondamente per la disponibilità e l'amicizia che mi avete dimostrato.
Vorrei inoltre ringraziare Dario, Federico F., Flavia, Gabriel, Giovanni, Luca, Lorenzo, Manuel, Mattia, Mauro, Raffaele e Samuele P., che, ancora prima di iniziare questo cammino, avete avuto un'influenza significativa nella mia crescita e nelle scelte che mi hanno condotto fino a qui. Con ciascuno di voi ho condiviso esperienze che hanno lasciato un segno profondo e che hanno contribuito a rendermi la persona che sono oggi. Il vostro affetto, la vostra amicizia e, più semplicemente, la possibilità di avere qualcuno con cui condividere pensieri e momenti, sono stati per me fondamentali. Abbiamo vissuto tanti momenti insieme, alcuni spensierati, altri più intensi, che porterò sempre con me. Mi dispiace sinceramente se non sono riuscito a mantenere

un contatto frequente, ma ogni volta che vi sento, anche dopo lunghi periodi, provo una gioia immensa nel riscoprire il valore del nostro legame. Sono profondamente grato per tutto ciò che avete rappresentato nella mia vita e farò tutto quello che è nelle mie possibilità per recuperare il tempo trascorso lontani. Il mio desiderio più grande è che possiate continuare a essere una parte significativa del mio futuro, così come lo siete stati nel passato. Con riconoscenza, vi ringrazio dal profondo del mio cuore.

Infine, desidero dedicare un pensiero speciale a coloro che non sono stati nominati esplicitamente ma che hanno avuto un ruolo importante nella mia vita. Un ringraziamento va anche ai ragazzi che hanno deciso di intraprendere questo percorso, triennale o magistrale che sia, nei quali vedo un futuro promettente e pieno di possibilità, e agli amici di famiglia, alcuni dei quali mi accompagnano sin da quando ero in fasce. A Lino, Jenny, Alessandro, Angelo, Chiara, Claudia B., Claudia S., Emanuele, Francesca, Francesco, Giulia S., Giuseppe R., Giuseppe S., Maria, Rita, Sara, Sofia e Bruno, voglio esprimere tutta la mia gratitudine. Grazie per tutto ciò che avete fatto per me, per il vostro sostegno, la vostra presenza e il calore che avete sempre saputo offrirmi. Siete stati una parte fondamentale del mio percorso di crescita personale. Vi porto sempre nel cuore e sappiate che vi voglio un mondo di bene.

A ciascuno di voi rivolgo i miei più sinceri auguri affinché possiate realizzare ogni obiettivo e affrontare con successo le sfide del cammino che sceglierete di intraprendere. Rimarrò sempre a vostra disposizione, pronto ad offrirvi il sostegno di cui mi avete sempre fatto dono. Spero proprio di non essermi dilungato, ma, data l'occasione, non avrei mai potuto rinunciare ad esprimere i sentimenti che nutro nei vostri confronti. Grazie infinite per tutto.